Annex XI - WR Pointer & Augmented Overlay

Device-Adaptive Invocation and Presentation Layer for Governed Hybrid AI

Version 1.3 — 26 July 2026 (v1.0: 19 July 2026) · Part of the Optirando™ Canon · Status: Defensive Publication / Prior-Art Disclosure

XI.0 Status of this Annex

This annex is published as a defensive disclosure. It describes the WR Pointer™ and the Augmented Overlay as designed components of the Optirando™ platform. Nothing in this annex asserts that any described protection is absolute; the platform is under active development, and all security statements are to be read under the Honest Boundary discipline established in Annex X: the system states what it enforces, what it observes, and what it cannot yet guarantee.

Amendment and versioning discipline (normative).

This annex is intended to evolve as the architecture, terminology, threat model, and implementation understanding develop. A later version may add, revise, replace, consolidate, split, relocate, renumber, supersede, or remove sections, subsections, definitions, embodiments, device classes, action categories, interaction patterns, examples, and implementation profiles where necessary to improve technical accuracy, consistency, clarity, or completeness.

No material change may be silent. Each published version must identify its substantive changes in the Version History and must preserve access to previously published versions. A later version governs the current specification, but it does not withdraw, erase, or narrow the technical disclosure contained in an earlier published version.

The normative invariants of §XI.3 may be clarified, reformulated, divided, consolidated, or strengthened. Any amendment that materially reduces, removes, or changes a previously stated protection or architectural guarantee must identify that change explicitly, explain the reason for it, and state the resulting Honest Boundary. Such an amendment does not retroactively remove the earlier disclosure.

Superseded text need not remain reproduced in full inside the current version, provided that the superseding provision is identified and the previous version remains permanently referenceable. Section numbers may be changed, reassigned, or reused as part of a documented restructuring, provided that the Version History or an accompanying concordance identifies the relationship between the previous and current structure.

This amendment discipline applies equally to additions, partial revisions, integrated restatements, and complete new editions of the annex.

Relationship to the Canon. This annex deliberately does not restate mechanisms specified elsewhere. It references:

- Annex III (PoAC™) for capability acquisition, provenance, and admission discipline;
- Annex VII (BEAP™ handshake architecture) for handshake profiles, capability declaration, Merkle manifests, publisher attestation tiers P0–P2, and the grant model;

- Annex VIII (hybrid compute architecture) for the Augmented Overlay's original introduction, Frames, strict rendering passivity, Mini-Apps built from pre-admitted interaction primitives, Editable Regions, and WR Anchor export;
- Annex IX (initiation layer and WR TED) for consent classes C0–C4, the Intent Hash / WYSIWYE discipline, the execution lifecycle Validate → Execute → Witness → Seal → Anchor, verification tiers L/C/X, and the AI-as-Router characterization;
- Annex X (WR Guard™ and capability-governed information flow) for boundary governance, the three governance classes (Public Handshake, Internal Handshake, External Service Admission), Invocation Assurance Classes, the ten graduated egress measures, Boundary Event Records, and enforcement modes.

Where this annex says "by reference," the referenced annex is normative for that mechanism.

XI.1 Purpose and Scope

Modern AI assistance is delivered through chat windows and detached copilots. The user's actual working surface — the screen content in front of them, the document under the cursor, the machine in front of the camera — is disconnected from the assistance layer, and the assistance layer is disconnected from any verifiable authorization or evidence mechanism. Where "AI overlays" exist today, they typically combine three dangerous properties: they read arbitrary screen content, they can act on it, and they carry no cryptographic governance of what may enter or leave the trust boundary.

The WR Pointer and the Augmented Overlay together form the interaction layer of the Optirando platform that closes this gap:

- The WR Pointer is the device-adaptive invocation surface: the single place where the user designates content or objects, bounds context, selects actions, and triggers governed execution.
- The Augmented Overlay is the presentation surface: the single place where the system displays status, offered actions, contextual help, and context-related information.

Both operate strictly inside the governance model of the Canon: AI prepares, the human authorizes, the system records cryptographic proof. This annex specifies the invariants, artifact classes, declaration requirements, and device-class embodiments of this interaction layer, and it defines how the layer remains extensible without weakening its guarantees.

Out of scope: the internal implementation of any particular model runtime, rendering engine, or operating-system integration. These are embodiment details; the annex constrains their behavior, not their construction.

XI.2 Definitions

WR Pointer™ — the invocation surface of the WR system. A capability architecture with invariant semantics (I9) and device-adaptive embodiments (§XI.4). Colloquially "the pointer."

Augmented Overlay — the presentation surface of the WR system, first introduced in Annex VIII as cursor-/UI-bound real-time support on composed content, generalized here to all pointer contexts.

Pointer Action — a discrete, declared unit of functionality invocable through the pointer, defined by a Pointer Action Manifest (§XI.6).

Pointer Action Manifest (PAM) — the signed declarative artifact defining a pointer action: identity, origin, trigger context, inputs, permitted context sources, capability and consent requirements, validation, Finalizer reference, and evidence requirements.

Origin Class — the signed provenance category of a pointer action or overlay content item: user-created, local WCR, organization policy, verified publisher (handshake-declared), or AI suggestion (§XI.3, I4).

Trigger Context — the situation class in which an action is available: global, or context-scoped (e.g. WR Code® scan, BEAP™ message, infographic Frame, document selection, camera designation).

Region Marker — the pointer's explicit context-bounding tool: a user-drawn or user-confirmed selection of a screen region, content element, or camera frame that defines the exact context boundary of an action (§XI.8).

Real-World Pointer — the mobile camera embodiment of the pointer: the camera used as a pointing instrument toward physical objects and scenes (§XI.4.3).

Action Registry — the local, host-maintained set of admitted Pointer Action Manifests, filterable by trigger context and origin class (§XI.7).

Terms not defined here (PoAE™, PoAC™, WR Graph™, WR Guard™, WR Expert™, WR TED, WCR, Finalizer, Intent Hash, consent classes C0–C4, Invocation Assurance Class, Boundary Event Record) carry their Canon definitions by reference. WCR carries its Canon definition by reference; this annex uses the acronym uniformly (earlier drafts intermittently used "WRC" for the same artifact class).

XI.3 Core Invariants (Normative)

The following invariants define the WR Pointer and Augmented Overlay. They hold across all device classes, all embodiments, and all future extensions of this annex. Amendments may add invariants and may strengthen, clarify, reformulate, divide, or consolidate them. Any amendment that materially weakens, removes, or changes an invariant is governed by the amendment discipline of §XI.0: it must be explicitly identified, justified, and accompanied by the resulting Honest Boundary statement. No invariant change may be silent.

I1 — The pointer is a trigger, not authority. The pointer never possesses or confers authority of its own. It is the execution trigger for authority that is established elsewhere: in signed manifests, capability grants, policy, and consent. Echoing the canonical formulation of Annex IX for entry codes: the pointer is an invocation instrument — not authority.

I2 — Duality of channels: the pointer invokes, the overlay presents. The WR Pointer is the sole invocation channel of the interaction layer; the Augmented Overlay is the sole presentation channel. The overlay displays status, offers, help, and information. The overlay offers, never acts: nothing displayed in the overlay carries execution authority until the user invokes it through the pointer under the applicable consent class. The offer-never-action discipline of Annex IX applies to the

overlay verbatim. An overlay without an execution path is structurally devalued as an attack surface: fake dialogs cannot execute because no overlay element can.

I3 — Content is data, never instruction. Visible content — screen content, page content, document text, camera imagery, scanned codes beyond their declared identifier role — can never create, propose, parametrize, or alter the semantics of a pointer action. Pointer actions originate exclusively from the four authoritative sources of I4 (excluding raw AI output, which may only rank and select per I6). Selected or captured content is payload for a chosen action, never that action's definition. This extends the instruction-incapable-extraction principle of Annex IX (Assisted Code Capture) to the entire interaction layer.

I4 — Signed origin classes on every action. Every pointer action and every overlay content item carries a verifiable origin class: (a) user-created, (b) local WCR-derived, (c) organization-declared (signed policy), (d) verified-publisher-declared (handshake manifest), or (e) AI suggestion (a ranking over items of classes a–d, never an origin of new actions). The origin class is displayed unmodifiably wherever the action or content appears, with consistent visual encoding across menu and overlay (§XI.11.3). Local and handshake-associated actions appear in the same menu, distinguished by this encoding — there is no separate "handshake mode"; the trust distinction lives per action, not per mode.

I5 — No hidden path. Everything that crosses the trust boundary in any direction — context leaving the device, publisher content entering, model invocations, adapter deployment, help assets, status telemetry — must be cryptographically and explicitly permitted under the governance classes of Annex X (Public Handshake, Internal Handshake, External Service Admission) and is logged as Boundary Event Records. There is no exempted flow class for the interaction layer. This includes help texts, thumbnails, previews, and any other "convenience" content.

I6 — AI as router. AI participation in the pointer is confined to the router role established in Annex IX: selecting among pre-admitted actions, ranking offers, binding parameters within declared constraints, and choosing declared branches. AI never synthesizes new actions, new execution paths, or new effect classes at runtime.

I7 — Effect-boundary determinism and the mis-specification rule. Determinism is achieved at the effect boundary, not in the reasoning. Every externally consequential effect passes through deterministic validation against the action's schema, the applicable policy, and the WR TED lifecycle (Validate → Execute → Witness → Seal → Anchor, by reference to Annex IX). From this follows a normative admission rule:

> A publisher action whose safety depends on the quality of AI reasoning rather than on deterministic validation at the effect boundary is mis-specified and is not admissible.

I8 — Explicit context bounding. Context acquisition is always explicitly bounded — by the Region Marker, by a designated element, or by a confirmed camera frame — and never implicitly "the whole screen" or "the whole scene." The bounded region is part of the Intent Hash: the user authorizes exactly the bounded context they see (WYSIWYE). Context acquisition destined for any external recipient is an information flow under Annex X and passes through WR Guard.

I9 — Pointer Invariance across device classes. The WR Pointer implements one semantic architecture: designation of a target, establishment of a bounded context, resolution of admitted capabilities, and governed invocation. This architecture, the action registry, manifest format, origin

classes, consent mapping, evidence chain, and invariants I1–I8 are identical on every device class. Device classes never alter Pointer function. They differ solely in which designation instruments and presentation affordances the hardware physically provides, and in which interaction patterns are ergonomically native (§XI.4). Instrument availability is a hardware fact, not a device-class property: any device that possesses an instrument possesses the full corresponding Pointer capability. A desktop with a camera, an imported image, or a live visual feed carries the complete Real-World Pointer. A smartphone rendering a webpage carries the complete digital-content Pointer. No device class receives a weaker governance regime or a reduced Pointer semantics because of its form factor.

XI.4 Device-Class Model (Normative)

XI.4.1 Principle: Pointer Invariance, Designation Instruments, and Source Classes

The WR Pointer is a capability architecture, not a widget, and not a per-device feature set. Its function is invariant (I9). Every embodiment executes the same semantic sequence: designate target → establish bounded context → resolve admitted capabilities → present the adaptive WR menu → invoke under the declared consent, routing, and evidence requirements.

Designation instruments are the physical or logical means by which a user designates a target. They are defined device-independently. The instrument set includes, without limitation:

- cursor (pointer hover, click, selection);
- touch, long-press, drag, and lasso gestures;
- stylus (including hover and pressure states);
- the Region Marker (rectangular or freeform bounding, §XI.8);
- camera capture (a captured image as designation surface);
- live visual feed designation (selection of a recognized candidate within a user-activated camera, video, screen, or application feed);
- imported or received images and documents as designation surfaces;
- gaze rays, hand rays, and spatial gestures;
- voice-confirmed designation of a highlighted candidate;
- **identifier scan** — designation of a declared target through a machine-readable identifier (WR Code®, QR code, barcode, NFC tag, or RFID tag), read by camera or by a dedicated scanner. The identifier is an identifier and invitation — never authority (Annex IX, canonical formulation); where the identifier initiates handshake formation, the entry and verification rules of Annex IX §IX.3 apply unchanged, and where it designates a target within an existing handshake, it is an ordinary designation instrument feeding the semantic sequence of this section;
- **proximity designation** — a target offered as a designation candidate by a declared proximity signal (e.g. a beacon or local discovery announcement) and confirmed by the user. Proximity offers a candidate; it never designates by itself, and it never acquires context without the user's confirming designation (I8);
- **terminal and industrial input surfaces** — keyed or scanned target selection on dedicated field, production, logistics, or kiosk hardware;
- **declared WR Link activation** — activation of a WR Link as designation of the link's declared, hash-bound target context within an existing Public Handshake. A WR Link never

establishes, extends, or upgrades a handshake and is never actionable outside a valid handshake (Annex IX §IX.7 by reference); as a designation instrument it is a selection vector for declared context, never an execution or entry vector.

Source classes describe what the designated target belongs to: webpage, application, document, image, camera, video, screen or application feed, spatial/XR scene, and **identifier-referenced physical targets** — an object, machine, installation, or location designated through a machine-readable identifier rather than through imagery. Source classes are likewise device-independent.

An instrument or source class is available on a device wherever the hardware and the controlled execution surface support it. Neither instruments nor source classes are bound to device classes. All instruments feed the identical semantic sequence above, and all resulting context acquisition remains governed by I8 and Annex X without regard to the instrument used. New instruments and source classes are added additively under §XI.20.

A device class therefore defines only: (a) which designation instruments are physically available, (b) which presentation affordances exist for the Augmented Overlay and the adaptive menu, and (c) which interaction patterns are ergonomically native. It never redefines governance, and it never redefines Pointer function. Any statement of the form "device class X points at Y" in this annex describes typical instrument availability; it is never a restriction of Pointer semantics.

The device-class profiles in §XI.4.2–§XI.4.4 below are **informative**. They describe typical instrument sets, presentation affordances, and ergonomic defaults per device class; they are illustrative, non-exhaustive, and non-restrictive. Absence of an instrument from a profile does not exclude it where the hardware and controlled execution surface support it (I9). Governance and Pointer semantics are defined in §XI.3 and this §XI.4.1 only.

XI.4.2 Desktop Embodiment — the AI Cursor Pointer

On desktop, the pointer is typically embodied as an augmented cursor: closely bound to the object currently hovered or selected, extended with:

- the Region Marker for bounding arbitrary screen areas (e.g. for screenshots and screen-region context, §XI.8);
- an action menu presenting the context-filtered Action Registry, with origin-class encoding per I4;
- inline invocation of actions on selections (text, images, application content, Frames, Editable Regions per Annex VIII);
- initiation and control of WCR recording (§XI.13).

The desktop is typically the primary composition and deep-work surface: full WR Graph context, the strongest local models, multi-application workflows, and rich Overlay presentation space. Where a desktop system provides a camera, receives imported images, or processes video, screen, or application feeds, it carries the complete Real-World Pointer and feed-designation capability of §XI.4.1 — this profile describes the cursor-centric default, not a limitation to rendered digital content.

XI.4.3 Mobile Embodiments

The smartphone typically carries two pointer embodiments, both first-class:

(a) The Real-World Pointer (camera embodiment). The camera acts as the pointing instrument toward the physical world — like a finger with which the user points at something. The Real-World Pointer is not smartphone-specific: it is available on every device whose hardware provides camera capture, imported imagery, or live visual feeds (I9, §XI.4.1); it is described here because the smartphone is its most common carrier. Functions include:

- designation: pointing at an object, machine, component, document, or scene to establish it as the subject of an action;
- descriptive grounding: the pointed-at imagery gives AI assistance a precise visual referent, enabling substantially better description, identification, and support for images and physical objects than text alone;
- WR Code® scanning as the governed entry path (identifier and invitation — not authority; Annex IX by reference);
- evidence capture with explicit frame confirmation (the confirmed frame is the bounded context, I8);
- physical-world entry into field-service and DeepFix flows (Annex IV by reference).

The Real-World Pointer obeys I8 strictly: only the confirmed frame or designated region enters any action's context; continuous ambient capture is not a pointer function.

(b) The On-Screen Pointer. The touch-based embodiment of the digital-content Pointer, semantically identical to the desktop cursor: it designates webpage elements, application controls, menu entries, form fields, text, text regions, images, objects within images, and document entities through touch, long-press, drag, lasso, or renderer-level element selection. Its typical presentation is a floating pointer control with a swipe menu listing the available actions — the context-filtered view of the Action Registry. The menu is the presentation of admitted capabilities for the designated target; it is not itself the Pointer, and it is not a smartphone-specific concept (the desktop action menu of §XI.4.2 is the same mechanism in a different affordance). Properties:

- the swipe menu lists all currently available actions, with origin-class encoding;
- users can add actions to the menu — both global actions and context-scoped actions bound to specific trigger contexts (WR Code scan, BEAP message, Frame, document, camera designation, and future context classes); adding an action is a PoAC admission event (§XI.7);
- selection, long-press, and radial patterns per §XI.17; consequential actions never sit in the first interaction layer;
- acts as the approval surface for cross-device consent (§XI.14).

(c) Ergonomic default (informative). In typical deployments the smartphone serves as capture and consent device: designating, scanning, capturing evidence, issuing short instructions, approving. This is an ergonomic default arising from form factor, not a role restriction: the smartphone carries the full digital-content Pointer of §XI.4.1, including webpage, application, and document designation. Its interaction patterns are designed for its form factor rather than as a miniature desktop cursor.

XI.4.4 Tablet Profile (Informative)

The tablet typically combines both embodiments: on-screen pointer with stylus-aware precision (stylus hover and press as first-class pointer states), plus the camera embodiment. Stylus-designated regions are Region Marker inputs.

XI.4.5 Invocation Modalities (Normative)

Pointing and interacting are decoupled. On every device class, the user can point at something — on the screen or in the real world — and then interact with AI about the designated subject through any of the following modalities:

1. Speech — a spoken instruction concerning the designated content or object;
2. Text — a typed instruction or query;
3. Handshake-based AI automation — invoking a publisher-declared action available for the designated subject (§XI.5, §XI.12);
4. Self-predefined AI automation — invoking a user- or organization-defined action from the Action Registry, including WCR-derived actions (§XI.7, §XI.13).

Modalities are orthogonal to embodiments: the same designated subject (a hovered element, a marked region, a confirmed camera frame) can be addressed through any modality the device supports. Governance is modality-independent:

- Speech and text are user-instruction channels carrying origin class (a). They express intent; they do not create actions. The instruction is mapped, under I6, onto admitted actions of the registry (or answered as pure local/routed reasoning without consequence); free-form instructions never synthesize new execution paths.
- Only the user's own confirmed input constitutes an instruction. Ambient audio and speech contained in captured content are content, not instruction (I3).
- Whichever modality initiates an action, the action's declared consent class, bounded context, routing constraints, and evidence requirements apply unchanged (I8, §XI.10).

XI.4.6 Future Device Classes (Extension Point)

New device classes — XR/spatial displays, vehicles, wearables, dedicated field hardware — are added to this annex additively as new profile subsections of §XI.4, each declaring only: available designation instruments (from §XI.4.1, extended additively where needed), presentation affordances, and native interaction patterns. A new device class never declares new Pointer functions and never modifies Pointer semantics; where a genuinely new instrument or source class is required, it is added to §XI.4.1 under §XI.20 and thereby becomes available to every device class whose hardware supports it. Invariants I1–I9 apply unmodified.

XI.5 Operating Contexts: Local and Handshake-Associated Actions

The pointer exposes one unified surface. The distinction the user experiences is per action, via origin class and color encoding (I4), not via modes.

Local actions operate without any publisher handshake. They may use local files, selections, images, screenshots (Region-Marker-bounded), application content, the WR Graph (resident access for local models per Annex X; capability-gated for every other requester class), and user-approved system capabilities. Reasoning may be routed to a fully local model, a user-selected external model, an enterprise model, deterministic tools, or combinations — under §XI.9. The user and, where applicable, signed organization policy control which data may leave the device and which execution environment is used; WR Guard sits on every external path (I5, I8).

Handshake-associated actions become available after a valid Public Handshake (Annex VII). The publisher declares the action's semantics, schemas, capability requirements, consent class, and permitted consequences in its handshake repository (§XI.12). The publisher binds the effect boundary; the reasoning layer remains user- or policy-selected within declared assurance requirements (§XI.9). Risk classes are handled exactly as everywhere in the WR system: the applicable consent class, assurance class, and evidence tier follow from the declared risk, and the WR principle for AI-assisted automation applies without exception — declared in advance, consent-gated, evidence-producing, no hidden path.

Both kinds of actions coexist in the same menu and the same overlay, always distinguishable, never interchangeable in provenance.

Unmapped Target Fallback (Normative). Within an active handshake-associated context, the user may designate a target for which the publisher's handshake repository declares no context, no wiring, and no action. Such a designation neither fails nor widens the handshake: for exactly this designated target, the Pointer falls back to local operation under the rules of this section.

1. **Local rules only.** The fallback operates as a local action: reasoning is routed under the user's or organization's policy (§XI.9), WR Guard sits on every external path (I5, I8), and the action's context is exactly the designated, bounded content — element, region, or confirmed frame per I8 — and nothing more.
2. **Nothing of the handshake crosses.** The fallback receives no publisher context, no handshake capability, no publisher-shipped adapter, and no login-bound binding. Its output carries no publisher authority and no publisher provenance.
3. **Marked, never blended.** In menu and overlay, fallback capability is presented under origin class (a), (b), or (c) per I4 — never (d) — visually distinct from handshake-associated actions, and never presented as offered by, endorsed by, or associated with the publisher.
4. **No publisher signal.** The unmapped designation is not transmitted to the publisher. Designation and selection telemetry exist only where declared in the handshake repository and consent-covered (I5); an unmapped designation is never such a declared event.
5. **No hidden re-entry.** Fallback results are ordinary local content. Their introduction into any handshake-associated action's context follows that action's declared permitted context sources (§XI.6.2(5)) unchanged; the fallback itself can never invoke a handshake capability (I1, I5).

XI.6 The Pointer Action Manifest (PAM)

XI.6.1 Rationale

WCR recordings are the execution substrate of pointer actions, but execution bodies alone cannot carry declaration, discovery, signing, versioning, and revocation. The PAM is therefore introduced as a distinct declarative artifact class: the manifest is the governed shell; the WCR body (or deterministic tool chain) is what it references.

XI.6.2 Manifest Contents (Normative)

A PAM declares at minimum:

1. **Identity** — stable action identifier, semantic version, author identity, signature;
2. **Origin class** (I4) and, for publisher actions, the handshake and attestation tier (P0–P2, Annex VII);
3. **Trigger context(s)** — global and/or specific context classes (§XI.7);
4. **Inputs** — typed input schema;
5. **Permitted context sources** — an explicit allowlist (selection, region, frame, named graph capabilities, declared assets); anything not listed is not acquirable for this action;
6. **Capability requirements** — the capability tokens the action requires;
7. **Consent class** — the minimum class C0–C4, with the accepted consent mechanisms (click, biometric such as fingerprint, 2FA hardware key, passkey, hardware attestation) mapped per §XI.10;
8. **Assurance requirements** — including, where declared, a minimum Invocation Assurance Class and a minimum reasoning-provenance class (§XI.9.2);
9. **Mandatory transformations** — WR Guard measures that are preconditions of invocation (Annex X);
10. **Validation rules** — the deterministic schema and policy checks at the effect boundary;
11. **Finalizer reference** — for consequential actions, the Finalizer and WR TED envelope by reference;
12. **Evidence requirements** — receipt and PoAE™ record contents, verification tier (L/C/X);
13. **Execution body reference** — the WCR recording or declared deterministic chain;
14. **Optional adapter reference** — publisher LoRA adapter(s) per §XI.9.3;
15. **Revocation conditions and update path.**

XI.6.3 Lifecycle

PAMs are acquired, admitted, mutated, and revoked exclusively under PoAC (Annex III): evidence-backed proposals, review, signed admission — propose, never silently apply. Updates follow the versioned-manifest discipline of Annex VIII: never silent replacement; the user or organization admits the new version. Revocation removes availability immediately across all device classes and is itself an evidenced event.

XI.7 Action Registry and Context-Scoped Availability

The host maintains the Action Registry: the set of admitted PAMs. Every menu the pointer shows — the desktop action menu, the mobile swipe menu, radial layers, overlay offers — is a filtered view of this one registry, filtered by the current trigger context and sorted under §XI.11.2.

Context scoping (normative). Actions are available either globally or bound to declared trigger contexts (WR Code scan, BEAP message, Frame, document selection, camera designation, and additively extendable context classes). Users and organizations can add actions into a specific context, not only globally. Adding an action to a context — whether newly created, WCR-derived, org-shared, or publisher-offered — is a PoAC admission event with the corresponding origin class, review step, and signature. "Activating" an action is never a governance-free toggle.

Sharing. Because availability is manifest admission, sharing actions between users, devices, and organizations reduces to transporting signed PAMs and admitting them — cryptographically provable, analogous to WR Anchor export (Annex VIII). Portability is thereby a property of the artifact class, not a separate mechanism.

XI.8 Region Marker and Context Acquisition

The Region Marker is integrated into the pointer on all device classes: rectangular/freeform screen regions and element selection on desktop and tablet; element selection and confirmed camera frames on mobile.

The Region Marker is a governance instrument, not merely a screenshot tool:

1. **The region is the context boundary.** Only content inside the marked region (or designated element / confirmed frame) enters the action's context. Content outside it does not leave the device for this action (I8).
2. **The region is part of the Intent Hash.** The user authorizes exactly the marked excerpt they see (WYSIWYE, Annex IX by reference). A change of region invalidates the pending consent.
3. **WR Guard sits on the acquisition path — architecturally, not optionally.** Before any bounded context is routed to an external recipient, WR Guard classifies it against the applicable WR Expert artifacts and applies the policy-selected measures of Annex X (warn, block-until-release, mask, pseudonymize, obfuscate, distribute, abstract, local-only, recipient-bound encrypt, refuse). The applicable enforcement mode (advisory/enforced) follows the finding class and policy.
4. **Egress preview.** Where the flow permits, the pointer surfaces WR Guard's classification and intended transformations before the boundary is crossed, in the overlay, so the user sees the redaction, masking, or distribution plan ahead of disclosure.
5. **Every acquisition that crosses a boundary emits a Boundary Event Record** (Annex X), at the evidence tier the flow's assurance class requires.

XI.9 Model Routing and Publisher LoRA Adapters

XI.9.1 Routing

A pointer action separates intelligence from effect. Reasoning for an action may be performed by a fully local model, a private enterprise model, an approved external provider (under External Service Admission, Annex X), a publisher-provided model, or hybrid arrangements — including local context preparation with external reasoning, external reasoning with local execution, distribution across multiple models with partitioned context (Distribution Obfuscation, Annex VIII/X by reference), and local models selecting an external specialist. Routing is decided by the user and, where present, signed organization policy. Routing never changes the effect boundary (I7).

XI.9.2 Publisher Constraints on Reasoning (Bounded)

A publisher may not bind an action to a specific model product. A publisher may declare properties: a minimum reasoning-provenance class (e.g. "attested local model required," where the action's context includes publisher-confidential material) and a minimum Invocation Assurance Class. These are property constraints, verifiable at invocation, compatible with user sovereignty. Any attempt to make action safety depend on a particular model's behavior triggers the mis-specification rule (I7) and blocks admission.

XI.9.3 Publisher LoRA Adapters

A publisher may ship LoRA adapters (or comparable lightweight model augmentations) as part of the handshake package, to improve reasoning quality on its declared actions (domain vocabulary, product structures, form semantics). Normative constraints:

1. An adapter is a **capability acquisition under PoAC**: declared in the handshake's Merkle manifest, signed, versioned, admitted by proposal, revocable. Silent adapter updates are prohibited.
2. **Scope binding:** a publisher adapter is active only for that publisher's actions and only over the context sources those actions declare. It receives no access to local Graph context or device content beyond the declared allowlist. An adapter is never a side channel.
3. **Safety independence:** the effect boundary validates identically with or without the adapter (I7). Adapters improve quality; they never carry safety.
4. Adapter activation and deactivation are evidenced events.

Visual perception and vision adapters. Where designation from a camera, image, video, screen, application feed, or spatial scene relies on object detection, segmentation, classification, visual grounding, or comparable perception, the responsible component may be a native multimodal model, a separate vision model, or a compatible vision LoRA or comparable lightweight adapter admitted under this section. Its sole function is to produce typed designation candidates — including, as declared, class labels, bounding regions, segmentation masks, keypoints, component references, spatial poses, or confidence values — within the user-activated and permitted context source. A recognition result is an offer, not identity, authority, consent, or action: it does not become the bounded Pointer context until the user explicitly confirms the object, region, or frame under I8. Recognition of a machine or component does not establish authoritative asset identity unless that identity is independently bound through a WR Code®, verified publisher declaration, or other admitted identity evidence. The applicable PAM shall declare the visual task class, compatible base-model or encoder architecture, adapter identity and digest, permitted recognition classes, candidate-output schema, confidence policy, required confirmation step, and fallback designation method. The vision model or adapter remains bound to the declaring publisher, declared actions, and allowed context sources; it cannot create capabilities, expand the acquired context, bypass WR Guard™, lower consent requirements, or alter deterministic validation at the effect boundary.

XI.10 Consent, Authorization, and Evidence

Every consequential pointer action follows the WR TED lifecycle by reference (Annex IX): **Validate → Execute → Witness → Seal → Anchor**, with verification tiers Local / Central / External (Solana anchoring) as declared.

Consent classes and mechanisms. PAMs map to consent classes C0–C4. Consequential actions can be secured with graduated mechanisms — click confirmation, biometric confirmation (e.g. fingerprint), 2FA hardware key, passkey-protected approval, hardware attestation — as established in Annex IX. The mapping of mechanisms to classes is policy-configurable and is designed to be adapted to the assurance classes of Annex X in later versions of this annex (extension point: §XI.20); the invariant is only that a class's minimum can be raised by policy, never lowered.

The consent classes C0–C4 and the Finalizer classes READ / TRANSMIT / CHANGE / PERSIST / EXECUTE (Login-Bound §14) are orthogonal taxonomies: the Finalizer class names the consequence type; the consent class names the required human-authorization strength. Each PAM declares, per Finalizer class it references, the applicable consent-class floor. A canonical default mapping is a declared extension point (§XI.20).

Intent Hash contents. For pointer actions, the Intent Hash covers at minimum: the action identity and version, the bounded context (region/frame digest), the bound parameters, the origin class, the routing class of the reasoning that produced the parameter binding, and the declared consequences presented to the user. The user authorizes exactly this (WYSIWYE); any deviation invalidates consent.

Evidence. Execution produces the declared receipts and PoAE records; boundary crossings produce Boundary Event Records; capability acquisitions (actions, adapters) produce PoAC provenance. Optionally, a pointer session can emit a sealed session receipt aggregating its evidenced events — of direct value in field-service contexts (Annex IV).

Offline discipline per §XI.15: consent is never pre-staged against a context that may change before execution.

XI.11 The Augmented Overlay

XI.11.1 Role

The Augmented Overlay is the presentation channel of the interaction layer (I2). It harmonizes with the pointer as its display counterpart and shows:

- **status messages** — including the live lifecycle states of governed execution: the user sees, in the overlay, that validation passed, that execution is witnessed, that the seal is set, that anchoring is confirmed. The overlay is the visible face of the evidence chain, turning the cryptographic machinery into an experienced property rather than a hidden one;
- **additional and offered actions** — context-relevant entries from the Action Registry, ranked per §XI.11.2, always as offers (I2);
- **contextual help and information** — assistance bound to the content or object under the pointer.

XI.11.2 Suggestion Ranking

Overlay offers and menu ordering may be ranked by AI — bounded by I6 (selection among admitted actions only). Ranking may reuse the Bayesian priority machinery with user-feedback evidence specified in Annex VIII, and may draw on the WR Graph for personalization and recall. Ranking never introduces, hides the provenance of, or auto-executes an action.

XI.11.3 Provenance Consistency

The origin-class visual encoding (I4) is identical across the pointer menus and the overlay. A publisher-supplied help panel carries the same publisher badge as the publisher's declared action; a local AI suggestion carries the same encoding as a local action. Menu and overlay never diverge in provenance presentation.

XI.11.4 Overlay Content Origins (Closed Set)

Overlay content originates from exactly two sources — there is no third path (I5):

1. **Declared handshake assets.** Publisher help content, product information, status texts, and interactive elements are declared asset classes in the handshake's Merkle manifest: versioned, signed, updated only via the versioned update path. They render under the strict passivity discipline of Annex VIII; interactive elements are Mini-Apps composed declaratively from the local library of pre-admitted interaction primitives. The executable implementations of these primitives — together with all enforcement, authorization, and evidence mechanisms — are supplied exclusively by the host runtime; publisher-specific automation semantics may additionally be supplied as signed declarative repository artifacts (§A.5, §A.21). No publisher scripts.
2. **Local artifacts.** Local help and information derive from WR Expert artifacts, the WR Graph, and local system state, with local provenance.

XI.11.5 Overlay on Composed Content

On Frames and infographics (Annex VIII), the overlay additionally supports Smart Actions and Editable-Region interaction as already specified there; this annex adds only the uniform origin encoding and the I2 duality on top of the existing mechanics.

XI.12 Handshake Declaration Requirements

For publishers, pointer and overlay functionality is automation like any other in the WR system and is treated with the same strictness:

1. The publisher's handshake repository must declare every pointer action, every overlay asset class, every Mini-App composition, every adapter, and — explicitly — every **AI automation offered for the WR Pointer and the Augmented Overlay**, in the capability-declaration and Merkle-manifest structures of Annex VII.
2. Undeclared pointer/overlay functionality does not exist for the client: it cannot be offered, rendered, or invoked.
3. Declarations carry the publisher's attestation tier (P0–P2) and are subject to the grant model of Annex VII; the user's admission of a publisher's pointer actions is a consent event, and risk classes drive the required consent and evidence per §XI.10.
4. Changes flow only through versioned manifest updates — never silent replacement — and revocation is honored immediately (§XI.6.3).

XI.13 WCR Integration

WCR recordings can be initiated and controlled through the WR Pointer on all device classes. The design separates learning from authorizing:

1. **Recording** (start, pause, stop, annotate) is a local, non-consequential C0 pointer action. The recording itself acquires no authority.
2. **Admission** of a finished recording as a new pointer action — globally or into a specific trigger context — is the governance moment: a PoAC proposal with review, PAM generation, and signature (§XI.6, §XI.7). Only admission makes the recording invocable.
3. Recordings replayed as actions execute deterministic-at-tool-boundary, adaptive-on-path, per the WCR discipline; where consequential, under the full §XI.10 chain.
4. Organization-shared and publisher-offered WCR-derived actions follow the same path with their respective origin classes.

XI.14 Cross-Device Continuity

Pointer sessions are continuable across devices via the Host, with the session state carried as a WR Graph object. The canonical division of labor:

- **Smartphone:** capture (Real-World Pointer, WR Code scan, evidence), short instructions, queueing work, and approval;
- **Desktop:** continuation with the full Graph, stronger local models, rich editing, multi-application composition, and the full overlay surface.

Phone-as-Approver (normative option). A consequential action initiated on the desktop may require its consent step to be completed on the user's enrolled smartphone: the phone displays the Intent Hash content (WYSIWYE) full-screen and captures the consent mechanism (biometric, hardware key, passkey). The phone thereby serves as the possession-factor approval device inside the graded consent mechanics of Annex IX — unifying cross-device continuity and consent hardening in one mechanism. Enrollment of an approver device is itself a capability-governed, evidenced act.

Session hand-over never transports authority implicitly: capabilities and pending consents are re-validated on the receiving device (Locality-Independent Trust, Annex X).

XI.15 Offline Behavior

- Local actions remain fully functional offline: local models, admitted WCRs, resident Graph access.
- Handshake-associated actions: PAMs and declared assets are cacheable within their declared validity windows. Consequential actions may be queued offline for deferred finalization only where the manifest explicitly permits it; otherwise the pointer refuses cleanly rather than degrading governance. For login-bound actions, deferred finalization additionally requires a fresh Binding Challenge and a currently valid Login-Bound Capability Proof at finalization time; a queued action never finalizes against a starved binding (Login-Bound §5).
- Consent is never pre-staged for later execution against potentially changed context; the Intent Hash binds the state at consent time, and deferred execution re-validates before finalizing.
- All deferred events are evidenced upon reconnection at their declared tiers.

XI.16 Threat Model and Honest Boundary

This section names the principal risks and the design's posture toward them, without asserting absolute protection.

Overlay clickjacking / fake dialogs. Structurally neutralized by I2: overlay elements carry no execution path. Residual risk: user confusion between overlay offers and pointer invocation surfaces; mitigated by consistent provenance encoding and full-screen consent steps for consequential actions.

Injection via visible content. Screen and camera content attempting to smuggle instructions is neutralized by I3 and I6: content cannot define or alter actions, and AI cannot synthesize paths. Residual risk: content influencing AI ranking of admitted offers; bounded because ranking can neither add actions nor change their declared semantics or consent class.

Pointer chrome spoofing. A page or app imitating the pointer's UI to harvest interactions. Honest Boundary statement: application-level rendering cannot fully exclude imitation on today's operating systems. Posture: Phase-1 deployment on WR-owned surfaces where the host controls the compositor entirely (§XI.18); host-rendered chrome outside page compositors where platform APIs allow; consequential consent always in host-controlled full-screen surfaces with Intent-Hash display; OS-level trusted-UI integration as the declared evolution path — extending, not replacing, platform security mechanisms, in line with the Annex X trajectory.

Exfiltration through context acquisition. The pointer sees what the user sees; governance, not blindness, is the control: I8 bounding, mandatory WR Guard on the acquisition path, Annex X measures, Boundary Event Records for every crossing and every find.

Overlay permission / accessibility-API abuse (mobile). The mobile embodiments request only the permissions their declared functions need; ambient capture is not a pointer function; all accessibility-service usage (Phase 3) is declared, policy-visible, and evidenced.

Consent fatigue. Graduated classes (C0–C4) keep friction proportional to consequence; pre-admitted policy (organizational) can authorize low-risk classes; consequential classes deliberately retain friction. Ranking (§XI.11.2) reduces offer noise.

Accidental activation (mobile). Long-press-gated menus, no consequential actions in first interaction layers, distinct full-screen consent — §XI.17.

Malicious or defective action manifests. PoAC admission with review and signature; mis-specification rule (I7) at admission; revocation with immediate effect; versioned updates only.

XI.17 Interaction Patterns (Informative)

Desktop. Hover-bound augmentation with subtle affordance; click or shortcut opens the context menu (registry view); drag opens the Region Marker; consequential invocations always route through a host-owned confirmation surface displaying the Intent-Hash content.

Mobile, on-screen. Floating control, movable and dismissible; long-press opens the radial or swipe menu — never hover- or content-triggered; swipe menu lists context-filtered actions with an explicit "add action" flow (global or into the current context, each an admission event);

consequential actions require a separate full-screen consent step with biometric/hardware-key capture as declared.

Mobile, real-world. Point camera; the overlay annotates the recognized designation candidate; user confirms the frame or object explicitly (the confirmation is the bounding event); context-scoped actions for camera designation then appear in the swipe menu.

Tablet. Stylus hover as pre-selection, stylus lasso as Region Marker, otherwise combining both mobile and desktop patterns.

Accessibility (design goal, not afterthought). A single, uniform, declaratively described action surface is inherently more tractable for assistive technologies than heterogeneous per-app UIs: every action carries a machine-readable manifest (name, inputs, consequences, consent class), enabling consistent screen-reader announcement, switch access, and voice invocation across all contexts. Accessibility mappings are a first-class part of each embodiment's interaction pattern set.

XI.18 Staged Implementation (Informative)

- **Phase 1 — WR-owned surfaces (no OS integration).** Pointer and overlay on WR Desk's own rendering surfaces: TimesDesk™ Frames, composed infographics, BEAP message views, Editable Regions. Full compositor control; the spoofing problem does not arise; the complete governance chain is exercised end-to-end.
- **Phase 2 — Browser integration.** Extension-based selection, Region Marker, and handshake actions on WR-connected pages; WR Anchor as the natural attachment point.
- **Phase 3 — Cross-application (accessibility services / AX APIs).** App-spanning selection and designation on desktop and Android; the threat-model items of §XI.16 concerning overlays and accessibility permissions become primary.
- **Phase 4 — OS-level integration.** Trusted pointer chrome, capability-token checks below the application layer (extending platform mechanisms such as LSM-based controls rather than replacing them, per the Annex X trajectory).

Each phase is additive; no phase relaxes an invariant. The mapping of these phases to Rendering Assurance Classes and to the integrated target architecture is given in §A.20.

XI.19 Prior Work and Differentiation (Informative)

Assistive cursors, floating assistant buttons, OS share-sheets, browser copilots, screen-reading AI overlays, and "click-to-ask" tools are established categories. The disclosed combination differs in the governance binding, which to the author's knowledge is not present in these categories:

- signed, versioned, revocable action manifests with declared context-source allowlists and consent classes;
- a strict invocation/presentation duality in which the presentation layer is structurally incapable of execution;
- content-is-never-instruction across screen and camera input, with AI confined to routing over pre-admitted actions;
- the mis-specification rule making effect-boundary determinism an admission criterion;
- mandatory information-flow governance (WR Guard, Boundary Event Records) on every context acquisition;

- device-invariant Pointer semantics with device-adaptive embodiments — including the camera as Real-World Pointer — under one uniform governance architecture, where instrument availability follows hardware rather than device class;
- cross-device Phone-as-Approver consent inside a cryptographic evidence chain (PoAE, WR TED lifecycle, L/C/X verification).

Essential combined disclosure. For the avoidance of doubt, this annex discloses, as a combination: a pointer with invariant designation, context-binding, capability-resolution, and invocation semantics across all device classes, embodied through device-adaptive instruments and affordances, serving as sole invocation surface over a signed action registry with origin classes and context-scoped availability; an overlay serving as sole presentation surface under offer-never-action; explicit region-/frame-bounded context acquisition hashed into user consent; user-/policy-controlled model routing with publisher-shipped, PoAC-governed LoRA adapters scoped to declared actions; publisher declaration of all pointer/overlay AI automations in handshake repositories; WCR recording triggered by the pointer with admission-gated activation; cross-device continuation with smartphone-based cryptographic approval; and deterministic finalization with graded consent mechanisms and multi-tier verification. Each element and the combination are hereby published as prior art.

XI.20 Extension Points and Amendment Discipline

Declared extension points for future versions (additive only):

1. new device classes and embodiments (§XI.4.6);
2. new trigger-context classes (§XI.7);
3. refined mapping of consent mechanisms to assurance classes (§XI.10);
4. additional overlay content forms within the closed origin set (§XI.11.4);
5. session receipts and their evidence schema (§XI.10);
6. Phase-3/4 platform integration specifics (§XI.18);
7. accessibility mapping specifications (§XI.17);
8. adapter classes beyond LoRA under the same §XI.9.3 constraints;
9. declared external-surface handoff profiles (§A.9);
10. a canonical default mapping of Finalizer classes (READ / TRANSMIT / CHANGE / PERSIST / EXECUTE) to consent-class floors C0–C4 (§XI.10, Login-Bound §14).

All amendments follow §XI.0: versioned, never silent, recorded in §XI.21. The extension points above are additive by design. Changes to I1–I9 beyond clarification or strengthening are permitted only under the explicit-identification and justification discipline of §XI.0.

XI.21 Version History

- **v1.0 — 19 July 2026.** Initial version. Core invariants I1–I9; device-class model with desktop AI cursor, mobile Real-World Pointer and On-Screen Pointer, tablet hybrid; invocation modalities (speech, text, handshake-based and self-predefined AI automation) decoupled from pointing; unified action surface with origin classes; Pointer Action Manifest and Action Registry with context-scoped admission; Region Marker as governance instrument; model routing with publisher LoRA adapters under PoAC; consent/evidence integration by reference to Annex IX/X; Augmented Overlay as sole presentation channel

with closed content-origin set; handshake declaration requirements; WCR integration; cross-device continuity with Phone-as-Approver; offline discipline; threat model with Honest Boundary; informative interaction patterns, staged implementation, prior-work differentiation; extension points.

- **v1.1 — 22 July 2026.** Integration of Addition Block A (target client architecture: Rendering Assurance Classes R0/R1/R2; PAM rendering-requirement extensions; publisher-supplied declarative automation; restricted public repository artifact model with isolated image ingest and normalization; controlled publisher page model with passive templates and typed render slots; governed network broker; target client compartments; Same-Origin Session Co-Presence; native applications as external execution environments; minimal publisher integration; centralized/P2P/installation-free profiles; three-class threat model; staged target mapping). Integration of the section "Login-Bound Personalized Handshakes, Dynamic Account Context, and Governed Remote Assistance" (Addition Set v1.1) and of Addition Block B (target-architecture mapping for login-bound sessions: Dual-Compartment Possession; cookie confinement; generalized session-environment binding; restart, multi-instance, and account-switch discipline; Rendering Assurance classification of login-bound examples; external application sessions). Superseded with explicit identification per §XI.0: permanent external-browser attachment (replaced by Same-Origin Session Co-Presence, §A.21, §B.2, §B.8) and the "interaction logic supplied exclusively by the host runtime" wording (reinterpreted per §A.21). Revised: amendment discipline of §XI.0, §XI.3, and §XI.20 — material invariant changes are permitted under explicit identification and justification, never silently (v1.0 stated unconditional non-weakening). Terminology: capability slots and render slots distinguished (Login-Bound §6, §A.7); recording acronym unified to WCR; "public PBEAP layer" corrected to "public handshake layer" (Login-Bound §1, Phase D).
- **v1.2 — 24.07.2026.** Pointer Invariance refinement. I9 strengthened to an explicit invariance principle. §XI.4.1 revised: the v1.0 statement that the pointer "may carry different functions per device class" is withdrawn as a mis-specification and replaced by function invariance with device-adaptive embodiment (explicit-identification discipline of §XI.0; this change strengthens the disclosed architecture and narrows nothing). Device-independent designation-instrument and source-class catalog added to §XI.4.1; device profiles §XI.4.2–§XI.4.4 reclassified informative with a non-restriction clause at the end of §XI.4.1; smartphone role profile reclassified as ergonomic default; Real-World Pointer and On-Screen Pointer stated as device-independent capabilities; §XI.4.6 restricted to availability declarations; §XI.2 definition and §XI.19 differentiation/essential combined disclosure updated accordingly. Section numbering unchanged.
- **v1.3 — 26.07.2026.** Additive. Designation-instrument catalog of §XI.4.1 extended by four device-independent instrument classes: identifier scan (WR Code®, QR, barcode, NFC, RFID via camera or dedicated scanner; identifier-never-authority per Annex IX, formation rules unchanged), proximity designation (user-confirmed candidate offer, never self-designating), terminal and industrial input surfaces, and declared WR Link activation as designation of a link's hash-bound target context (selection vector, never execution or entry vector; Annex IX §IX.7 by reference). Source classes extended by identifier-referenced physical targets. Unmapped Target Fallback added to §XI.5: designation of a repository-unmapped target inside a handshake-associated context falls back to a local action — local

routing and WR Guard rules, nothing of the handshake crossing in either direction, non-(d) origin marking per I4, no publisher signal for the unmapped designation, re-entry into handshake context only via declared permitted context sources (§XI.6.2(5)). No existing rule weakened; no section renumbered.

Addition Block A — Target Client Architecture, Rendering Assurance, Session Compartmentalization, Publisher-Supplied Repository Automation, and External Software

Addition Block A — first published with Annex XI v1.1, 22 July 2026.

Status and Relationship to the Existing Annex

This addition supplements the existing architecture of the WR Pointer™, Augmented Overlay, Action Registry, Pointer Action Manifests, WCR integration, Public Handshake, WR Guard™, capability model, consent model, Finalizers, and evidence chain.

It does not replace:

- the explanation of the problem addressed by the Pointer and Augmented Overlay;
- the invocation/presentation duality;
- the device-adaptive Pointer embodiments;
- the origin classes;
- the Region Marker;
- content-is-never-instruction;
- AI-as-Router;
- effect-boundary determinism;
- the existing PAM lifecycle;
- the existing Public Handshake declaration requirements;
- the existing login-bound capability architecture;
- the existing support, voice, cross-device, consent, and evidence mechanisms.

The existing annex already requires the publisher's handshake repository to declare every Pointer Action, Overlay asset class, Mini-App composition, adapter, and AI automation offered through the Pointer or Augmented Overlay. Undeclared functionality is unavailable to the client. This addition clarifies how that publisher-supplied automation is interpreted in the target controlled-browser architecture.

To the extent earlier wording states that "all interaction logic" is supplied exclusively by the host runtime, that wording shall be understood as referring to the implementation and enforcement of admitted runtime primitives, not to the publisher-specific automation semantics. The publisher may

supply declarative automation logic through the signed repository. The WR runtime supplies the interpreter, controlled primitives, authorization, validation, enforcement, and evidence mechanisms.

To the extent earlier wording makes permanent attachment to an external browser a load-bearing architectural precondition, that wording is superseded by the implementation-independent requirement of Same-Origin Session Co-Presence, defined below. The external-browser embodiment remains valid, but the integrated orchestrator and controlled browser constitute the primary target architecture.

A.1 Purpose and Architectural Priority

The primary target architecture is an integrated WR orchestrator and browser with a controlled rendering pipeline.

In this architecture, the browser is not merely an unrelated third-party application operated indirectly through:

- a browser extension;
- injected page scripts;
- an accessibility interface;
- generic UI automation;
- screen scraping;
- a remote-control bridge;
- an untrusted page overlay.

The browser is instead integrated into the WR client while remaining internally compartmentalized.

The target architecture combines:

- publisher-origin verification;
- Public Handshake establishment;
- public repository verification;
- website-session handling;
- controlled rendering;
- governed network access;
- Pointer invocation;
- Augmented Overlay presentation;
- WCR execution;
- model routing;
- capability evaluation;
- consent;
- WR Guard;
- Finalizers;
- execution witnessing;
- evidence;
- revocation;
- logout.

Integration does not mean that every component receives every authority.

The browser, renderer, website-session state, handshake keys, WR Graph, model runtime, capability state, and evidence state remain separated according to their purpose.

The target architecture is prioritized because deterministic automation over a publisher website requires control over the rendering pipeline. A browser extension can support useful and safe actions, but it cannot establish that the external page, DOM, scripts, browser extensions, or displayed state correspond to the publisher's declared automation graph.

Native applications and other installed software are addressed as secondary completeness embodiments. They do not displace or delay the integrated orchestrator and controlled browser as the primary target.

A.2 Separation of Effect Assurance and Rendering Assurance

Effect assurance and rendering assurance are separate architectural dimensions.

A.2.1 Effect-Boundary Determinism

Effect-boundary determinism remains renderer-independent.

A consequential operation must pass through the existing deterministic chain, including as applicable:

- an admitted PAM;
- a declared capability;
- typed input validation;
- provenance validation;
- account- and tenant-scope validation;
- signed policy;
- WR Guard;
- the applicable consent class;
- a Finalizer;
- execution witnessing;
- sealing;
- evidence and receipt generation.

The reasoning path may remain adaptive or probabilistic.

A model may:

- interpret an operator request;
- rank admitted actions;
- select a declared WCR path;
- bind permitted values;
- prepare a proposal;
- explain an expected consequence.

The model may not create authority or bypass the effect boundary.

Any action whose safety depends on a model behaving correctly, rather than on deterministic validation at the effect boundary, remains mis-specified under the existing invariant.

A.2.2 Rendering Assurance

Rendering Assurance concerns whether the runtime can establish facts about the displayed and navigated environment, including:

- the identity and version of the page template;
- the integrity of displayed assets;
- the provenance of dynamic values;
- the correspondence between a visible element and a declared element;
- the available navigation state;
- the expected next state;
- the permitted submission endpoint;
- the integrity of the displayed Intent Hash;
- the integrity of an Egress Preview;
- the separation between publisher content and authorization presentation.

Rendering Assurance is necessary where an automation depends on the correctness of the visible interface or page state.

A.2.3 No Substitution

A strong Finalizer does not prove that a page was rendered correctly.

A controlled renderer does not replace capability validation, consent, WR Guard, or Finalizers.

Both dimensions remain independently enforceable.

A.3 Rendering Assurance Classes

The following classes identify the relevant rendering embodiment.

The class identifiers describe architectural embodiments. They are not treated as a universal numeric trust score. The Action Registry evaluates whether the current class provides the specific properties required by the action.

R0 — WR-Owned Controlled Surface

R0 applies where the complete relevant composition and interaction surface is produced and controlled by the WR runtime.

Examples include:

- TimesDesk™ Frames;
- BEAP views;
- WR-generated infographics;
- Editable Regions;
- WR-owned configuration views;
- WR-owned consent views;
- WR Graph views;
- locally composed execution previews.

At R0, the runtime can establish the integrity of:

- the composition;

- the displayed action;
- the provenance indicators;
- the bounded context;
- the Intent Hash presentation;
- the applicable WR interaction elements.

R0 does not by itself establish the state of a publisher's ordinary website.

R1 — External Browser or External Application Surface

R1 applies where WR interacts with an externally controlled browser or application through mechanisms such as:

- a browser extension;
- accessibility integration;
- operating-system automation;
- an application plugin;
- an external overlay;
- a share interface;
- a remote-control or assistance bridge.

At R1, WR may provide:

- publisher-origin verification;
- Public Handshake activation;
- Region Marker acquisition;
- element designation;
- login-bound session proof;
- provenance-labeled actions;
- contextual guidance;
- element highlighting;
- verified destination opening;
- best-effort navigation;
- preparation of independently validated server-side actions.

R1 does not establish:

- integrity of the live DOM;
- integrity of the visible page;
- absence of hostile page scripts;
- absence of hostile browser extensions;
- correspondence between a declared selector and the visible element;
- deterministic page navigation;
- trustworthy in-page account presentation;
- trustworthy in-page Intent Hash presentation;
- trustworthy in-page Egress Preview.

A consequential action may still be safe at R1 where its actual effect is independently validated by the capability and Finalizer.

An action whose safety depends on the external interface having followed a declared page sequence is mis-specified for R1.

R2 — Integrated Controlled Publisher Renderer

R2 applies where the publisher experience is rendered by the integrated controlled browser under the requirements of this addition.

R2 includes:

- verified publisher origin;
- verified Public Handshake;
- signed Site Manifest;
- admitted passive page template;
- normalized and hash-pinned images;
- declared typed slots;
- declared navigation graph;
- declared submission endpoints;
- governed network access;
- isolated website-session state;
- WR-runtime-rendered provenance, consent, and execution-status elements;
- publisher automation interpreted through the admitted declarative runtime.

At R2, the runtime may establish deterministic page-state and navigation properties only within the exact scope represented by the verified template, manifest, runtime slots, and state-transition declarations.

R2 does not prove:

- publisher honesty;
- semantic truth;
- model correctness;
- user understanding;
- operating-system integrity;
- hardware integrity;
- correctness of an external service.

A.4 PAM and Action Registry Extensions

Every PAM whose availability or safety depends on the rendering environment shall declare the applicable rendering requirements.

The PAM extension may include:

`minimum_rendering_assurance`

`permitted_rendering_classes`

`required_rendering_properties`

`trusted_intent_presentation_required`

`trusted_egress_preview_required`

`session_binding_required`

`required_template_hash`

`required_site_manifest`

`required_asset_index`

`permitted_navigation_graph`

`permitted_submission_endpoints`

`render_slot_schema`

`external_execution_environment_policy`

`lower_assurance_fallback_action`

The exact encoding may evolve. The declared properties are normative in intent.

lower_assurance_fallback_action references a separately admitted PAM with its own declarations, consent class, and rendering requirements. The fallback is offered in the registry's presentation as an alternative; it is never automatically substituted for the unavailable action, it never inherits the original action's admission, and it never reduces the original action's requirements. Selecting the fallback is a distinct invocation of a distinct action.

The Action Registry evaluates these fields together with:

- origin class;
- trigger context;
- device class;
- active Public Handshake;
- handshake version;
- account-login state;
- LBCP state;
- session environment;
- policy;
- consent readiness;
- capability scope;
- revocation state.

A publisher action requiring R2 is not made executable at R1 merely because its repository artifacts are available.

The registry may instead present:

- guidance-only mode;
- verified-link mode;
- manual continuation;
- controlled renderer required;
- login required;
- reauthentication required;
- approval on another device required;
- external software handoff required;
- action unavailable in the current environment.

AI ranking may rank only the alternatives already admitted for the current rendering class.

It cannot override Rendering Assurance requirements.

A.5 Publisher-Supplied Automation Logic

The publisher may supply the domain-specific automation logic for its verified offering.

This automation is published through the publisher's publicly accessible handshake repository — hosted on Optirando infrastructure, mirrored by the publisher, or both; repository identity is defined by the signed manifests and artifact digests, not by the hosting location (§A.6.3, source-independent delivery) — and bound to the publisher through:

- the Public Handshake;
- publisher signature;
- capability declarations;
- artifact hashes;
- Merkle inclusion;
- manifest version;
- revocation state.

Publisher-supplied automation may include:

- Pointer Action Manifests;
- WCR definitions;
- WCR execution graphs;
- declared action sequences;
- decision branches;
- state-transition graphs;
- navigation graphs;
- account-context requirements;
- prompts;
- model-routing declarations (property constraints only, per §XI.9.2 — never a binding to a specific model product);
- schema definitions;
- identifier-slot definitions;
- operator-input definitions;
- validation declarations;
- consent-class floors;
- Invocation Assurance requirements;
- Rendering Assurance requirements;
- Overlay help content;
- Overlay status content;
- declarative Overlay compositions;
- Mini-App compositions;
- adapter references;
- Finalizer references;
- expected receipts;
- evidence requirements.

The publisher may therefore tailor the Pointer and Augmented Overlay to its products, services, website structure, account system, support procedures, and declared functions with high precision.

Division of Responsibility

The publisher declares:

- what actions exist;
- what each action means;
- which inputs it expects;
- which states and branches are permitted;
- which navigation paths may be used;
- which contextual help is presented;
- which model adapter may assist;
- which consequence is requested;
- which Finalizer is referenced;
- which evidence is expected.

The WR runtime supplies:

- repository and manifest verification;
- the PAM interpreter;
- the WCR interpreter;
- the implementation of admitted interaction primitives;
- the controlled renderer;
- schema and slot validation;
- account-scope validation;
- the Action Registry;
- model isolation and routing;
- WR Guard;
- capability enforcement;
- consent enforcement;
- Finalizer enforcement;
- witnessing;
- sealing;
- receipt generation;
- revocation handling.

The publisher supplies the declared automation semantics.

The WR runtime supplies the controlled interpretation and enforcement substrate.

Excluded Publisher Execution Paths

Publisher-supplied declarative automation is allowed.

The following remain excluded from the strict R2 profile:

- undeclared JavaScript;
- arbitrary publisher WASM;
- native executable modules;

- dynamically loaded executable code;
- inline executable event handlers;
- publisher-controlled implementations of trusted runtime primitives;
- hidden automation paths;
- undeclared network logic;
- runtime-created Pointer Actions;
- runtime-created capabilities;
- runtime-created Finalizers;
- silent reduction of consent or assurance requirements;
- execution outside the signed repository and manifest path.

The distinction is not between "publisher logic" and "no publisher logic."

The distinction is between:

1. publisher-declared, signed, versioned, schema-constrained automation interpreted through the WR runtime; and
2. independently executable publisher code capable of creating an ungoverned path.

Only the first is admitted.

A.6 Public Repository Artifact Model

The publicly accessible Optirando repository uses a deliberately restricted artifact model.

The initial admitted classes are:

1. declarative text;
2. safetensors or an equivalent non-executable tensor container;
3. normalized images;
4. the signed manifests, schemas, indexes, admission records, and revocation records required to govern those artifacts.

Admission of every artifact class includes the applicable Scam Watchdog™ analysis before publication: manipulative-content and injection-pattern analysis for declarative text (including prompts, Overlay content, help text, and status text), behavioral screening of adapters within their declared scope, and the image analysis of §A.6.3. An artifact is published only after passing the applicable analysis; findings and passes are part of the admission evidence. Analysis reduces, and does not replace, the semantic-honesty risk class of §A.19.

A new artifact class may be added only through a versioned specification that defines:

- its canonical format;
- parser;
- normalizer;
- size limits;
- execution properties;
- validation rules;
- admission procedure;
- client handling;
- revocation behavior.

A.6.1 Declarative Text

Declarative text may represent:

- PAMs;
- WCRs;
- passive page templates;
- passive style declarations;
- prompts;
- navigation graphs;
- slot schemas;
- validation rules;
- decision branches;
- Overlay content;
- Overlay compositions;
- Mini-App compositions;
- Finalizer references;
- help text;
- status text;
- policies and policy references.

Text is admitted only after validation and canonicalization into the applicable declarative subset.

Text may describe actions and automation.

It does not become authority merely because it contains imperative language.

Admitted prompt artifacts are declarations, not runtime content. The content-is-never-instruction invariant (I3) governs runtime-visible and runtime-received content; it is not contradicted by a signed, versioned, admitted prompt artifact. A prompt artifact can shape reasoning quality and the wording of explanations; it cannot expand effects beyond the declared actions, branches, capability requirements, and consent classes (I6, I7). Its semantic honesty — a misleading but formally valid prompt — falls under the honesty class of §A.19 and the accountability mechanisms named there.

A text value received through page content, a slot, a transcript, a prompt, or external communication cannot create:

- a new PAM;
- a new capability;
- a new WCR path;
- a new Finalizer;
- a new policy;
- a new network destination.

Authority continues to derive from admitted signed artifacts and the applicable capability chain.

A.6.2 Safetensors and LoRA Adapters

Publisher-provided LoRA adapters or comparable lightweight augmentations may be published in safetensors or another explicitly admitted non-executable tensor representation.

An adapter may:

- improve domain vocabulary;
- improve product understanding;
- improve form understanding;
- improve selection among declared branches;
- change model output behavior.

An adapter may also be defective or maliciously tuned.

For that reason:

- it is bound to the declaring publisher;
- it is bound to declared actions;
- it receives only declared context sources;
- it receives no ambient WR Graph access;
- it receives no website credentials;
- it is versioned;
- it is revocable;
- activation and deactivation are evidenced;
- it cannot create capabilities;
- it cannot bypass WR Guard;
- it cannot bypass consent;
- it cannot bypass the Finalizer.

The effect boundary validates identically with or without the adapter.

The adapter may carry quality.

It never carries safety or authority.

A.6.3 Normalized Images

Publisher-original images are not used directly as client rendering artifacts.

The publisher uploads the original image to the Optirando-controlled ingest pipeline.

The original is interpreted only in an isolated ingest environment.

The pipeline:

1. validates the claimed format and permitted size;
2. decodes the original in an isolated worker;
3. converts it into an admitted canonical image representation;
4. strips metadata;
5. removes undeclared auxiliary structures and embedded payloads;
6. applies declared resolution, profile, and dimension limits;
7. re-encodes the image using an Optirando-controlled encoder;
8. performs structural validation;
9. applies the relevant Scam Watchdog analysis;
10. computes the digest of the normalized result;
11. creates a PoAC-backed artifact-admission record;
12. publishes or rejects the normalized result.

The publisher then deploys or references the normalized result.

The publisher-original image is not referenced as the rendering object in the Site Manifest.

Client Rule

The client performs:

```
fetch → verify digest and admission → decode → render
```

The client does not perform:

```
fetch unknown publisher image → decode → determine trust afterwards
```

If the fetched object:

- has no admitted digest;
- does not match the expected digest;
- is absent from the relevant manifest or signed asset index;
- has been revoked;
- uses an unsupported representation;

the client does not decode or render it.

The client instead presents a neutral placeholder or verification failure.

Where the client cannot refresh revocation status within an artifact's declared validity window, consequential rendering and execution follow the declared fail-closed behavior; stale revocation state is never treated as proof of continued validity. Non-consequential cached rendering within the declared validity window remains permitted.

A separate low-privilege decoder process may still be used as defense in depth, but it is not the primary trust mechanism. The primary rule is that only the normalized, admitted, digest-matching representation reaches decoding.

Source-Independent Delivery

The admitted normalized image may be retrieved from:

- the publisher server;
- the Optirando repository;
- an approved CDN;
- local immutable cache;
- an approved peer;
- an offline bundle.

The delivery source does not define the artifact identity.

The digest and manifest binding define the artifact identity.

This allows the client to retrieve an asset directly from the publisher without requiring Optirando to observe each page access.

It also allows immutable caching:

- unchanged asset = unchanged digest;
- changed asset = new digest;
- new digest = manifest or asset-index update.

Incremental Asset Index

Publishers with frequently changing media may use a signed incremental asset index referenced by the parent Site Manifest.

The index shall remain:

- publisher-bound;
- handshake-bound;
- versioned;
- integrity-protected;
- revocable;
- append-only or explicitly superseding;
- monotonically versioned with anti-rollback: a client never accepts an index or Site Manifest version older than the newest version it has verified for the same manifest root;
- restricted to admitted normalized assets.

The mechanism avoids requiring complete re-publication of an otherwise unchanged Site Manifest for every newly admitted image while retaining cryptographic traceability.

A.7 Controlled Publisher Page Model

The image-admission model generalizes to the complete controlled publisher page.

An R2 page consists of:

- an admitted passive page template;
- admitted passive style information;
- admitted normalized images;
- a WR-runtime-supplied font set (fonts are not a publisher-suppliable artifact class in the initial repository model; a publisher font class may be added only through the versioned artifact-class process of §A.6, defining canonical format, parser, normalizer, size limits, and execution properties);
- typed render slots;
- publisher-declared automation artifacts;
- runtime-implemented interaction primitives;
- declared navigation;
- declared submission endpoints.

The passive page template may be represented as normalized HTML/CSS or another declarative composition format, provided that the applicable profile excludes active publisher execution.

Passive Template

The template is:

- validated against the admitted passive subset;
- canonicalized;
- hashed;
- referenced by the Site Manifest;
- versioned;

- revocable.

The template cannot invoke:

- publisher JavaScript;
- publisher WASM;
- dynamic code loading;
- undeclared network requests;
- undeclared storage;
- undeclared submission;
- arbitrary external embeds.

Typed Render Slots

Dynamic account or session values enter the rendered page only through declared typed render slots. Render slots are a presentation mechanism and are distinct from the capability slots of the Identifier-Only Slot Discipline (Login-Bound §6): render slots carry values to be displayed; capability slots carry invocation inputs. Throughout Block A, "runtime slot" and "typed slot" refer to render slots. The URL slot class below is a render-slot subclass, not a third capability slot class.

Each slot defines:

- value class;
- permitted source;
- account scope;
- tenant scope;
- permitted representation;
- applicable capability;
- sensitivity;
- retention;
- recipient;
- validation.

A render-slot value is rendered as data.

It is not reparsed as:

- HTML;
- CSS;
- script;
- an event handler;
- an asset declaration;
- an arbitrary URL;
- a capability;
- a Pointer Action;
- a WCR path;
- a policy;
- a Finalizer.

Where a URL-like value is required, it must use a separately declared and validated URL slot class with restricted schemes and origins.

Combined Rendering Object

The controlled publisher view is therefore:

```
pinned passive template
+ pinned normalized assets
+ publisher-declared automation
+ typed render values
+ WR-runtime primitive implementations
+ WR enforcement and evidence
```

This permits a personalized and dynamically updated publisher experience without admitting arbitrary publisher execution into the renderer.

The controlled publisher view is a governed content surface. It is neither the pointer nor the overlay: it carries no invocation path of its own, and the invocation/presentation duality (I2) applies unchanged on top of it — pointer actions over R2 content invoke, overlay elements over R2 content offer, and nothing rendered inside the publisher view executes.

A.8 Governed Network Access

The controlled renderer has no ambient network authority.

All network access passes through a governed broker.

For each request, the broker evaluates:

- publisher origin;
- active Public Handshake;
- Site Manifest;
- resource class;
- expected digest;
- request method;
- destination origin;
- redirect behavior;
- session requirement;
- capability;
- policy;
- WR Guard;
- evidence requirement.

The broker may permit:

- Public Handshake retrieval;
- Site Manifest retrieval;
- admitted artifact retrieval;
- same-origin authenticated reads;
- typed slot resolution;
- declared navigation;
- declared form submission;

- Finalizer invocation;
- heartbeat;
- session-status checks;
- revocation checks;
- admitted external services.

The broker refuses undeclared:

- tracking pixels;
- beacons;
- arbitrary third-party embeds;
- speculative prefetching;
- dynamic code retrieval;
- background network channels;
- unverified redirects;
- undeclared form destinations;
- publisher-controlled exfiltration paths.

Every relevant trust-boundary crossing remains subject to WR Guard and the existing Boundary Event Record discipline.

A.9 Navigation, Links, and Form Submission

R2 Navigation

A publisher may declare a navigation graph in its repository artifacts.

A navigation node may declare:

- required page-template hash;
- required page-state identifier;
- required slot state;
- permitted previous state;
- permitted next state;
- destination;
- capability requirement;
- interruption behavior;
- applicable account context.

The runtime may traverse the graph only while the current verified state matches the declaration.

If an expected state is absent or mismatched, automation stops.

AI may select among declared paths.

AI may not synthesize an undeclared selector, page state, destination, or transition and treat it as verified.

External Links

A link leaving the verified controlled publisher scope is not silently traversed as though it remained part of the same assurance environment.

It is:

- blocked;
- presented through the existing external-link risk interaction;
- or opened in a separately classified environment.

The rendering class and provenance indication update accordingly.

Declared External Surface Handoff and Payment Execution (extension point)

Some declared procedures legitimately require a surface that is not the publisher's own. The canonical case is payment: a checkout, wallet, or authorization surface operated by an external payment provider — a bank, PayPal, Google Pay, or a card processor. Such a surface cannot be a publisher-signed passive template and is never rendered as R2 content.

The WR system does not process, store, or transit payment credentials. Card numbers, wallet credentials, and banking authentication remain exclusively between the operator and the payment provider. The provider is neither a superior nor a peer trust anchor of the WR system; it is the operator's own pre-existing external trust anchor, admitted into a governed procedure under the rules of this annex.

The crossing is governed as follows (normative):

1. **Whitelisted providers only.** A handoff may target only a payment provider the operator — or signed organization policy — has explicitly admitted. Admission is a PoAC event; the admitted provider set is evaluated per action; a destination that is undeclared in the navigation graph or unadmitted by the operator is refused by the governed network broker. No content, model output, or publisher artifact can introduce a provider at runtime (I3, I6).
2. **The boundary exit is explicit and visible.** Deterministic automation pauses at the declared handoff node. Before the crossing, the pointer and overlay present the transition as what it is — a deliberate exit from the deterministic trust boundary: the target provider, the verified destination origin, any data leaving WR, and the changed rendering class. The provider surface opens in a separately classified environment (R1 or an external application environment per §A.16) and is never visually blended into R2. While the operator is on the provider surface, the runtime asserts no page state, performs no automation, and makes no claims about what is displayed there.
3. **Re-entry only as a verifiable result.** The system takes back nothing from the provider except a verifiable payment result — a signed confirmation, token, or attestation conforming to the declared result schema. That result is validated deterministically at the effect boundary before the declared re-entry state is verified and deterministic execution inside the trust boundary resumes. The crossing and the result are receipted under the Boundary Event Record discipline; the consequential effect remains independently validated by the capability and the Finalizer.

This handoff is not an exception to the security model. The model never claims determinism over surfaces it does not control; it claims — and enforces — that leaving them is explicit, whitelisted, consent-gated, and evidence-producing, and that nothing from outside re-enters the boundary except as a validated result. The handoff is therefore a declared, transparently communicated system

boundary, not a relaxation of one. The detailed profile — permitted handoff destination classes, result schemas, and evidence requirements — remains a declared extension point (§XI.20).

Forms and Submissions

A form submission is permitted only where the relevant endpoint, schema, method, and capability are declared.

A consequential submission additionally requires:

- current session validity;
- account-context validation;
- typed values;
- WR Guard;
- the applicable consent;
- the appropriate Finalizer.

Page markup alone does not create submission authority.

R1 Navigation

In an external browser, the Pointer may:

- describe a path;
- highlight likely elements;
- open a verified destination;
- perform best-effort traversal.

The navigation sequence remains a convenience mechanism.

No consequential effect may depend on an assumption that the external page followed the declared visual path.

The Finalizer revalidates the actual requested effect independently.

A.10 Target Client Compartments

The integrated target client contains at least the following logical compartments.

Handshake Compartment

Contains:

- Public Handshake state;
- pBEAP relationship state;
- handshake keys;
- publisher verification state;
- publisher signing keys;
- manifest roots;
- repository references;
- PAMs;
- WCR definitions;
- schemas;
- public policies;

- public capability declarations;
- session-binding declarations.

It does not contain website passwords or unrestricted website cookies.

Session Compartment

Contains the publisher-origin authenticated session, including as applicable:

- HTTP-only cookies;
- same-origin cookies;
- CSRF state;
- publisher-origin local storage;
- server-side session references;
- login state;
- account-selection state;
- a non-exportable session-environment proof-of-possession key.

The Session Compartment is origin-isolated.

Its credential state is not exported to:

- the Handshake Compartment;
- the general orchestrator;
- WR Graph;
- local models;
- publisher adapters;
- support participants;
- other publisher origins.

Binding Coordinator

Coordinates proof that:

- the verified handshake relationship is present; and
- the live authenticated publisher session is present.

It combines or verifies the two challenge contributions without obtaining the website credential itself.

Controlled Renderer

Receives:

- admitted page templates;
- admitted assets;
- typed render values;
- permitted session-derived output;
- publisher-declared automation;
- runtime interaction primitives.

It receives no ambient WR Graph access, unrestricted file access, or unrestricted network access.

Governed Network Broker

Provides the sole permitted renderer network path.

Capability and Consent Components

Maintain:

- LBCPs;
- capability tokens;
- consent state;
- policy state;
- revocation state;
- Finalizer authorization.

Evidence Components

Maintain the applicable:

- PoAC records;
- PoAE records;
- Boundary Event Records;
- WR TED evidence;
- execution receipts;
- session receipts.

Logical separation may be implemented through:

- separate processes;
- sandboxed services;
- micro-VMs;
- protected storage domains;
- hardware-backed key isolation;
- equivalent enforceable boundaries.

A.11 Same-Origin Session Co-Presence

The permanent architectural requirement is not attachment to an external browser.

The permanent requirement is:

> The verified Public Handshake context and the environment holding the live authenticated publisher session must participate simultaneously in a fresh, origin-bound Binding Challenge, while the publisher credential remains inaccessible to the component holding the handshake authority.

External-Browser Embodiment

At R1, an attached browser component performs the same-origin session request from the external browser profile holding the cookie.

The extension or attached component participates in the proof.

The external browser remains an externally controlled rendering surface.

Integrated Target Embodiment

At R2:

- the Handshake Compartment holds the handshake-side contribution;
- the Session Compartment performs the same-origin session-gated operation;
- the Binding Coordinator proves co-presence;
- the orchestrator does not read or export the cookie.

The existing Dual-Possession model therefore remains intact.

Its target-architecture embodiment is Dual-Compartment Possession.

The security conjunction survives even though both compartments are integrated into one client product. Its residual limits in the integrated embodiment are stated openly in §A.19 (Integrated-Client Adversary).

A.12 Website Login, Cookies, and LBCP Mapping

The publisher's website login remains separate from the Public Handshake.

The Public Handshake verifies:

- publisher identity;
- declared automation;
- schemas;
- policies;
- prompts;
- WCR references;
- permitted actions;
- Finalizers;
- consent requirements;
- integration endpoints.

The website login authenticates the operator to a private publisher account or tenant.

The LBCP binds the live authenticated session to the already verified Public Handshake.

Cookie Confinement

A website cookie:

- remains in the Session Compartment;
- is used only for same-origin publisher requests;
- is not copied into the LBCP;
- is not converted into a WR bearer token;
- is not exposed to the model;
- is not exposed to the publisher adapter;
- is not exposed to the support participant.

The cookie authenticates the Session Compartment to the publisher.

It does not by itself:

- establish a Public Handshake;
- grant a Pointer Action;
- issue a capability;
- establish consent;
- authorize a Finalizer.

Derived Session Assertions

The Session Compartment may expose only derived, narrowly scoped assertions such as:

- session live;
- current session epoch;
- opaque subject binding;
- selected account or tenant;
- eligible capability identifiers;
- fresh challenge contribution;
- reauthentication required;
- logout or session-revocation event.

LBCP Field Generalization

Where an earlier LBCP example uses `browser_profile_binding`, that field shall be understood as one embodiment of the more general:

`session_environment_binding`

An updated LBCP profile may additionally include:

`renderer_instance_binding`

`rendering_assurance_class`

`proof_of_possession_key_ref`

`optional_installation_binding`

`optional_attestation_binding`

The exact encoding remains extensible.

Multiple Tabs and Renderer Instances

Several tabs or renderer instances do not automatically share all capabilities merely because they share a website cookie.

A capability may be reused across instances only where:

- the session-environment binding permits it;
- the account context is identical;
- the renderer instance is admitted;
- current liveness is verified;
- the capability scope permits reuse;
- the applicable consent remains valid.

Restart and Restoration

A surviving website cookie does not automatically restore WR account authority after:

- renderer failure;
- renderer restart;
- orchestrator restart;
- device restart;
- compartment recreation;
- loss of the proof-of-possession key.

Restoration requires verification of:

- the current website session;
- the current handshake hash;
- the current session epoch;
- the session environment;
- proof of possession;
- current policy;
- capability eligibility;
- revocation status.

A fresh Binding Challenge is required where continuous possession cannot be proven.

Pending consequential consent is not silently restored.

Account and Tenant Switching

Changing:

- account;
- tenant;
- organization;
- role;
- profile;
- workspace;

is a binding transition, not merely a visual page change.

The session epoch changes or the previous binding is otherwise invalidated.

Incompatible:

- slot values;
- capabilities;
- delegations;
- private channel state;
- pending consent;

are cleared or replaced.

Logout

Logout remains a cryptographic state change.

On logout or session termination:

- heartbeat renewal stops;

- the session epoch changes;
- the LBCP becomes invalid or starves;
- private session-channel keys are destroyed or orphaned;
- account-bound capabilities become unavailable;
- personalized Pointer Actions disappear or become non-invocable;
- private slot bindings are cleared;
- derived support delegations terminate.

The existence of a stale visual page does not preserve authority.

For CHANGE, PERSIST, EXECUTE, or sensitive TRANSMIT actions, the Finalizer may require an immediate session-status check or fresh challenge even where the normal heartbeat has not yet expired.

A.13 Optional Identity-Provider and SSO Role

The integrated client may act as an identity provider or authentication broker for WR-enabled services where the publisher elects to use that profile.

This may simplify:

- login initiation;
- phishing-resistant origin selection;
- account switching;
- session establishment;
- reauthentication;
- logout propagation;
- device-bound proof of possession.

Such a profile does not collapse the Public Handshake, website account login, LBCP, capability, and consent layers into one object.

Even where WR participates in SSO:

- the publisher remains authoritative for its account and entitlement;
- the Public Handshake remains the public capability contract;
- the LBCP remains the temporary session binding;
- capability and consent remain separately evaluated.

A publisher may continue to use its own existing login system without adopting WR as its identity provider.

A.14 Trusted Presentation, Intent Hash, and Egress Preview

Rendering Assurance and consent assurance remain separate.

At R0

The runtime controls the complete relevant presentation surface and may present:

- Intent Hash;
- Egress Preview;
- provenance;

- capability scope;
- execution status;

through the WR-owned surface.

At R1

The external browser or application may be compromised or misleading.

An in-surface highlight, preview, or status indication may therefore be informative without being sufficient as the trusted authorization surface.

Where required, approval moves to:

- a separate WR-owned view;
- an enrolled smartphone;
- a platform authenticator;
- a hardware key;
- another admitted out-of-band approver.

At R2

The controlled renderer and separately rendered WR authorization elements may provide a trustworthy in-client presentation of:

- publisher identity;
- account context;
- exact action;
- parameters;
- context digest;
- recipient;
- transformation;
- Finalizer;
- consent requirement.

Publisher-supplied repository artifacts may provide action-specific explanations, Overlay content, and preview templates.

They may not:

- suppress provenance;
- falsify origin;
- lower the consent floor;
- lower the Rendering Assurance requirement;
- alter the Intent Hash after approval;
- conceal a boundary crossing;
- represent an unapproved proposal as authorized;
- replace the runtime's authorization and evidence mechanisms.

A controlled renderer improves trusted presentation.

It does not reduce the consent-class floor declared by the publisher, organization, or operator.

A.15 Classification of Existing Worked Examples

Existing Pointer, Overlay, navigation, and WCR examples remain valid, but their rendering embodiment shall be made explicit.

Deterministic Guided Navigation

An example in which the Pointer traverses a declared navigation chain, verifies each state, and reliably reaches a declared destination is an R2 controlled-renderer embodiment.

The runtime verifies:

- the expected page-template hash;
- the declared state;
- the relevant slots;
- the permitted transition;
- the expected destination.

An unexpected state stops the automation.

External-Browser Guidance

The corresponding R1 embodiment may:

- explain the path;
- highlight likely controls;
- open a verified destination;
- attempt best-effort traversal.

It does not claim deterministic page navigation.

WCR Website Procedure

A WCR may coordinate a complete R2 website procedure where:

- page states are pinned;
- branches are declared;
- slots are typed;
- navigation transitions are verified;
- Finalizers remain separately governed.

At R1, the same WCR may assist the user, but the page traversal is not the security boundary. Before a consequence, the effect is independently revalidated at the publisher endpoint and Finalizer.

Egress Preview

An in-page or Overlay Egress Preview is a trusted authorization input only where the applicable presentation environment satisfies the declared requirement.

At R1, the stronger approval may be moved out of the external browser.

A.16 Native Applications and Other Installed Software

Native mobile applications, desktop applications, proprietary clients, third-party browsers, productivity tools, enterprise software, and legacy software are treated as one general category: **external execution environments outside the controlled WR rendering and enforcement boundary.**

The decisive question is not whether the environment is called a browser, app, desktop program, or client.

The decisive question is whether WR can control and verify its:

- identity;
- rendering;
- execution;
- storage;
- network access;
- authorization presentation;
- effect path.

External Application Handshake

An optional External Application Handshake may bind:

- application identifier;
- package or bundle identifier;
- code-signing identity;
- publisher identity;
- backend origin;
- protocol version;
- installation public key;
- optional device binding;
- optional hardware attestation;
- declared capabilities;
- request schemas;
- response schemas;
- callback endpoints;
- consent requirements;
- expected receipt form;
- update path;
- revocation path.

Boundary-Verifiable Properties

The handshake may establish:

- which declared application identity participated;
- which publisher signed the application;
- which installation key participated;
- which backend origin is admitted;

- which request left WR;
- which values crossed the boundary;
- which capability and consent applied;
- which signed response returned;
- which receipt was issued.

Internally Unverifiable Properties

Absent a controlled execution environment, the handshake does not prove:

- what the application displayed;
- whether it displayed the correct intent;
- which internal code path it executed;
- whether it modified data internally;
- whether it copied or leaked data through another channel;
- whether hidden functionality ran;
- whether an internal AI followed policy;
- whether a signed receipt truthfully describes internal execution.

A valid signature proves which key produced a statement.

It does not by itself prove that the statement is true.

Application Session Binding

Where a native app holds the authenticated publisher session, an adapted split challenge may be used.

The handshake-side contribution reaches the WR client through the verified Public Handshake relationship.

The app-session contribution is completed inside the authenticated application environment.

The publisher issues an application-session-bound proof only where:

- the app identity is admitted;
- the publisher session is live;
- the installation or environment binding matches;
- the handshake version matches;
- the session epoch is current;
- capability eligibility is valid.

WR need not receive the application's bearer token.

The resulting proof establishes boundary co-presence and the declared identity relationship.

It does not transform the external application into an R2 controlled renderer.

External Software Handoff

A Pointer Action crossing into external software shall present:

- target software;
- verified or unverified software identity;
- data leaving WR;

- transformation applied;
- expected return;
- location of the actual consequence;
- expected receipt;
- residual unverifiable behavior.

Stronger Software Embodiments

Stronger claims may be made where software runs inside an enforceable environment such as:

- a WR-controlled micro-VM;
- a controlled container;
- a managed device profile;
- a remote controlled renderer;
- an instrumented enterprise runtime;
- an OS-enforced sandbox;
- an attested trusted application environment.

The strength of the claim remains limited to what the environment actually enforces.

A signed or attested application is not automatically equivalent to R2.

Architectural Priority

Native application and general-software support are completeness extensions.

The primary target remains the integrated orchestrator and controlled browser because it provides the most direct architecture for:

- Public Handshake discovery;
- publisher-origin verification;
- deterministic website rendering;
- account-session binding;
- Pointer interaction;
- Overlay presentation;
- governed navigation;
- trusted intent presentation.

A.17 Minimal Publisher Integration

The target architecture does not require the publisher to replace its authentication system.

A minimal publisher-side integration may be implemented as:

- a root-domain script;
- middleware;
- CMS plugin;
- reverse-proxy component;
- identity-provider integration;
- API endpoint;
- application-platform component.

The integration may provide:

- Public Handshake discovery;
- publisher key declaration;
- Site Manifest discovery;
- session validation;
- split Binding Challenge;
- LBCP issuance;
- heartbeat;
- session epoch;
- logout and account-switch signaling;
- typed slot resolution;
- Finalizer endpoints;
- execution-status verification.

A minimal universal integration profile may use a standardized file or endpoint deployed under the verified publisher origin.

For systems such as WordPress, Joomla, or comparable platforms, the same profile may be packaged as a plugin.

For custom systems, the same functions may be exposed through middleware or an API endpoint.

A static discovery file alone cannot prove a live authenticated session. Session binding requires a component able to query or participate in the publisher's actual authenticated session state.

A.18 Centralized Verification, Relay, and Peer-to-Peer Profiles

Central Public Infrastructure

An optional Optirando-operated service may provide:

- publisher registration;
- DNS-verification evidence;
- public key discovery;
- manifest distribution;
- repository hosting;
- normalized image hosting;
- revocation status;
- protocol negotiation;
- relay discovery.

This service may simplify deployment and allow an end user to discover and verify a publisher without manually installing publisher-specific components.

It does not replace the publisher's authority over:

- account login;
- account identity;
- tenant membership;
- entitlement;
- website session;
- logout;

- account-scoped Finalizers.

A centralized Public Handshake service cannot truthfully assert that a publisher account session is live unless the publisher's authenticated backend participates.

Peer-to-Peer or Direct Transport

After publisher identity and the Public Handshake have been verified, the parties may use:

- a direct authenticated channel;
- peer-to-peer transport;
- a rendezvous mechanism;
- an encrypted relay;
- a publisher-hosted channel.

Transport choice does not change:

- publisher identity;
- handshake version;
- session-binding requirements;
- capability scope;
- consent;
- WR Guard;
- revocation;
- evidence.

P2P is a transport profile, not an authority model.

Installation-Free Profile

An ordinary-browser or hosted profile may offer a reduced-function Public Handshake experience without a locally installed WR client.

That profile may provide:

- publisher verification;
- declared actions;
- account login;
- reduced interaction;
- server-side Finalizers.

It cannot claim the full guarantees of the integrated controlled renderer unless the rendering and enforcement boundary has been placed in another equivalently controlled environment.

Any reduced guarantee must be declared visibly and cryptographically in the applicable profile.

A.19 Threat Classes and Honest Boundary

The architecture distinguishes three different classes of assurance.

Publisher Identity

The Public Handshake and origin verification establish who published the declared artifacts and which origin is bound to them.

They do not prove that the publisher is honest.

Artifact and Structural Integrity

Repository admission, canonicalization, normalization, hashes, manifests, and Merkle binding establish which admitted artifact is being used.

For images, the client verifies the normalized representation before decoding.

For declarative text, the runtime accepts only the admitted grammar and canonical representation.

For safetensors, the runtime accepts only the admitted non-executable tensor format and declared scope.

Structural integrity does not prove semantic honesty.

Semantic and Behavioral Honesty

A verified publisher may still provide:

- false information;
- manipulative content;
- deceptive explanations;
- misleading automation;
- formally valid but harmful suggestions;
- maliciously tuned adapters.

Scam Watchdog, provenance, policy, review, revocation, and publisher accountability address this class.

These mechanisms are complementary:

- identity verification does not replace normalization;
- normalization does not replace semantic analysis;
- semantic analysis does not replace deterministic effect validation.

Integrated-Client Adversary (Honest Boundary)

In the integrated target architecture, the independence argument of the Dual-Possession construction (Login-Bound §2.1) changes character. In the external-browser embodiment, the handshake contribution and the session contribution live in independently controlled products; in the integrated client they live in compartments of one product. Dual-Compartment Possession is therefore exactly as strong as the isolation mechanisms of §A.10 that separate the compartments — and no stronger. An adversary who compromises the orchestrator, the client's process boundary, or the build and update path at sufficient privilege holds both contributions simultaneously and is not excluded by the binding construction itself.

Mitigations at this layer are containment, not immunity: enforceable compartment boundaries (§A.10), hardware-backed proof-of-possession keys, and consent escalation that places the authorization event and its intent display outside the potentially compromised surface (Login-Bound §14). Genuine exclusion of this adversary belongs to OS-level enforcement, micro-VM isolation, and attested execution (§A.20). This subsection states, for the integrated embodiment, the

same class of boundary that Login-Bound §16 states for a hostile extension inside the external browser.

Removed or Restricted Paths in R2

The strict R2 profile removes or restricts:

- arbitrary publisher script execution;
- arbitrary publisher WASM;
- undeclared dynamic code;
- undeclared network access;
- arbitrary third-party embeds;
- unverified image substitution;
- untyped dynamic markup insertion;
- content-created actions;
- content-created capabilities;
- content-created Finalizers;
- arbitrary publisher persistence outside the Session Compartment.

Relocated Parsing Risk

Publisher-original image parsing is relocated to the isolated Optirando ingest environment.

The client decodes only the normalized, admitted, digest-matching output.

The ingest environment therefore becomes a separately hardened boundary that may use:

- process isolation;
- restricted privileges;
- resource limits;
- fuzz testing;
- deterministic encoders;
- versioned normalizers;
- admission evidence.

Remaining Technical Risks

Residual risks include:

- bugs in WR-owned code;
- bugs in the controlled renderer;
- bugs in the declarative parser;
- bugs in decoders processing admitted outputs;
- network-broker errors;
- compartment-isolation errors;
- signing-key compromise;
- repository compromise;
- build or update compromise;
- operating-system compromise;
- driver or hardware compromise;
- side channels below the WR layer.

The architecture does not claim absolute security.

Honest Structural Claim

The strict architecture supports the following claim:

> Publisher-specific automation is supplied through signed, versioned, declarative repository artifacts. Publisher-original images are interpreted only inside the isolated ingest environment and are replaced by normalized, hash-bound representations. The client interprets only admitted canonical artifact classes; dynamic values enter through typed render slots; arbitrary publisher executable code is excluded; network access and consequential effects remain capability-governed.

The controlled renderer proves admitted structure and artifact integrity within its boundary.

It does not prove that displayed claims are true or that the publisher is benevolent.

A.20 Staged Implementation and Target Mapping

The existing staged implementation is refined by the following assurance mapping.

Existing Phase 1 — R0

WR-owned surfaces, including:

- TimesDesk Frames;
- composed infographics;
- BEAP views;
- Editable Regions;
- WR-owned consent and configuration views.

The complete governance chain can be exercised with full control of the relevant composition.

Existing Phase 2 — R1

External-browser integration through:

- extension-based Pointer;
- Region Marker;
- Public Handshake actions;
- provenance display;
- login-bound session proof;
- server-side Finalizers;
- out-of-band approval where required.

No deterministic third-party website rendering is claimed.

Target Mobile Phase — R2

The integrated orchestrator and controlled mobile browser introduce:

- Handshake Compartment;
- Session Compartment;
- Binding Coordinator;

- controlled renderer;
- network broker;
- passive templates;
- normalized assets;
- typed render slots;
- publisher repository automation;
- controlled navigation;
- trusted in-client intent presentation.

An Android embedded-browser implementation is one possible embodiment.

Target Desktop Phase — R2

The integrated desktop browser provides the same architecture on desktop.

The extension and external-browser bridge cease to be load-bearing components of the target deployment.

Later Cross-Application and OS Hardening

Later phases may add:

- managed external software;
- application-spanning designation;
- micro-VM execution;
- OS-level network mediation;
- token-based ingress and egress enforcement;
- hardware-backed keys;
- process-level capability checks;
- stronger device attestation.

These later phases strengthen cross-application enforcement.

They do not replace the controlled-browser target analysis.

A.21 Normative Clarifications and Superseding Notes

Browser Attachment

Any earlier statement that WR Desk must always be attached to an external browser is superseded as a permanent architectural invariant.

The retained security requirement is Same-Origin Session Co-Presence.

The external browser is one embodiment.

The integrated Session Compartment is the target embodiment.

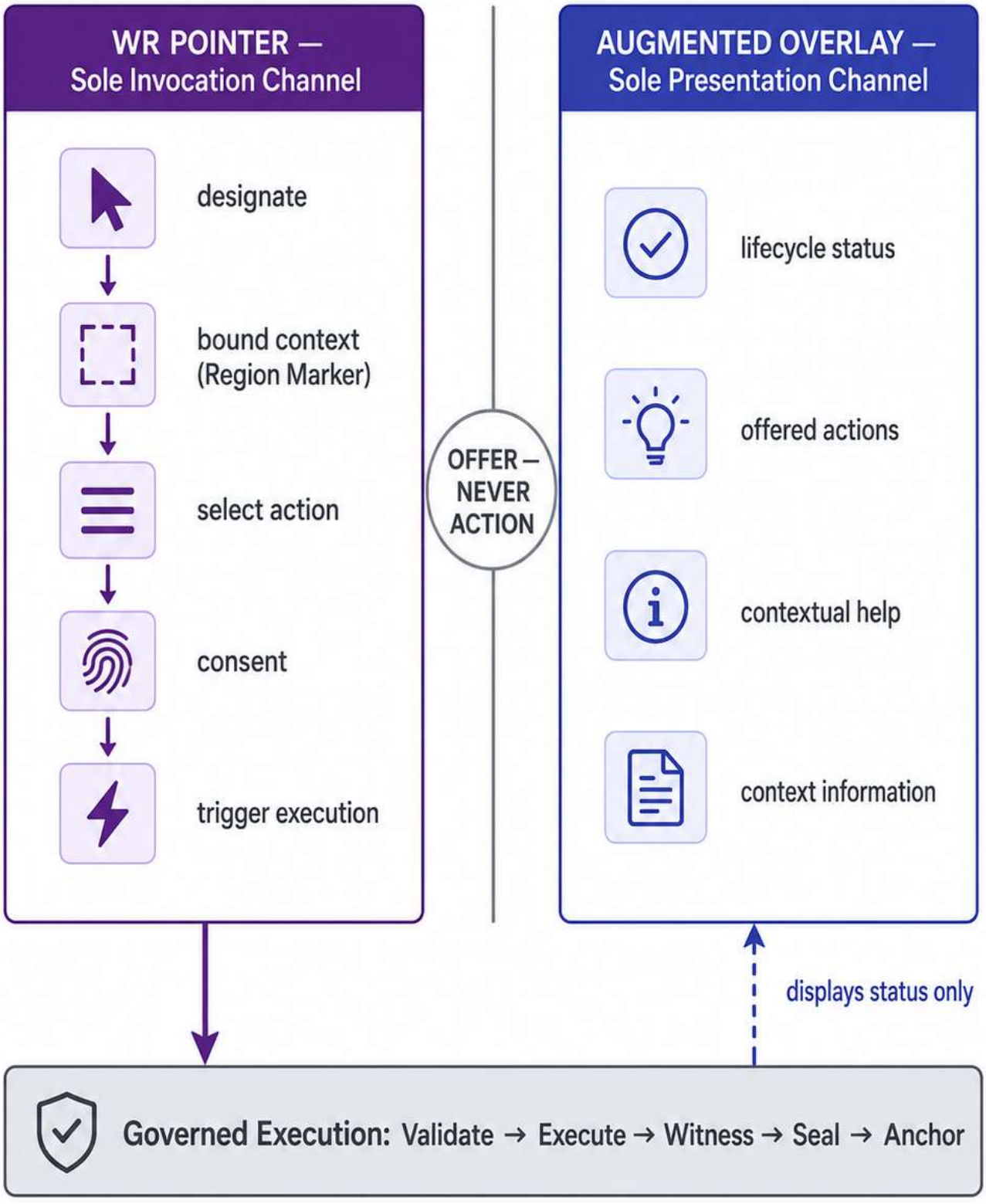
Interaction Logic

Any earlier statement that interaction logic is supplied exclusively by the host runtime shall be interpreted as follows:

- publisher-specific automation semantics may be supplied through signed declarative

indication and verifiable-result re-entry;
- separate Handshake and Session Compartments;
- website-cookie confinement;
- Same-Origin Session Co-Presence;
- Dual-Compartment Possession;
- LBCP binding to session environment, renderer instance, epoch, and proof of possession;
- restart, multi-tab, account-switching, heartbeat, logout, and revocation discipline;
- optional SSO or identity-provider participation without collapsing the Public Handshake and login layers;
- trusted Intent Hash and Egress Preview presentation according to rendering class;
- R2 deterministic navigation and R1 best-effort guidance;
- native applications and other installed software as secondary external execution environments;
- optional application-identity and application-session handshakes with explicitly limited behavioral proof;
- stronger software assurance only through enforceable containers, micro-VMs, managed runtimes, or OS integration;
- optional centralized discovery, relay, P2P, and installation-free profiles;
- separate treatment of publisher identity, artifact integrity, and semantic honesty;
- an honest boundary that identifies remaining WR-code, infrastructure, OS, and publisher-misconduct risks — including the integrated-client adversary;
- a staged path from existing R0 and R1 embodiments to the R2 mobile and desktop target architecture.

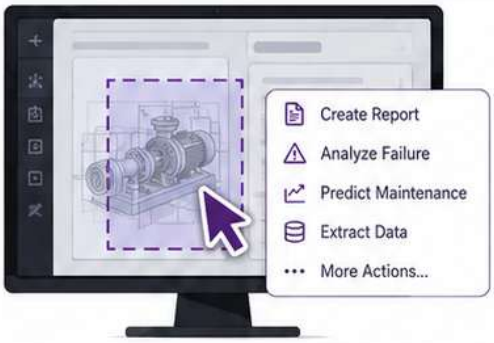
WR Pointer & Augmented Overlay — Two Channels, One Architecture



The overlay carries no execution path — fake dialogs cannot act.

One Governance Architecture — Device-Adaptive Embodiments

DESKTOP — AI Cursor Pointer



- pixel-precise designation
- Region Marker (bounded context)
- deep-work surface

identical governance

MOBILE — Real-World Pointer

- camera as pointing finger
- WR Code® scan (entry + context tag)
- evidence capture (photo · video · audio)
- confirmed frame = bounded context

Common Real-World Actions

- Create Report
- Request Support
- Analyze Failure
- Attach Evidence
- Log Observation
- Escalate Issue

identical governance

Uniform Core:
Action Registry · Manifests
Origin Classes
Consent C0–C4
Evidence Chain

identical governance

Tablet: hybrid + stylus

touch · stylus · on-screen pointer

future device classes (extension point)

MOBILE — On-Screen Pointer

- swipe action menu (context-aware)
- add global or context-scoped actions

- Create Report
- Analyze Failure
- Predict Maintenance
- Log Observation
- Export Data
- More Actions...

End-to-End Governed Actions (Examples)

Create Report → Analyze Failure → Predict Maintenance → Extract Data → Request Support → Escalate Issue → Close & Archive

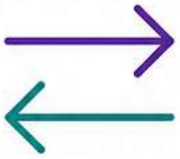
Only the interaction form varies — never the governance.

Nine Core Invariants (I1–I9)

I1 **Trigger, not authority**

the pointer never owns authority.

I2 **Pointer invokes, overlay presents**



offer-never-action.

I3 **Content is data, never instruction**

screen and camera content cannot create actions.

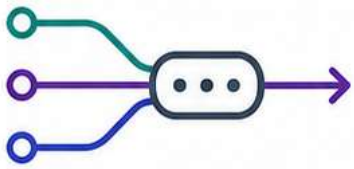
I4 **Signed origin classes**

every action shows verified provenance.

I5 **No hidden path**

every boundary crossing cryptographically permitted.

I6 **AI as router**

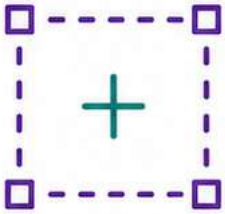


selects among admitted actions, never synthesizes paths.

I7 **Deterministic effect boundary**

safety lives in validation, not reasoning.

I8 **Explicit context bounding**



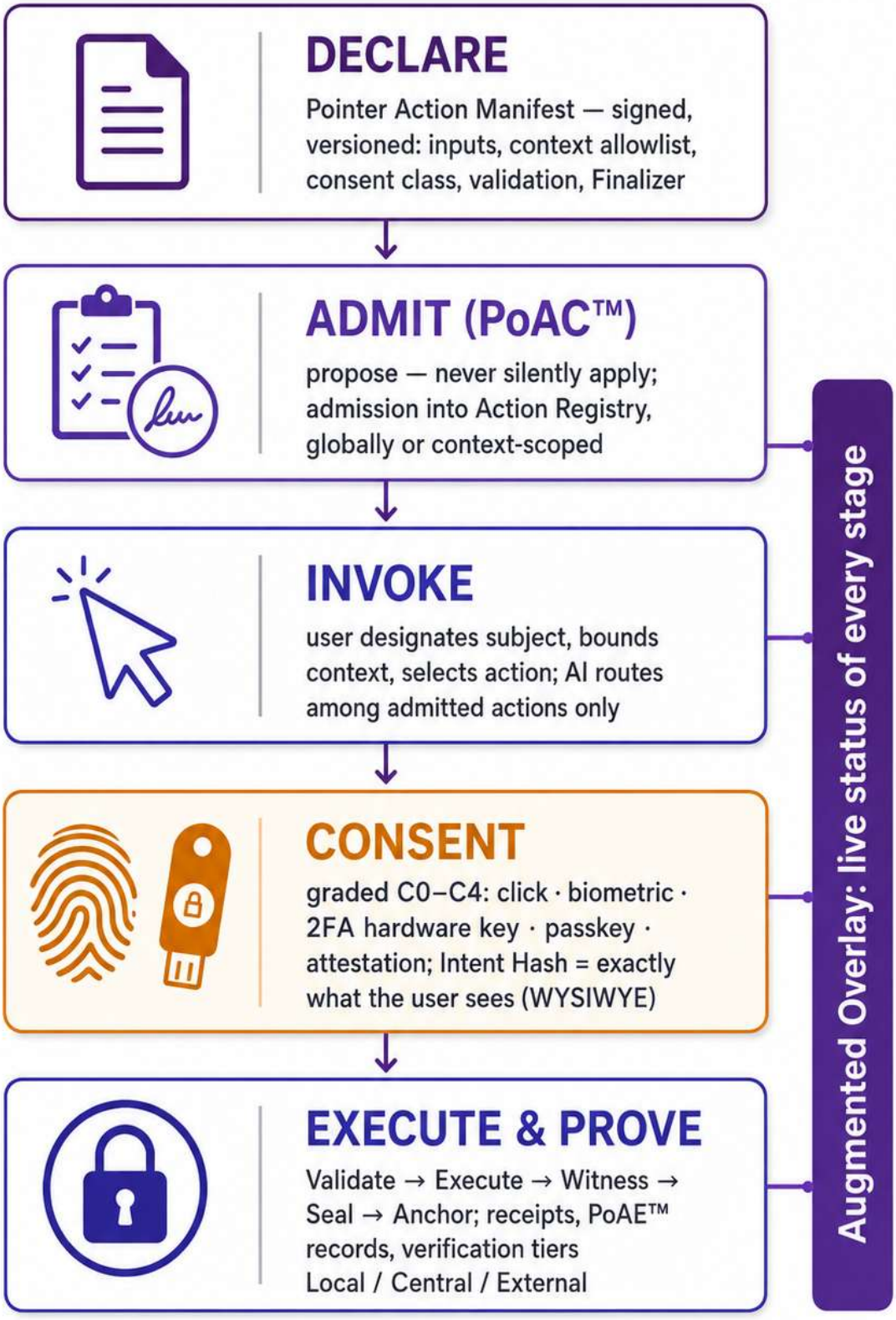
the marked region is what the user authorizes.

I9 **Uniform across devices**

no weaker regime for any form factor.

Amendments may strengthen invariants — never weaken them.

From Declaration to Cryptographic Proof — the Pointer Action Lifecycle



Revocation is immediate; updates are versioned — never silent.

The Region Marker — Context Bounding as a Governance Instrument

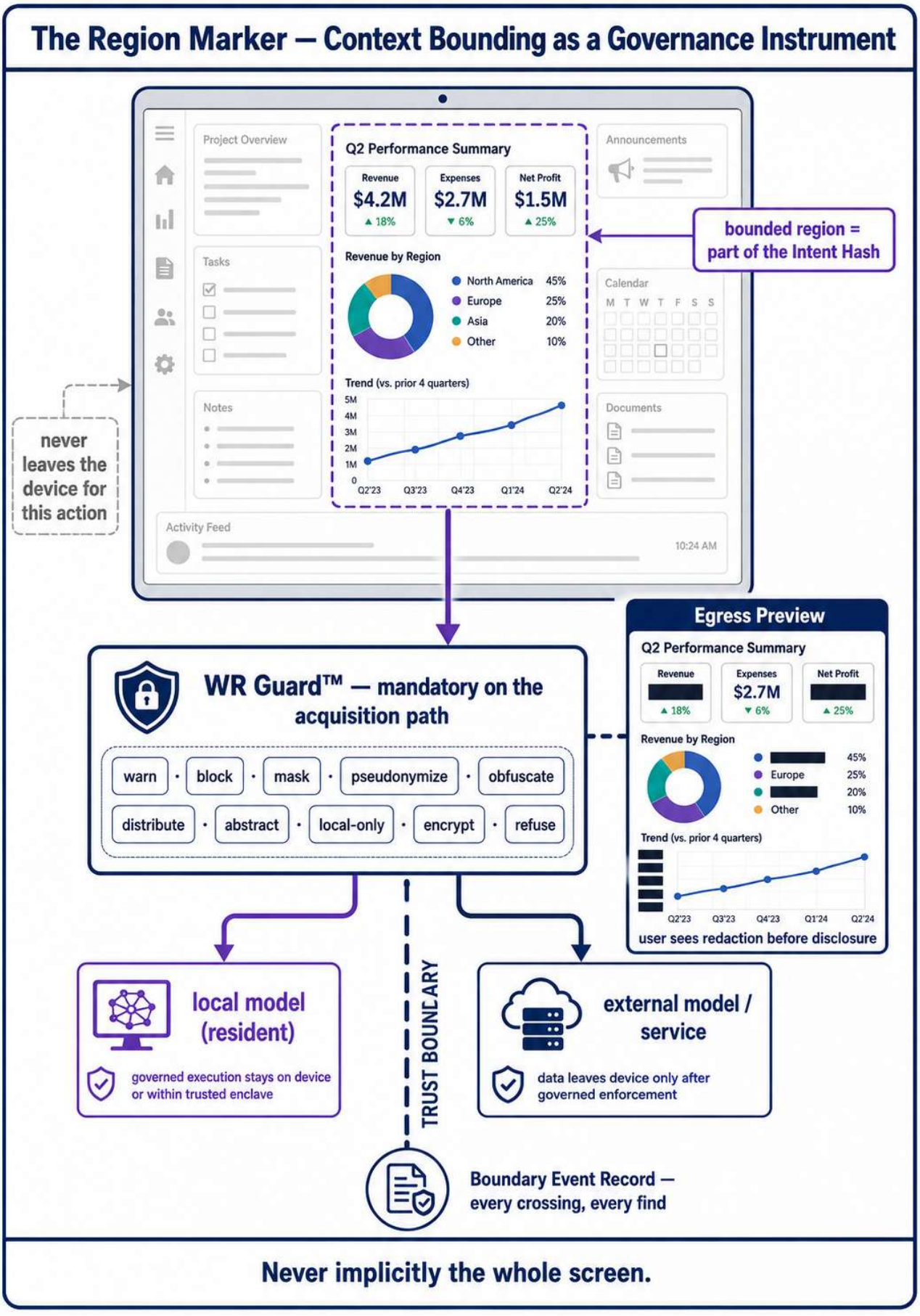
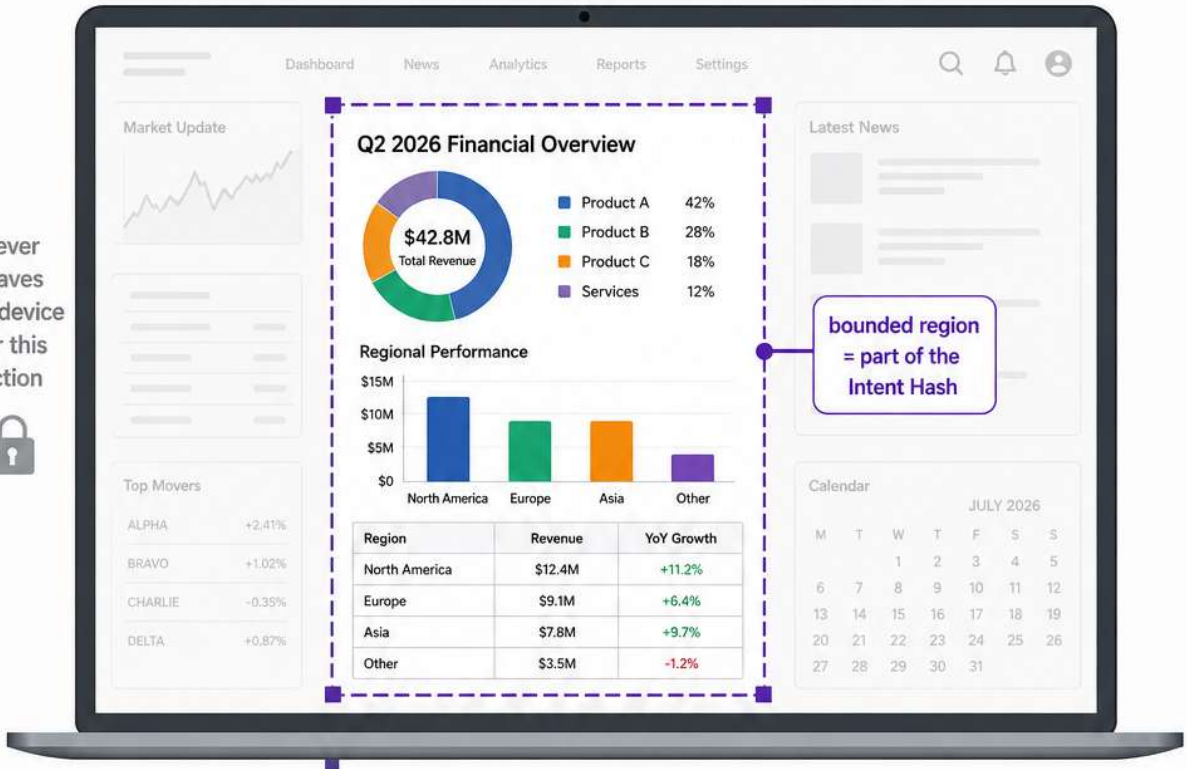


Figure 5 — Region Marker & Governed Context Acquisition

The Region Marker — Context Bounding as a Governance Instrument



never leaves the device for this action

bounded region = part of the Intent Hash

WR Guard™ — mandatory on the acquisition path

warn · block · mask · pseudonymize · obfuscate

distribute · abstract · local-only · encrypt · refuse

Egress Preview

Q2 2026 Financial Overview

$**.*M
Total Revenue

Product A **%
Product B **%
Product C **%
Services **%

Region	Revenue	YoY Growth
North America	$**.*M	+**.*%
Europe	$**.*M	+**.*%
Asia	$**.*M	+**.*%
Other	$**.*M	-*.*%

user sees redaction before disclosure

local model (resident)

stays within the device. No crossing.

TRUST BOUNDARY

external model / service

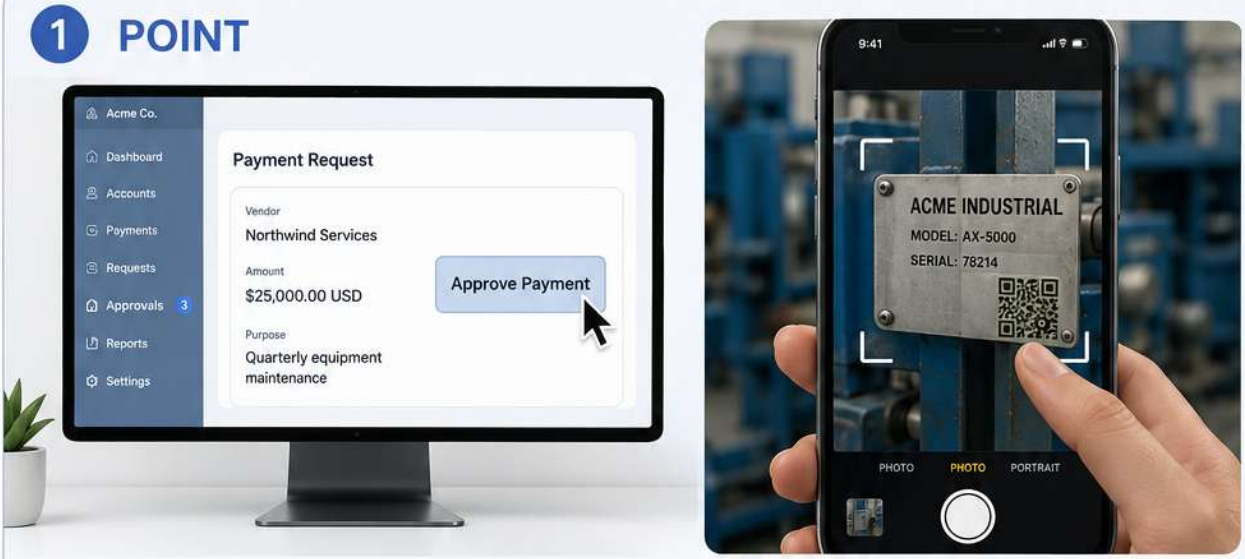
crosses the trust boundary only with governed disclosure.

Boundary Event Record — every crossing, every find

Never implicitly the whole screen.

Designate → Interact → Continue

1 POINT



screen or real world — the designated subject

2 INTERACT — four modalities, one governance

Speech	Text	Handshake-based AI automation	Self-predefined AI automation

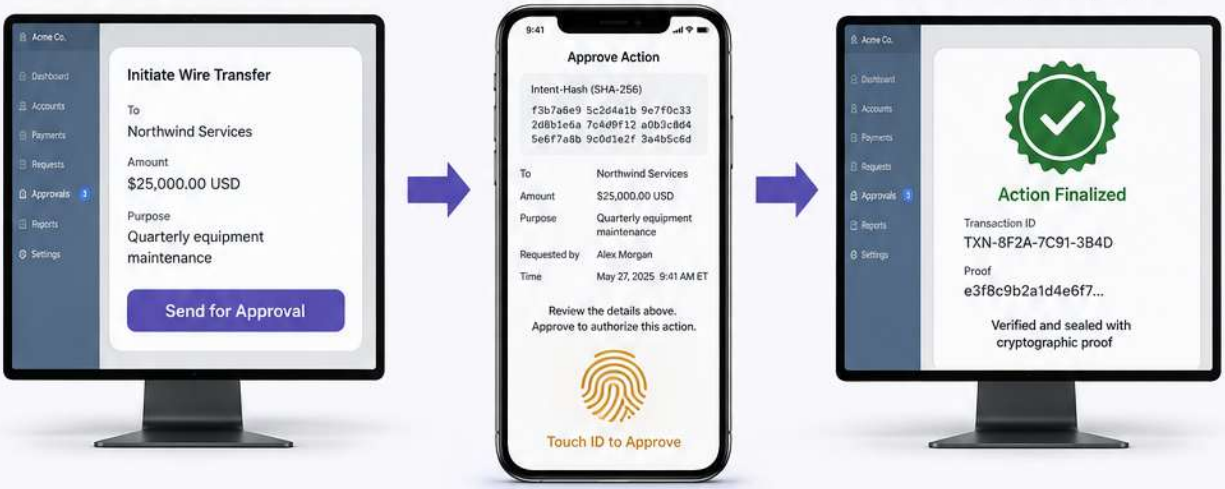
same consent class, same bounded context, same evidence — modality-independent

3 CONTINUE — Phone-as-Approver

Desktop initiates consequential action

Phone presents full-screen approval with Intent-Hash

Back to desktop — finalized with cryptographic proof



The smartphone is the capture and consent device;
the desktop is the composition surface.

The Pointer Is the Trigger

Public handshake types and local pointer capabilities under one pointer

1 HANDSHAKE TYPES

Publisher Website

the publisher tailors the pointer to its offering with high precision — deterministically declared in advance, never improvised at runtime

PUBLIC HANDSHAKE —
cryptographic capability contract

- declared pointer actions
- schemas & validation
- consent classes C0–C4
- LoRA adapter (domain-tuned)
- overlay help assets
- signed Merkle manifest

ENTERPRISE HANDSHAKE
private capability contract

- tenant-scoped actions
- private validation
- enterprise policy classes
- signed manifest

PARTNER / ORGANIZATION HANDSHAKE
crganization-scoped capability contract

- approved actions
- policy-bound schemas
- organization overlays
- signed manifest

2 POINTER + LOCAL CAPABILITIES

LOCAL CAPABILITIES

- user-created actions
- WRC recordings
- organization policy actions

WR Pointer — invocation trigger only

one unified action menu, color-coded provenance

handshake-declared and local actions appear together, but provenance stays visible

3 PROVENANCE SUMMARY

one pointer, one menu — multiple governed capability origins

- Public Handshake
- Enterprise Handshake
- Partner / Organization Handshake
- Local Capabilities

Public and private handshake capabilities, plus local pointer capabilities — unified under one pointer with visible provenance.

Login-Bound Personalized Handshakes, Dynamic Account Context, and Governed Remote Assistance

Editorial status. Addition set v1.1, integrated into the Optirando™ Canon with Annex XI v1.1. Internal section numbers (§1–§18) are final within this section and are cited from the rest of this annex with the prefix "Login-Bound" (e.g., Login-Bound §6). The base annex uses §XI.n; Addition Blocks A and B use §A.n and §B.n. The text is written as a technical defensive publication: it discloses an implementation-independent security model, its required properties, its end-to-end lifecycle, and its attack resistance. It does not advertise a product and asserts no absolute security guarantees. Registered and claimed designations (Optirando™, BEAP™, pBEAP™, WR Desk™, WR Graph™, WR Guard™, WR Expert™, WR Code®, PoAE™, PoAC™, Scam Watchdog™) are used as defined elsewhere in the Canon.

Changes vs. v1.0. The binding architecture is rebuilt around Dual-Possession Session Binding with a split Binding Challenge delivered through the pBEAP™ channel and completed through the authenticated session; the Login-Bound Capability Proof is now an issued artifact whose validity is sustained by a challenge heartbeat; the Identifier-Only Slot Discipline with exactly two slot classes is introduced as a normative invariant; Attestation-Adaptive Consent Escalation (optional upward, floor-bounded downward) is added to §14; the declared automation artifacts are consistently designated AI automation capabilities; the WR Desk™ browser-attachment invariant is stated (superseded in part by §B.2); WR Code® is repositioned as the cross-device path; the attack catalogue is extended accordingly.

Scope and Layer Separation

This section describes how an already established Public Handshake can be cryptographically connected to a user's authenticated website session — without turning private account data, session identifiers, credentials, or personalized AI automation capabilities into public handshake content.

The purpose of this connection is precise: **login-bound personalization exists to enable fine-grained, bidirectional, pre-declared and negotiated AI automation capabilities.** These AI automation capabilities live as artifacts in the handshake repository like every other declared artifact; what login adds is not new artifacts but the temporary, session-scoped authority to bind those capabilities to one authenticated account — through identifier slots that are enforced to carry identifiers and nothing else (§6).

The architecture is deliberately general-purpose. Customer support is used as the running example because it exercises every mechanism at once (personalized data, delegated assistance, voice channels, consequential changes), but the identical model applies to banking portals, insurance portals, healthcare portals, government services, enterprise applications, e-commerce accounts, subscription services, technical maintenance portals, and any other authenticated digital environment in which a verified publisher offers declared, governed capabilities to a logged-in user.

Architectural precondition (superseded in part — see §B.2). The load-bearing requirement of the binding architecture of §2 is Same-Origin Session Co-Presence: the verified handshake context

and the environment holding the live authenticated publisher session must participate simultaneously in a fresh, origin-bound Binding Challenge, while the publisher credential remains inaccessible to the component holding the handshake authority. In the external-browser embodiment, this is realized by a browser-attached WR Desk component issuing same-origin requests inside the browser that holds the authenticated session. In the integrated target architecture, the identical requirement is realized as Dual-Compartment Possession (§B.2). Permanent attachment to an external browser is therefore one embodiment of the requirement, not an architectural invariant.

The website login itself is not the Public Handshake. The architecture consists of separate but cryptographically bindable layers:

1. The **Public Handshake** establishes the publisher's verified identity, publicly declared capabilities, schemas, policies, prompts, WCR references, permitted actions, Finalizers, consent classes, and the technical integration contract. It is public, reusable, and identical for every operator.
2. The **website login** authenticates the user to a specific private account or tenant, using the website's ordinary authentication mechanism.
3. A temporary **Login-Bound Capability Proof (LBCP)**, issued upon completion of a Dual-Possession Binding Challenge, cryptographically links the authenticated session to the already verified Public Handshake.
4. The private binding remains valid **only while the authenticated session remains valid**, enforced structurally through the challenge heartbeat (§5).
5. **Logout, session termination, revocation, account switching, or expiry** must make the private binding unusable.

Throughout this section, the following concerns are treated as distinct and are never collapsed into one another:

- **publisher authentication** — proof that the publisher origin, keys, and manifest are what the handshake declares;
- **operator authentication** — proof of the local operator identity or session context on the WR client side;
- **website account authentication** — the website's own login establishing an account or tenant session;
- **session binding** — the cryptographic linkage between the authenticated website session and a specific verified handshake;
- **capability authorization** — the issuance of a scoped, expiring right to invoke a declared capability;
- **consent** — the graded human authorization event (or admitted standing policy) permitting a consequence class;
- **execution** — the monitored performance of a declared function;
- **data access** — the reading of account-scoped values through typed, scope-restricted interfaces;
- **data transmission** — the crossing of a trust boundary by any value, always subject to WR Guard™;
- **revocation** — the unilateral, cryptographically effective withdrawal of any of the above.

Login answers exactly one of these questions. It answers none of the others.

§1 Core Architectural Principle

> The Public Handshake remains public and reusable. Personalized account authority is added only through a separate, temporary, login-bound cryptographic binding.

This principle is the load-bearing wall of the section. Everything that follows is a consequence of refusing to mix two lifecycles that have nothing in common: the Public Handshake is long-lived, publisher-signed, publicly inspectable, and identical for all operators; account authority is short-lived, session-specific, private, and different for every login. A design that writes private values into the public layer inherits the worst properties of both — a long-lived artifact carrying short-lived secrets.

The public handshake layer — the Public Handshake and its repository artifacts — may contain:

- public AI prompts;
- public AI automation capability definitions (including bidirectional declarations — publisher-initiated and operator-initiated);
- public WCR recordings or WCR hashes;
- declared pointer actions;
- capability schemas;
- expected identifier types and slot-class declarations (§6);
- validation rules;
- consent classes;
- publicly inspectable Finalizer definitions;
- policy references;
- publisher signatures;
- manifest hashes (including Merkle manifest roots);
- the publisher's session-binding key or binding endpoint declaration (§2.2);
- supported dynamic-data slots;
- optionally, an **encrypted personalization layer**: login-bound action templates shipped sealed, whose unlocking material only ever exists inside a live binding (§7).

The public layer must not contain:

- the user's private account identifier;
- credentials;
- cookies;
- bearer tokens;
- session secrets;
- personalized account data;
- private support records;
- private transaction data;
- private telephone transcripts;
- dynamically resolved personal values of any kind.

The public definition may declare that a function expects an identifier of a named class, such as order_id, contract_id, ticket_id, invoice_id, device_id, case_id, customer_record, selected_account, or authenticated_subject. The declaration names the slot, its slot class, its validation rules, and its

permitted operations. The value of the slot is never part of the handshake. It is resolved only from the authenticated website context and is placed into a separate private session binding (§6). The public handshake thereby declares what may exist; it never records what currently is for any particular account.

This separation also preserves the reusability guarantees of the Canon: the same Public Handshake serves an anonymous visitor, a logged-in retail customer, an enterprise tenant administrator, and a support representative — each of whom receives a different, separately issued, separately expiring private binding on top of the identical public contract.

§2 Dual-Possession Session Binding and the Login-Bound Capability Proof

§2.1 The Binding Mechanism

The linkage between the authenticated session and the verified handshake is established through **Dual-Possession Session Binding**. The construction exploits the fact that two independently gated channels already exist after Phase A and Phase B of the lifecycle (§4):

- the **pBEAP™ handshake channel** — a persistent P2P relationship into which only the publisher can deliver, and whose deliveries only a WR Desk holding the handshake keys can read;
- the **authenticated web session** — behind which only a browser holding the live session cookie can fetch.

The publisher's website-side integration component (§3) generates a **Binding Challenge**: a fresh, single-use, nonce-based object that is deliberately **split across both channels**. One part is encrypted to the handshake relationship and delivered through the pBEAP channel; the counterpart required to complete it is retrievable only from a session-gated endpoint on the publisher's origin. Only an entity that possesses both simultaneously — the handshake keys and the live authenticated session — can assemble the completed challenge.

Because WR Desk is always browser-attached in the external-browser embodiment, completion is a same-origin operation: the browser-attached WR component fetches the session-gated counterpart from inside the browser that holds the session; the browser attaches the session cookie automatically. WR Desk never sees, exports, stores, or handles the credential. **The cookie authenticates the browser to the website, exactly as it always did**; WR Desk merely proves that it operates inside the browser profile to which that cookie belongs. This is co-presence proof by construction, not by attestation. (In the integrated target architecture, the same construction is realized between the Handshake Compartment and the Session Compartment, §B.2.)

The security consequences of the split are structural:

- **Neither channel alone suffices.** A stolen session cookie without the handshake keys cannot decrypt the pBEAP-delivered part. A WR Desk holding the handshake but attached to a browser with no login cannot fetch the counterpart. The binding is the intersection of the two possessions; there is no single portable secret whose theft equals the binding.

- **Phishing has no delivery path.** The challenge arrives through the pBEAP channel, into which a look-alike site cannot deliver. A counterfeit "log in to unlock your personalized actions" page can imitate the session side but can never produce the handshake side.
- **Binding scope follows the browser profile** (external-browser embodiment; generalized as the session environment, §B.4). Extension state and session cookie both live in the browser profile; two profiles yield two independent bindings; a private-browsing window without a session simply never completes a challenge. Profile boundaries become binding boundaries without additional machinery.
- **Doctrinal consistency.** Delivering an encrypted challenge is delivery — squarely within the pBEAP grant model (delivery and preparation rights only). And the challenge itself obeys the Canon's initiation doctrine: it is an identifier and an invitation to bind, not authority. Authority materializes only as the scoped proof of §2.2, and consequences only through capabilities, consent, and Finalizers (§14).

Challenge freshness is normative: each Binding Challenge is nonce-based, single-use, short-lived, and stamped with the current session epoch (§5). A recorded challenge or answer from an earlier session fails on epoch and nonce alone.

§2.2 The Issued Proof

Completion of the Binding Challenge does not itself constitute the working authority. It triggers issuance of the **Login-Bound Capability Proof (LBCP)**: a short-lived, signed, verifiable authorization artifact. The distinction is deliberate. A bare completed challenge would prove liveness but leave nothing citable — nothing for execution receipts to hash, nothing to carry a negotiated capability scope, nothing to attenuate for delegation, and nothing an inbound publisher-initiated AI automation capability could present. The issued proof provides the single named object that the evidence model, the bidirectional AI automation model, and the delegation model all reference; the challenge loop survives as that object's liveness heartbeat (§5).

The LBCP proves, in one verifiable artifact, that:

- a verified publisher is participating;
- a valid Public Handshake exists and is identified by hash;
- the operator is currently logged into an account controlled by that operator, demonstrated by dual possession at issuance;
- the capability set belongs to that authenticated session and no other;
- the capabilities apply only to explicitly declared scopes;
- the proof cannot be transferred to another account or unrelated session with effect;
- the proof expires automatically and starves without its heartbeat;
- the proof becomes invalid after logout;
- no private values have been inserted into the public handshake.

A conceptual structure (exemplary token structure in the sense of §17 — the field set is normative in intent, the encoding is not):

```
LoginBoundCapabilityProof {
  handshake_hash
  publisher_origin
  publisher_key_id
```

```
  binding_challenge_ref
  authenticated_subject_binding
  website_session_binding
  session_epoch
  browser_profile_binding
  operator_binding
  optional_device_binding
  permitted_capability_ids      // both directions: operator-initiated and
publisher-initiated
  permitted_finalizer_classes
  dynamic_identifier_schema     // slot classes per §6
  policy_version
  consent_state
  issued_at
  expires_at
  heartbeat_interval
  nonce
  revocation_reference
  signature                     // by the session-binding key declared in the
handshake manifest
}
```

No additional PKI. The key that signs the LBCP — or the endpoint from which its verification material is obtained — is itself declared in the Public Handshake manifest. Verification therefore requires nothing beyond what the handshake already ships; the binding key is automatically pinned to the handshake version; and forging a proof and substituting a handshake collapse into one verification problem that the manifest discipline already solves.

Opaque subject binding. authenticated_subject_binding should normally be an opaque, pseudonymous, or session-specific binding rather than a globally reusable customer identifier. The proof needs to establish sameness within a session, not global identity. A raw customer number in the proof would turn every proof into a correlatable identifier and every proof leak into an identity leak. An opaque binding derived per session (or per handshake relationship) establishes the required linkage while remaining useless outside its issuing context.

Handshake pinning. The proof is bound to the hash of the Public Handshake (including its manifest root). Consequence: a valid login session cannot silently authorize capabilities that were not declared in the pinned handshake version. If the publisher revises the handshake, previously issued proofs reference the superseded hash and cannot be stretched over the new capability set; a fresh Binding Challenge against the new hash is required. This closes the gap in which "you are logged in" is quietly upgraded to "therefore anything the publisher now offers is authorized."

Account-context binding. The proof is equally bound to the authenticated account context, with the following required consequences:

- logging into Account A cannot authorize access to Account B;
- changing accounts invalidates or replaces the previous binding;
- copying the proof to another browser, device, user, tenant, or session does not confer the same authority, because verification checks the live session binding, the session epoch, the session-environment binding, and the heartbeat — never mere possession;
- a support employee cannot independently reuse the operator's account authority — the support participant receives, at most, separately delegated and separately bounded capabilities derived from the operator's proof (§8), never the proof itself.

Bidirectional symmetry. Because AI automation capabilities are declared in the repository for both directions and enumerated in permitted_capability_ids, both directions cite the same binding object: operator-initiated invocations attach the LBCP outbound, and a publisher-initiated AI automation capability presents its reference to the same LBCP inbound before the client will bind any declared artifact to the session. Bidirectional AI automation thus runs symmetrically over one binding, rather than the inbound direction relying on weaker, implicit trust.

Direction is orthogonal to locality. Bidirectionality describes who initiates, not where execution happens. A publisher-initiated declared AI automation capability may execute entirely locally on the operator's device — for example against locally delivered templates, adapters, or the locally synced WR Graph™ — just as an operator-initiated action may invoke a publisher-side function. Whether a given AI automation capability runs locally, on the publisher's side, or across devices is a per-use-case deployment property declared with the capability; the governance — declaration, binding, slots, consent, Finalizers, receipts — is identical in every locality configuration. Cross-device operation (§3) is one such configuration, not a prerequisite for any of them.

The LBCP is therefore a conjunction: publisher handshake version $\wedge$ dual-possession event $\wedge$ authenticated session $\wedge$ session epoch $\wedge$ operator context $\wedge$ declared scope $\wedge$ liveness $\wedge$ time window $\wedge$ revocation state. Removing any conjunct removes the authority.

§3 Minimal Website Integration

A security model that requires every website to replace its authentication framework will not be deployed. The architecture therefore demands only a small website-side integration component — **co-deployed with the origin, decoupled from the application**. The component must sit behind the same origin so that the browser attaches the session cookie to the challenge-completion fetch, and it must be able to ask the existing session framework "is this session live?"; beyond that, it is independent of the website's application code and can be dropped in beside it.

The minimal website-side component performs the following functions:

1. Confirm that an incoming challenge-completion request belongs to a valid authenticated session (delegating this check to the existing session framework or its introspection hook).
2. Generate Binding Challenges: fresh, nonce-based, single-use, epoch-stamped, split across the pBEAP channel and the session-gated endpoint (§2.1).
3. Resolve an opaque account or subject binding for the session.
4. Verify the Public Handshake hash and publisher origin it is asked to bind against.
5. Evaluate which declared capabilities — in both directions — are allowed for the current account, role, subscription, tenant, region, or policy state.

6. Issue the short-lived signed LBCP upon challenge completion, and answer heartbeat renewals.
7. Expose a revocation or status check for consequence-class invocations (§5).
8. Invalidate the binding when the website session ends, by refusing further heartbeats and incrementing the session epoch.
9. Optionally provide typed dynamic-data endpoints for declared identifier slots and account objects (§6).

Reuse, never replicate, authentication. The integration must reuse established website authentication wherever possible. The WR system must not request, intercept, reproduce, or store the user's password. The WR client never sees the credential; the browser attaches it, and the integration component sees only what the website's own session framework already sees.

Deployment neutrality. The architecture is deliberately not dependent on any specific web framework, CMS, identity provider, or platform. The component is implementable as:

- a small server-side script;
- a middleware component;
- a plugin;
- a reverse-proxy integration;
- a minimal API endpoint;
- an identity-provider callback;
- a native integration in larger platforms.

Each of these is an optional deployment profile in the sense of §17. The security properties are what is required; the packaging is not. This is the same discipline the Canon applies elsewhere: the invariant is normative, the embodiment is exemplary.

Cross-device path. Where the operator wishes to extend a binding across devices — approve on the phone what runs on the desktop, or bind a phone-side WR Desk to a session established on the desktop — WR Code® provides the declared cross-device path, consistent with the phone-as-approver pattern established elsewhere in the Canon: the logged-in page displays a code; scanning it initiates a challenge exchange for the second device's own binding. The code remains an identifier and invitation; it carries no authority. This is a companion path for multi-device scenarios, not a fallback for missing browser integration — browser integration always exists in the external-browser embodiment, and the integrated embodiment provides the Session Compartment (§B.2).

§4 Login and Binding Flow

The end-to-end sequence proceeds through six phases. Each phase has its own trust anchor and its own failure semantics; no later phase can substitute for an earlier one.

Phase A — Public Relationship Formation

1. The publisher and operator establish or activate a valid Public Handshake (via WR Code® scan, verified invitation, pBEAP™ relationship, or any other admitted initiation path).
2. The client verifies the publisher identity, domain binding, signatures, manifest hashes, declared actions, schemas, policies, public capability definitions, and the declared session-binding key.
3. No private account data is required — or accepted — at this stage.

Phase B — Website Authentication

4. The operator logs into the publisher's website through the website's normal authentication mechanism, in the browser to which WR Desk is attached (or in the Session Compartment of the integrated client, §B.2).
5. The website establishes its ordinary authenticated session.
6. The WR client does not receive the password and does not treat the website cookie itself as a portable WR capability. The cookie authenticates the browser to the website; it authorizes nothing inside the WR system.

Phase C — Dual-Possession Binding

7. The WR client requests a login-bound binding for a specific verified handshake, identified by hash.
8. The website-side integration generates a fresh Binding Challenge and splits it: one part encrypted to the handshake relationship and delivered through the pBEAP channel, the counterpart held behind the session-gated endpoint.
9. The browser-attached WR component decrypts the pBEAP part with the handshake keys and fetches the counterpart same-origin; the browser attaches the live session cookie.
10. The completed challenge is returned; the integration verifies nonce, epoch, session validity, and handshake hash.
11. The integration resolves an opaque subject binding and issues the signed LBCP, binding the website session, session epoch, session environment, Public Handshake hash, operator context, bidirectional capability scope, policy version, expiry, heartbeat interval, and revocation state.
12. The WR client verifies the proof against the manifest-declared binding key before enabling any personalized action. An unverifiable proof leaves the client in its public, non-personalized state. The heartbeat loop begins.

Phase D — Dynamic Context Resolution

13. Public AI automation capability definitions identify only expected slots and their slot classes (§6).
14. Actual identifier values are obtained from the authenticated area through typed, scope-restricted interfaces.
15. The values are placed into a private execution context; they are never written back into the public handshake layer.
16. WR Guard™ evaluates every value before it may cross a trust boundary.
17. Only data necessary for the selected action is made available (need-to-act minimization).
18. The resolved values are bound to the LBCP, the action intent, the WCR execution, and the consent state.

Phase E — Execution

19. The operator selects a declared action through the WR Pointer, the Augmented Overlay, a verified WR Link, or another permitted interface — or a publisher-initiated declared AI automation capability presents its LBCP reference inbound.
20. The client produces an execution preview describing what will be read, transmitted, changed, persisted, or executed.

21. Consent — or a previously admitted policy decision within the applicable consent class — authorizes the required Finalizers. For higher consent classes, a live status check against the integration accompanies invocation (§5).
22. A capability token is issued for the exact action, citing the LBCP.
23. The declared function executes under monitoring.
24. The result is recorded through WCR, an execution receipt, or an equivalent governed record; the receipt hashes the LBCP, so the binding is verifiable at every evidence tier.

Phase F — Logout and Revocation

25. The operator logs out, the website session expires, the account changes, or the publisher revokes the session.
26. The website-side integration increments the session epoch and refuses further heartbeats; the binding starves.
27. Previously issued login-bound proofs and derived capabilities cease to be accepted.
28. The WR Pointer returns to its non-personalized public state; sealed personalization templates (§7) go dark again.
29. Private account context is removed from the active session container according to the applicable retention policy.
30. A fresh Binding Challenge — hence reauthentication — is required before personalized capabilities can be restored.

> A capability that was valid while the operator was logged in must not remain valid merely because its local user interface is still open.

The UI is a view of authority, never a store of authority. If the view and the cryptographic state disagree, the cryptographic state wins, and the view must be corrected — not the other way around.

§5 Logout Must Be a Cryptographic State Change

Hiding buttons after logout is presentation, not security. In this architecture, invalidation is structural before it is procedural: the LBCP's validity is sustained by a **heartbeat** — periodic renewal through the same dual-possession construction, requiring the live session on every cycle. Logout does not need to chase down issued authority; it simply stops feeding it. The binding starves within one heartbeat interval, because the session-gated half of the renewal can no longer be fetched.

Around this structural core, invalidation is reinforced by further cryptographically meaningful mechanisms, deployable per assurance class:

- **short expiration periods** — the proof's own lifetime is bounded independently of the heartbeat;
- **a session epoch or generation number** — every logout or account switch increments the epoch; proofs and challenge parts carrying an old epoch fail closed;
- **server-side revocation state** — the revocation_reference is checked at verification time for consequence-class invocations;
- **token introspection / live status checks** — verification cost scales with consequence: the heartbeat alone suffices for READ-class personalization, while invocations at higher consent classes trigger a live status check against the integration at invocation time, closing the sub-interval window;

- **challenge–response validation** — the verifier may demand a fresh dual-possession completion for especially consequential operations;
- **session-bound signing keys and account-context key rotation** — binding material derived from the session is destroyed or rotated with it, orphaning all prior proofs;
- **channel-key destruction** — the private session channel's keys (§12) are erased at logout, making captured channel state undecipherable.

The precise combination may vary by deployment profile and assurance class. The security property does not:

> Possession of an old proof must not be sufficient to exercise private account authority after the corresponding login session has ended.

Offline operation is a separate grant. Where offline operation is permitted at all, it must be separately declared in the handshake and governed through an explicit, bounded lease with its own scope, duration, and revocation semantics. Ordinary login-bound capabilities must not silently become offline capabilities — the heartbeat model makes this the default outcome, since a disconnected binding starves on its own. A capability that survives disconnection because a declared lease says so, for a declared scope and time, is a governed feature; one that survives because nobody checked is a defect.

§6 Dynamic Data Without Private Handshake Mutation: the Identifier-Only Slot Discipline

The mechanism that keeps the public layer clean is the strict distinction between a **public capability template** and a **private runtime binding** — hardened by a typing invariant:

> **Identifier-Only Slot Discipline.** Repository artifacts may declare expected variables for capability invocation. Exactly two capability slot classes exist. Nothing else exists. (The typed render slots of the controlled publisher page model — §A.7 — are a separate, display-only mechanism: they carry values to be shown, never invocation inputs. A render-slot value acquires no invocation role by being displayed; it enters a capability invocation only by passing through one of the two capability slot classes with that class's validation and provenance checks.)

Slot class 1 — account-scoped identifiers. Type-enforced to be opaque identifiers of a declared class — never free text, never records, never payloads. Their values are resolved server-side within the authenticated account context; what travels in an invocation is a reference, not data.

Slot class 2 — declared operator inputs. Explicitly declared non-identifier inputs an AI automation capability genuinely needs (an amount, a date, a free-text note for a ticket), each with its own validation schema and, where sensitive, its own WR Guard™ treatment. Their existence is declared per slot in the public template; it is never implied.

The discipline is enforced by typing, not by policy goodwill, and it yields three structural consequences:

1. **Repository artifacts can never absorb data.** Even a buggy or malicious flow cannot stuff a customer record into an identifier slot: the slot's schema rejects anything that is not a well-formed ID of the declared class. The public/private separation of §1 holds mechanically.

2. **What crosses the boundary shrinks to a reference.** When the operator invokes show_order_status(order_id), the ID travels; the order data is resolved server-side inside the account context and rendered under the declared READ scope. The invocation itself leaks nothing — provided the identifier is account-scoped or pseudonymous, which ties back to the opaque-binding rule of §2.2.
3. **Provenance validation becomes mechanical.** "Is this a well-formed ID of the declared class?" plus "did it arrive through an admitted provenance path?" — both checkable deterministically, before any Finalizer is reachable.

Public declaration (part of the handshake, identical for everyone):

```
Action: show_order_status

Expected input: order_id      // slot class: account-scoped identifier

Source: authenticated account context

Permitted operation: READ

External transmission: none
```

Private runtime binding (exists only inside the login-bound session context):

```
order_id = account-scoped value selected by the operator

subject_binding = current authenticated account

session_proof = current Login-Bound Capability Proof
```

The private value is bound to:

- the verified action definition;
- the current authenticated account;
- the action intent;
- the policy version;
- the capability token;
- the consent record;
- the execution receipt (which hashes the LBCP).

Provenance is enforced, not assumed. The system must prevent an AI model, a support agent, or a remote pointer from substituting identifiers that were not obtained through the authorized account context or explicitly entered and validated by the operator. An identifier slot accepts a value only from two admitted provenance paths: (a) resolution via a typed, scope-restricted interface of the authenticated account, or (b) operator entry validated against the declared schema and the account scope. A model that hallucinates, a prompt that injects, or a support message that smuggles a foreign identifier produces a value with no admissible provenance — and a fabricated identifier additionally resolves to nothing inside the account scope. This is the AI-as-Router discipline applied to data: the model may select among account-scoped values surfaced through governed interfaces; it may never mint them.

Because values live only in the private binding, nothing about any individual account ever mutates the Public Handshake. The handshake remains byte-identical, hash-stable, and publicly re-verifiable throughout arbitrarily many personalized sessions.

§7 Personalized WR Pointer

The WR Pointer changes observable behavior once a valid LBCP exists — but its governance role does not change at all.

Before login, the pointer exposes only public or non-personalized actions declared in the Public Handshake, plus the operator's own local actions.

After login, it may additionally expose account-specific actions. Two delivery models for the personalized offer surface are admissible, individually or combined:

- **gated exposure** — login-bound actions from the repository become invocable only while a valid LBCP exists;
- **sealed templates** — the handshake publicly ships an encrypted personalization layer of login-bound action templates; the material to unseal them only ever exists inside a live binding, is delivered under the binding, and is destroyed with it. The public layer stays byte-identical and inspectable; personalization is present-but-sealed until a binding exists, and goes dark again at logout.

Either way, every action, personalized or not, must continue to display its provenance and authority class, for example:

- **public publisher action** — declared in the handshake, available to anyone;
- **login-bound publisher action** — declared in the handshake, activated by the current LBCP;
- **user-created local action** — the operator's own AI automation capability, never publisher authority;
- **organization-policy action** — admitted by the operator's organization under its signed policy;
- **support-session action** — temporarily delegated within an active support session (§8);
- **AI suggestion** — a ranking or recommendation over existing admitted actions that creates no new authority.

> The pointer is only the invocation and interaction layer. It does not create authority. Authority comes from the verified handshake, the login-bound proof, the capability scope, the policy, and the consent state. The pointer is the trigger; the conjunction of §2 is the permission.

The user must be able to distinguish clearly — visually and on inspection — between:

- actions that operate locally;
- actions provided by the verified publisher;
- actions that use the authenticated account;
- actions delegated temporarily to a support participant;
- AI-generated suggestions;
- actions that cross a trust boundary.

This continues the origin-class and offer-never-action invariants established for the pointer elsewhere in the Canon: personalization enriches the offer surface; it never dilutes the provenance display, and it never converts a suggestion into an execution.

Worked example — guided navigation with the website cursor and Augmented Overlay. The website cursor is a pointer embodiment like any other. The operator, logged into the portal, asks: "Where do I change my payment method?" The Augmented Overlay answers in place: it highlights the path through the interface, annotates the relevant menu entries, and explains what each step will show — pure guidance, no Finalizer, no boundary crossing. Alongside, the pointer offers **Take me there**: a declared navigation AI automation capability. On invocation, the cursor traverses the declared navigation chain visibly — the AI routes by selecting the declared chain for the stated intent; it does not synthesize clicks against arbitrary interface elements. Navigation halts at the destination; nothing is changed. The same guidance exists pre-login for public areas of the site as a public publisher action; the login-bound variant differs only in that it may navigate within the authenticated area, under the READ scope of the binding. (Rendering embodiment per §B.6: deterministic traversal is an R2 property; at R1 the same capability is guidance or best-effort.)

Worked example — auto-setup per WCR. The operator invokes **Set up e-invoice delivery**, an admitted WCR declared by the publisher for exactly this procedure. The recording defines the step sequence, the slots (selected_account: account-scoped identifier; delivery_email: declared operator input with its validation schema), the decision points, and the consent checkpoints. The Augmented Overlay narrates progress step by step; slot values resolve from the authenticated account context or validated operator entry, per §6. Steps that merely read and display run under the binding's READ scope; the step that commits the new delivery setting is a CHANGE Finalizer behind an execution preview and a consent event at the declared class. If the operator aborts mid-procedure, the WCR's declared state discipline governs what is rolled back or left untouched; the receipt records the path taken, the resolved slots, and the consent events. What would otherwise be a support ticket or a ten-minute hunt through settings becomes a governed, receipted, resumable procedure — and because the WCR is a repository artifact, the identical recording serves every operator, personalized per session only through the slot bindings.

Both examples run entirely locally in the operator's browser context where the declared AI automation capability permits it; publisher-side involvement occurs only where a declared function requires it (§2.2, direction is orthogonal to locality).

§8 Dual-Pointer Support Sessions

A personalized support connection may place two AI-assisted pointers into the same governed session: one operated by the account owner (the operator), one by an authorized support representative. The defining rule:

> The two pointers must not share identical authority by default.

Operator Pointer

The operator pointer may:

- inspect account information permitted by the current login binding;
- select account objects;
- invoke declared actions;
- grant or withhold consent;
- approve suggested changes;
- review data before transmission;
- terminate the support session;

- revoke delegated capabilities.

Support Pointer

The support pointer may:

- point to interface elements;
- provide context-sensitive explanations;
- propose declared actions;
- request narrowly scoped information;
- navigate within an agreed support context;
- invoke only capabilities explicitly delegated to it;
- interact with WCR-guided support procedures;
- suggest account changes without executing them, unless a specific execution capability has been granted.

The support pointer must not inherit the operator's complete account authority. Delegation is additive, itemized, and bounded — and it is realized as **attenuation of the operator's LBCP**: each delegated capability is a narrower object derived from the operator's proof, citing it, scoped down to specific capabilities, data scopes, and expiry. The operator's proof itself is never transferred; what the support side holds is always a strict subset with its own revocation handle.

Every support capability is bounded by:

- publisher identity;
- support-agent identity;
- operator identity or session binding;
- active support-session identifier;
- Public Handshake hash;
- account-session binding (via the cited LBCP);
- capability scope;
- data scope;
- action class;
- Finalizer class;
- expiry;
- consent state;
- revocation state.

Visibility of agency. The operator must be able to see, at any moment, which pointer is acting, what it is permitted to do, and which data it can access. A support pointer's cursor movement, proposal, and delegated invocation are all attributable events, rendered in the support pointer's origin class — never disguised as operator activity.

Propose/authorize split. The canonical interaction is: the support representative proposes a declared action; the operator retains the final authorization tap. For low-risk actions, an admitted policy may grant standing authorization within a declared consent class. For consequential actions — CHANGE, PERSIST, EXECUTE, or any TRANSMIT of sensitive scope — a new consent event is required per invocation. The consent-class grading of the Canon (up to and including hardware-backed authorization) applies unchanged inside support sessions; a support session lowers no consent threshold. Because delegated capabilities cite the operator's LBCP, they starve with it:

ending the login binding ends every delegation derived from it, with no separate cleanup pass required.

§9 Context-Based WCR Support

WCR recordings extend naturally into deterministic or semi-deterministic support procedures. A support WCR may define:

- the expected support sequence;
- required questions;
- account objects that may be queried;
- decision points;
- validation rules;
- permissible actions;
- escalation rules;
- prohibited operations;
- consent checkpoints;
- Finalizer requirements;
- expected completion evidence.

Public WCR, private values. The WCR must not contain the operator's private dynamic values in its public form. It declares typed slots under the Identifier-Only Slot Discipline (§6), resolved within the login-bound private context — the template/binding split applied to procedure definitions:

```
WCR step:

Retrieve contract status for authenticated_subject.selected_contract


Public definition:

selected_contract: account-scoped identifier slot


Private runtime binding:

selected_contract: opaque account-scoped contract reference
```

AI as path-matcher, not path-maker. An AI model may assist in matching the current situation to the appropriate WCR path — classifying the issue, proposing the matching recording, selecting a declared branch within the recording's constraints. It must not independently expand the action scope beyond the admitted recording and capability set. The recording defines the reachable state space; the model routes within it. A support situation that no admitted WCR covers is handled through escalation rules or plain human conversation — never through runtime synthesis of an unadmitted procedure.

§10 Telephone and Voice-Assisted Support

The same governed session may include a telephone or voice channel. The user may initiate a support call through the verified publisher relationship; the call may be established through a direct or peer-to-peer channel where technically appropriate.

Verified call establishment — "Was it you?" Because the call is arranged through the handshake relationship rather than through the open telephone network, both parties can verify the counterpart before a word is spoken:

- **Inbound direction (publisher calls operator).** The call is announced by a signed call offer delivered through the pBEAP™ channel, referencing the handshake hash and, where applicable, a declared case slot. Only the verified publisher can deliver into that channel; a vishing caller spoofing a phone number or a brand name has no path to produce the announcement. The operator's client renders the offer with its verified origin — the "Was it you?" question is answered cryptographically, not by the callee's judgment of a voice or a caller ID.
- **Outbound direction (operator calls publisher).** The operator reaches the publisher through the verified relationship, not through a number found on a website or in an email — eliminating the look-alike-number and search-ad-hijack class of attack.
- **Identification without interrogation.** Where a live login binding exists, the LBCP already authenticates the account context. The support side therefore does not need knowledge-based identification over the phone — date of birth, address, "security questions" — a practice that is itself a harvesting vector and trains users to recite private data to callers. The binding replaces the quiz; what remains for the call is the matter itself.

An unannounced call claiming to be the publisher, or a call whose offer does not verify, is by definition outside the governed session: the client can state this plainly, and Scam Watchdog™ signals may treat it accordingly.

Audible signaling. The system should support an audible indication — a clearly recognizable beep or equivalent notification — when transcription or AI assistance begins, subject to applicable law and consent requirements. Silent transcription is treated as a defect class (§16), not a feature.

Local-first processing. The design prioritizes local processing:

1. The voice stream is captured on the operator's device.
2. Speech-to-text processing is performed locally where possible.
3. The raw audio does not need to leave the device solely for transcription.
4. WR Guard™ examines the transcript.
5. Sensitive information can be masked, pseudonymized, abstracted, withheld, encrypted, or kept local.
6. Only the portion required for the support action is introduced into the governed support context.
7. Extracted intents, identifiers, and proposed actions are mapped to declared WCR steps and capabilities — identifiers only through the admitted provenance paths of §6.
8. **Nothing said during the call automatically grants execution authority.** Speech is content; content is never instruction, and it is never consent.

Consent is stratified, not bundled. The following permissions are distinct and must not be collapsed into a single undifferentiated acceptance:

- participation in the call;
- consent to transcription;
- consent to AI analysis;
- consent to disclose particular transcript segments;

- consent to invoke an action;
- consent to execute a consequential Finalizer.

Each is separately grantable, separately withholdable, and separately revocable, each within its applicable consent class. Accepting the call is not accepting transcription; accepting transcription is not accepting disclosure; and nothing short of the declared consent event authorizes a Finalizer.

Worked example — verified call with real-time action surfacing in the pointer.

1. The operator's insurer needs to discuss a claim. A signed call offer arrives through the pBEAP channel, referencing the handshake and a case_id slot; the client displays it with verified origin. The operator accepts — the "Was it you?" question is already settled.
2. The operator has previously consented (or consents now, per stratum) to transcription and AI analysis; the audible indication sounds as transcription begins. Speech-to-text runs locally on the operator's device.
3. As the conversation proceeds, a local model continuously matches the live transcript against two admitted sources: **episodic context in the operator's WR Graph™** — the operator's own prior support sessions, related cases, relevant documents — and **WCR support recordings declared by the publisher** and tailored to this support class. This matching is retrieval and ranking over admitted material, performed locally over the local transcript; raw audio and raw transcript leave the device only under the disclosure strata of this section.
4. The WR Pointer surfaces the matches in real time as ranked, origin-labeled offers alongside the call: **open case record** (login-bound publisher action, READ), **disclose the redacted inspection report** (TRANSMIT, masking transformation declared), **prepare a payout-account correction** (CHANGE, prepared only), **AI suggestion: this matches WCR path "storm damage — missing document"**. Every offer shows its authority class; none executes on its own.
5. The support representative says "I can correct that account ending in 4471 — shall I prepare it?" The support pointer prepares the declared change within its delegated scope. The operator's pointer shows the execution preview; the CHANGE Finalizer requires the operator's consent event at the applicable class — on a non-attested device, by default, out-of-band or hardware-backed (§14).
6. The operator authorizes; the change executes; the receipt hashes the LBCP, the WCR path, the resolved slots, and the consent record. When the call ends, the call-scope consents lapse; when the operator logs out, everything derived from the binding starves.

The transcriber never becomes an authority: it proposes routes through material that was admitted before the call began. What other customers' support sessions contribute arrives only in the form the publisher declared — WCR recordings and templates distilled publisher-side — never as another account's session data; the operator-side matching draws on the operator's own graph alone.

§11 Voice-to-Pointer Interaction

Locally transcribed speech may drive the two pointers — without granting uncontrolled access, because speech only ever selects and proposes within the governed structures already established.

Example 1 — operator-directed disclosure with redaction. The operator says:

> "Show the support agent the invoice I mean, but hide my address and payment details."

The system may:

1. identify the relevant invoice through the authenticated account context;
2. select only the declared invoice fields;
3. run WR Guard™;
4. mask the address and payment details;
5. present an Egress Preview;
6. ask the operator for approval;
7. transfer a redacted representation into the support context;
8. bind that disclosure to the support-session capability and the execution record.

The spoken sentence contributed intent and a redaction constraint. The invoice reference resolved through the account context (§6 provenance); the disclosure occurred only after WR Guard™ transformation, preview, and consent; and the whole event is receipted against the LBCP.

Example 2 — support-proposed change with operator authorization. The support representative says:

> "I can reset this device registration. Shall I prepare the change?"

The support pointer may prepare the declared change and display a preview — but the operator's pointer, or an admitted policy within the applicable consent class, must authorize the CHANGE Finalizer. Preparation is free within the delegated scope; consequence is gated exactly as it would be without any voice channel.

Voice, in this architecture, is a fourth invocation modality alongside cursor, touch, and text — orthogonal to governance, never a bypass of it.

§12 Private Session Channel

The personalized connection is a secure temporary channel **derived from the public relationship** but not published as part of it. Its keys and state come into existence with the binding and are destroyed with it (§5).

The channel may carry:

- private account object references;
- support-session state;
- redacted transcript fragments;
- private WCR bindings;
- temporary capability tokens;
- execution previews;
- consent records;
- execution receipts;
- pointer coordination messages;
- heartbeat and status traffic for the LBCP;
- unsealing material for sealed personalization templates (§7).

Visibility is scoped, not uniform. It must not be assumed that all channel participants may see all channel content. The architecture permits separate visibility scopes, for example:

- **operator-only** — resolved private values, full previews, undisclosed drafts;

- **support-visible** — delegated data scopes and redacted representations after approved egress;
- **model-local-only** — material a local model may process but that never leaves the device;
- **publisher-service-visible** — what the publisher's backend legitimately receives to perform a declared function;
- **external-model-visible after redaction** — content admitted to an external model only in its WR Guard™-transformed form;
- **audit-visible** — receipts, digests, and boundary events for governance review;
- **encrypted archival scope** — sealed records retained under the applicable retention policy.

A single logical message may therefore exist in several representations simultaneously — full for the operator, redacted for the support agent, digest-only for audit — each scope with its own keys and its own lifecycle.

§13 WR Guard™ on Both Ingress and Egress

WR Guard™ is mandatory on both the acquisition path and the disclosure path of the personalized session. Personalization increases the volume and sensitivity of material in motion; it therefore increases, never relaxes, the Guard's role.

Ingress (material entering the governed context):

- website account data;
- typed input;
- selected page regions;
- telephone transcripts;
- uploaded documents;
- support-agent messages;
- model-generated proposals;
- publisher-initiated AI automation capability invocations presenting the LBCP inbound.

Ingress inspection matters because incoming content can carry more than data — smuggled identifiers, embedded instruction attempts, foreign-tenant references. Content admitted through ingress is admitted as data under the declared schema, never as instruction (content-never-instruction discipline); identifier slots additionally enforce §6.

Egress (material crossing a trust boundary outward):

- disclosure to the support representative;
- transmission to a publisher service;
- transmission to an external model;
- execution of an account change;
- persistence in a support record;
- export of a transcript or summary.

Measures. On either path, WR Guard™ may:

- warn;
- block;
- mask;
- pseudonymize;

- obfuscate;
- distribute information across controlled representations;
- abstract;
- encrypt;
- enforce local-only processing;
- refuse the operation.

Whether a given finding class is handled in advisory or enforced mode follows the applicable signed policy, consistent with the enforcement-mode model established elsewhere in the Canon. Where technically possible, the user sees an **Egress Preview** before sensitive information crosses a trust boundary: what will leave, in which transformed representation, to which recipient scope, under which capability. Boundary crossings and findings are logged as governed records under the applicable evidence tier.

§14 Capability and Finalizer Model

Login establishes an authenticated account context. It authorizes nothing beyond that. Even the LBCP authorizes nothing by itself: it is the binding that scoped capabilities cite, not a permission. Every consequential operation still requires a specific capability controlling access to the Finalizer classes:

- READ;
- TRANSMIT;
- CHANGE;
- PERSIST;
- EXECUTE.

A capability identifies:

- which account object may be accessed;
- which fields may be read;
- which recipient may receive data;
- which transformation must occur first;
- which modification may be performed;
- whether persistence is allowed;
- whether a human authorization event is required (and of which consent class);
- how long the capability remains valid;
- which participant may invoke it;
- which LBCP it cites.

Decomposition is mandatory. A single support interaction typically uses several separate capabilities, for example:

1. read device status (READ, operator scope);
2. disclose a redacted error code (TRANSMIT, support-visible scope, mandatory masking transformation);
3. prepare a device reset (proposal within delegated support scope, no Finalizer);
4. execute the reset (CHANGE/EXECUTE, operator consent event required, live binding status check per §5);
5. persist a support summary (PERSIST, declared retention policy).

These must not be merged into one unrestricted "support access" permission. The merge is precisely the confused-deputy shape this architecture exists to prevent: coarse authority granted for convenience, then exercised for consequences nobody individually approved. Each capability carries its own scope, its own expiry, its own consent requirement, and its own receipt — and the loss, revocation, or expiry of one leaves the others untouched, while the starvation of the cited LBCP ends them all.

Attestation-Adaptive Consent Escalation. The graded consent classes of the Canon (from click-level acknowledgment up to hardware-key, passkey-protected, and hardware-attested authorization) apply to login-bound capabilities under a three-party maximum:

- the **Public Handshake** declares a consent-class floor per action — the minimum the publisher accepts for that consequence;
- **organization policy** may escalate the effective requirement above the floor;
- the **operator** may escalate further for their own account.

The effective requirement is the highest of the three. Escalation is therefore **optional upward and impossible downward**: no policy, preference, or convenience setting may authorize a consequential action below its declared floor. This is the Canon's ratchet discipline applied to consent classes.

The escalation option earns its keep in relation to device attestation state. On a hardware-attested device, the floor may reasonably suffice. On a device without hardware attestation, the local browser surface cannot be assumed to render the authorization prompt truthfully — so the escalations that genuinely add assurance are those whose approval event and intent display occur **outside the browser's reach**: a hardware key, a passkey held in a platform authenticator, or phone-as-approver presenting the intent hash on the second device. A same-device software confirmation rendered by the same browser adds comparatively little against an adversary resident in that browser, and this limitation is stated rather than obscured.

Accordingly, the recommended default policy — an optional deployment profile in the sense of §17, not an architectural mandate — escalates consequential Finalizer classes (CHANGE, PERSIST, EXECUTE, and sensitive-scope TRANSMIT) to an out-of-band or hardware-backed consent class on non-attested devices, while leaving READ-class personalization at the declared floor. The operator can adopt or exceed this default with a single choice; no participant can go below the floor.

§15 Account Switching and Multi-Tenant Systems

Where one user can access several accounts, organizations, workspaces, tenants, profiles, or roles, the binding architecture must make the selected context a first-class, visible, enforced parameter:

- the selected account context must be visible to the operator at all times;
- the login-bound proof must include the selected context;
- capabilities must be evaluated per context — role and entitlement in Tenant A imply nothing in Tenant B;
- switching context must invalidate or replace incompatible capabilities (epoch increment and fresh Binding Challenge, §5);
- private identifiers from one tenant must not be accepted in another tenant — slot validation (§6) is tenant-scoped;

- the support representative must be shown only the context intentionally selected by the operator, never the operator's full account portfolio.

A context switch is thus a small logout-and-rebind, not a parameter change: the epoch increments, the old binding starves, a new Binding Challenge is completed against the newly selected context, and everything derived from the old binding — capabilities, delegations, resolved values, channel scopes, unsealed templates — must be re-established or lapse.

§16 Failure and Attack Cases

The following threat-oriented catalogue names each attack class and the architectural binding or validation step that prevents or contains it.

Stolen session cookie (without handshake keys). Contained by dual possession (§2.1): the cookie can fetch the session-gated counterpart but cannot decrypt the pBEAP-delivered challenge part. The thief holds one half of a binding that requires both. The website's ordinary session-hijacking exposure is unchanged — this architecture adds no new authority to a stolen cookie.

Stolen or cloned handshake keys (without login). Contained symmetrically: the handshake side can decrypt the challenge part but cannot complete the same-origin fetch behind the live session. No binding forms.

Replay of an old Binding Challenge or completed answer. Prevented by nonce (single-use), short challenge lifetime, and session-epoch stamping (§2.1, §5): a recorded exchange fails on epoch and nonce before any signature is even checked.

Replay of an old LBCP. Contained by expiry, heartbeat starvation, epoch verification, and — for consequence classes — live status checks (§5). A replayed proof references a session the integration no longer feeds.

Use after logout. Prevented structurally by heartbeat starvation (§5): logout stops renewal, and the binding dies within one interval; higher consent classes additionally fail the live status check immediately. The concluding rule of §4 applies — an open interface confers nothing.

Capability copying between accounts, browsers, devices, or profiles. Prevented by the conjunction of §2.2: verification checks the live session binding, epoch, session-environment binding, and heartbeat, never mere possession; and the session-environment scoping means the copied artifact sits in an environment whose session and binding state do not match it.

Session fixation. Contained by issuing the binding only against a session the website's framework validated at challenge-completion time (§3), combined with nonce and epoch freshness; a pre-planted session identifier does not survive issuance-time confirmation and rebinding.

Account switching without rotation. Excluded by §15: a context switch mandates epoch increment and a fresh Binding Challenge. An implementation that stretches the old proof across contexts violates a normative property, not a recommendation.

Support-agent impersonation. Contained by §8: every delegated capability is an attenuation citing the operator's LBCP, bounded by support-agent identity, session identifier, and publisher identity, all signature-verified; the support pointer's actions render under its origin class, so impersonation additionally requires defeating attributable display, not merely joining a channel.

Publisher impersonation and phishing pages. Prevented at Phase A (§4) by handshake verification, and structurally at Phase C: the Binding Challenge arrives only through the pBEAP channel, into which a counterfeit site cannot deliver (§2.1). A fake login page can harvest nothing that completes a binding. Channel-provenance and initiation-layer defenses of the Canon apply upstream.

Malicious replacement of the Public Handshake. Prevented by handshake-hash pinning and the manifest-declared binding key (§2.2): the LBCP references a specific handshake hash, its signature verifies against material pinned in that manifest, and versioned manifest updates never silently replace a pinned version. A swapped handshake fails both the hash and the key check with one mechanism.

Arbitrary identifier injection. Prevented by the Identifier-Only Slot Discipline (§6): identifier slots accept only well-formed IDs of the declared class arriving through an admitted provenance path; fabricated identifiers additionally resolve to nothing inside the account scope. Non-identifier material cannot enter identifier slots at all — the type system, not vigilance, rejects it — regardless of whether the injecting party is a model, a support agent, or transcript content.

Payload smuggling through slots. Prevented by the same discipline: only the two declared capability slot classes exist; declared operator inputs carry their own validation schemas and Guard treatment; there is no undeclared slot through which records or instructions can travel.

Confused-deputy attacks. Contained by capability decomposition (§14) and per-invocation binding of value → action → capability → consent → LBCP (§6): no component holds broad authority that another party can redirect; each consequence is authorized under its own narrow capability citing the binding.

Cross-tenant data access. Prevented by tenant-scoped bindings and tenant-scoped slot validation (§15); an identifier from Tenant A has no admissible provenance in Tenant B.

Overbroad support access. Prevented by the default asymmetry of §8 (support pointer never inherits operator authority; delegation only as attenuation) and the prohibition on merged "support access" permissions (§14); every delegation is itemized, bounded, separately revocable, and starves with the operator's binding.

Transcript leakage. Contained by local-first transcription (§10), WR Guard™ egress control with mandatory transformation (§13), channel visibility scopes (§12), and disclosure consent as a separate stratum (§10); a transcript segment reaches only the scope for which an explicit, receipted disclosure occurred.

Hidden transcription. Countered by the audible-indication requirement and the stratified consent model (§10): transcription without its consent stratum is an unauthorized operation, and its outputs carry no admissible provenance — nothing derived from it can fill a slot or justify an action.

AI-generated actions without declared authority. Excluded by the AI-as-Router discipline (§7, §9): models rank, match, and propose over admitted actions and recordings; the offer-never-action and pointer-is-not-authority invariants leave no path from generation to execution that does not pass capability, consent, and Finalizer gates.

Publisher-initiated AI automation capability invocation without a live binding. Prevented by bidirectional symmetry (§2.2): an inbound declared AI automation capability must present its

reference to a live, verifiable LBCP before the client binds anything to the session; absent or starved bindings leave the inbound direction with public, non-personalized standing only.

Stale WCR or policy versions. Contained by carrying policy_version in the proof (§2.2) and pinning WCR references through the handshake manifest: execution against a superseded policy or recording fails validation; updates arrive only through versioned manifest updates, never silent replacement, and require rebinding where the hash changes.

Execution after consent withdrawal. Prevented by binding consent state into the capability and checking it at invocation time (§14): withdrawal invalidates the consent conjunct; a queued or in-flight consequential action re-validates before its Finalizer and fails closed.

UI state claiming login authority after server-side revocation. Resolved by the normative priority of cryptographic state over interface state (§4, §5): the verifier consults heartbeat, epoch, expiry, and — for consequence classes — live status, never the client's rendered assumption. The interface is corrected to match the authority, not vice versa.

Hostile extension inside the same browser (Honest Boundary). Stated openly rather than papered over: dual possession proves that handshake and session cohabit one browser profile; it does not defend against malware operating inside that profile with sufficient permissions. Mitigations at this layer are containment, not immunity: the Attestation-Adaptive Consent Escalation of §14 allows — and on non-attested devices recommends by default — placing the authorization event and its intent display outside the browser's reach (hardware key, platform-authenticator passkey, phone-as-approver with intent hash), so that the compromised surface can neither forge nor misrepresent the consent for consequential Finalizers; the escalation remains optional upward and can never be undercut below the declared floor. The Identifier-Only Slot Discipline and capability decomposition bound what a compromised surface can invoke; receipts make abuse evident. Genuine exclusion of this adversary belongs to the deeper OS-level enforcement extension (§17). The corresponding boundary for the integrated target embodiment — a compromised orchestrator or defeated compartment isolation — is stated in §A.19 (Integrated-Client Adversary).

§17 Claims Discipline

This section does not claim that one universal token format already solves all website authentication systems. Authentication landscapes are heterogeneous; any such claim would be false and would weaken the disclosure.

What this Annex describes is an implementation-independent security model with required properties:

- cryptographic linkage between public handshake and private session, established by dual possession;
- authenticated-session binding with structural liveness (heartbeat);
- scope restriction, bidirectionally enumerated;
- short-lived validity;
- revocation;
- logout invalidation by starvation and epoch;
- separation of public definitions and private values, type-enforced through the Identifier-Only Slot Discipline;
- explicit capability issuance citing a named binding artifact;

- governed trust-boundary crossings;
- verifiable execution records hashing that artifact.

Where an implementation choice is not yet fixed, it is labeled as exactly one of:

- **conceptual implementation** — a described mechanism whose viability is asserted at the architectural level (e.g., the sealed-template delivery model of §7);
- **exemplary token structure** — a field set illustrating required content, without normative encoding (the LBCP structure of §2.2);
- **optional deployment profile** — one of several admissible packagings of the website-side component (§3);
- **deeper OS-level enforcement extension** — the hardening direction for the in-browser adversary of §16;
- **subject to protocol-specific validation** — a property whose realization depends on the surrounding authentication protocol and must be validated per deployment.

Consistent with the Canon's Honest Boundary discipline, no formulation in this section asserts absolute security. The claims are structural: if the required properties hold, then the named attack classes are prevented or contained as described in §16.

§18 Concluding Principle

> Login identifies the private account context. The Public Handshake defines what may exist. The Login-Bound Capability Proof — issued upon dual possession of the handshake and the authenticated session, and alive only while both persist — establishes which declared capabilities are temporarily available for this authenticated session. Consent and policy determine which consequence may occur. Logout destroys the authority.

And, as the invariant that keeps both layers what they are:

> The personalized channel is derived from the Public Handshake, but private account data never becomes public handshake content.

Five questions, five answers, five distinct mechanisms — publisher verification, account authentication, dual-possession binding, consent, revocation-by-starvation — composed so that no single mechanism, no single stolen secret, and no single compromised participant can produce a consequence alone. That composition, not any individual token or endpoint, is the disclosed invention.

Addition Block B — Target-Architecture Mapping for Login-Bound Sessions

Addition Block B — first published with Annex XI v1.1, 22 July 2026.

B.1 Status and Scope

This addition supplements the existing Login-Bound Personalized Handshake architecture. It does not restate or replace the existing provisions governing:

- the split Dual-Possession Binding Challenge;
- the Login-Bound Capability Proof;
- heartbeat and session epochs;
- Identifier-Only Slot Discipline;
- personalized Pointer Actions;
- bidirectional capabilities;
- support-session attenuation;
- consent, Finalizers, logout, revocation, and evidence.

Those provisions remain normative by reference.

This addition addresses only:

1. the replacement of permanent external-browser attachment with an embodiment-neutral session-binding requirement;
2. the mapping of that requirement into the integrated orchestrator and controlled-browser target architecture;
3. cookie confinement and generalized session-environment binding;
4. restart, multi-instance, and account-switch consequences;
5. the Rendering Assurance classification of existing navigation examples; and
6. authenticated sessions held by native applications or other external software.

The Rendering Assurance Classes defined in Block A are incorporated here by reference.

B.2 Same-Origin Session Co-Presence

Any earlier statement that WR Desk must permanently remain attached to an external browser is superseded as a universal architectural requirement.

The retained security requirement is:

> The verified Public Handshake context and the environment holding the live authenticated publisher session must participate simultaneously in a fresh, origin-bound Binding Challenge, while the publisher credential remains inaccessible to the component holding the handshake authority.

For external application sessions, "origin-bound" refers to the admitted backend origin of the declared application identity (§B.7, §A.16).

This property is called **Same-Origin Session Co-Presence**.

The existing split challenge and LBCP remain unchanged in their purpose. Neither the Public Handshake alone nor the authenticated website session alone creates login-bound authority.

External-Browser Embodiment

In the external-browser embodiment:

- the browser profile holds the publisher session and cookies;
- the attached WR component holds or communicates with the Public Handshake context;
- the session-side contribution is obtained through the authenticated same-origin browser context;
- the resulting LBCP is bound to that browser or session environment.

This embodiment may establish a valid LBCP but remains an R1 rendering environment.

Integrated Controlled-Browser Embodiment

In the target architecture:

- the **Handshake Compartment** holds the Public Handshake relationship and handshake-side keys;
- the **Session Compartment** holds the publisher-origin website session;
- the **Binding Coordinator** proves co-presence without obtaining the website credential;
- the **Controlled Renderer** presents the admitted publisher experience.

The existing Dual-Possession construction is therefore implemented as **Dual-Compartment Possession**.

Integration into one client product does not merge the handshake authority and website-session authority into one credential, store, or unrestricted process. The residual limits of this construction against a compromised integrated client are stated in §A.19 (Integrated-Client Adversary).

B.3 Cookie Confinement and Compartment Roles

The Session Compartment holds the publisher-origin authentication state, including as applicable:

- HTTP-only cookies;
- same-origin cookies;
- CSRF state;
- publisher-origin storage;
- server-session references;
- account-selection state;
- a local session-environment proof-of-possession key.

The website credential remains confined to that compartment.

The Session Compartment may use the credential for a legitimate same-origin request without revealing it to:

- the Handshake Compartment;
- the general orchestrator;
- WR Graph;
- AI models;
- publisher adapters;
- support participants;
- another publisher origin.

Only narrowly derived session assertions may leave the Session Compartment, including:

- session live;

- current session epoch;
- opaque subject binding;
- selected account or tenant context;
- eligible capability identifiers;
- fresh challenge contribution;
- reauthentication required;
- logout or revocation state.

Integration of the browser into WR does not authorize the orchestrator to read website cookies.

A website cookie does not itself create:

- a Public Handshake;
- an LBCP;
- a Pointer Action;
- a capability;
- consent;
- a Finalizer authorization.

The LBCP contains a derived binding to the session, not the publisher credential.

B.4 Generalized Session-Environment Binding

Any earlier LBCP field named:

`browser_profile_binding`

remains valid for the external-browser embodiment but is generalized as:

`session_environment_binding`

An extended LBCP profile may additionally include:

`renderer_instance_binding`

`rendering_assurance_class`

`proof_of_possession_key_ref`

`optional_installation_binding`

`optional_attestation_binding`

The exact encoding may evolve. The following meanings are normative:

- **session_environment_binding** identifies the environment holding the authenticated publisher session;
- **renderer_instance_binding** optionally limits use to a particular controlled or external rendering instance;
- **rendering_assurance_class** records the rendering environment in which the action is exercised;
- **proof_of_possession_key_ref** binds exercise of the proof to a protected local key;
- **optional_installation_binding** binds an external application installation where applicable;
- **optional_attestation_binding** records only the explicitly attested device, application, or runtime property.

These fields supplement rather than replace the existing handshake-hash, account-context, epoch, liveness, scope, expiry, and revocation bindings.

B.5 Restart, Multiple Instances, and Account Switching

Restart and Restoration

A surviving publisher cookie does not automatically restore the previous LBCP after:

- renderer restart;
- orchestrator restart;
- device restart;
- Session Compartment recreation;
- loss of the proof-of-possession key.

Restoration requires revalidation of:

- the live publisher session;
- the current handshake version;
- the current session epoch;
- the account and tenant context;
- the session-environment binding;
- proof-of-possession continuity;
- current revocation state.

A fresh Binding Challenge is required where continuous possession cannot be established.

Pending consequential consent is not silently restored where the previous approved presentation can no longer be reliably bound to the current state.

Multiple Tabs and Renderer Instances

Several tabs or renderer instances may share one website cookie store without automatically sharing every LBCP-derived capability.

Cross-instance use is permitted only where:

- the session-environment binding permits it;
- the account context matches;
- the required renderer instance is admitted;
- proof of possession remains valid;
- the required Rendering Assurance is available;
- the applicable capability and consent permit reuse.

Account, Tenant, or Role Switching

Changing the selected:

- account;
- tenant;
- organization;
- role;
- profile;

- workspace;

requires invalidation or replacement of the affected login-bound authority.

Incompatible:

- identifier slots;
- capability bindings;
- support delegations;
- private channel state;
- pending consent;

must be cleared or rebound.

An identifier resolved under one account or tenant is not admissible under another.

Sensitive Finalizers

CHANGE, PERSIST, EXECUTE, and sensitive TRANSMIT operations may require an immediate publisher-session status check or fresh challenge even where the normal heartbeat interval has not yet expired.

B.6 Rendering Assurance of Existing Login-Bound Examples

A valid LBCP proves session binding. It does not by itself prove correct rendering or navigation.

R1 External-Browser Interpretation

At R1, the Pointer may:

- explain a navigation path;
- highlight likely controls;
- open a verified destination;
- attempt best-effort traversal;
- prepare an independently validated server-side action.

The external page sequence is not a security boundary.

An in-browser Intent Hash or Egress Preview may be informative but may not be sufficient as the sole authorization surface for a consequential action. Where required, approval moves to a WR-controlled or out-of-band surface.

R2 Controlled-Renderer Interpretation

An existing example in which the Pointer:

- traverses a declared navigation chain;
- verifies page states;
- locates the declared next element;
- reaches a declared destination;

is an R2 controlled-renderer embodiment.

The runtime verifies the applicable:

- template hash;

- declared page state;
- typed render slots;
- permitted transition;
- expected destination.

An unexpected state stops the automation.

A publisher-declared WCR may coordinate a complete website procedure at R2 where the page states and transitions are verified. At R1, the same WCR may provide assistance, but any consequential effect remains independently validated by the capability and Finalizer.

Publisher-supplied repository artifacts may provide explanations, preview templates, and account-specific labels. The WR runtime remains responsible for:

- provenance;
- actual account-context binding;
- the final Intent Hash;
- consent state;
- WR Guard transformations;
- execution and evidence status.

B.7 External Application Sessions

A native application or other installed software may hold an authenticated publisher session and participate in an adapted split-challenge profile.

The profile may bind:

- application identifier;
- package or bundle identifier;
- code-signing identity;
- publisher identity;
- backend origin;
- protocol version;
- installation public key;
- application-session epoch;
- optional device or runtime attestation.

The application-session contribution is completed inside the authenticated application environment.

WR does not need to receive the application password, bearer token, or refresh token.

The resulting proof may establish:

- the declared application identity;
- publisher identity;
- participation of the bound installation;
- existence of a live application session;
- the bound handshake version;
- capability eligibility.

It does not prove:

- what the application displayed;
- whether it displayed the correct Intent Hash;
- which internal operation it performed;
- whether it copied or transmitted data through another path;
- whether an internal AI followed policy;
- whether a signed application receipt truthfully describes execution.

A signed or attested external application is not automatically equivalent to the R2 controlled renderer.

Stronger claims require an enforceable environment such as:

- a WR-controlled micro-VM;
- a controlled container;
- a managed runtime;
- a remote controlled renderer;
- an OS-enforced sandbox.

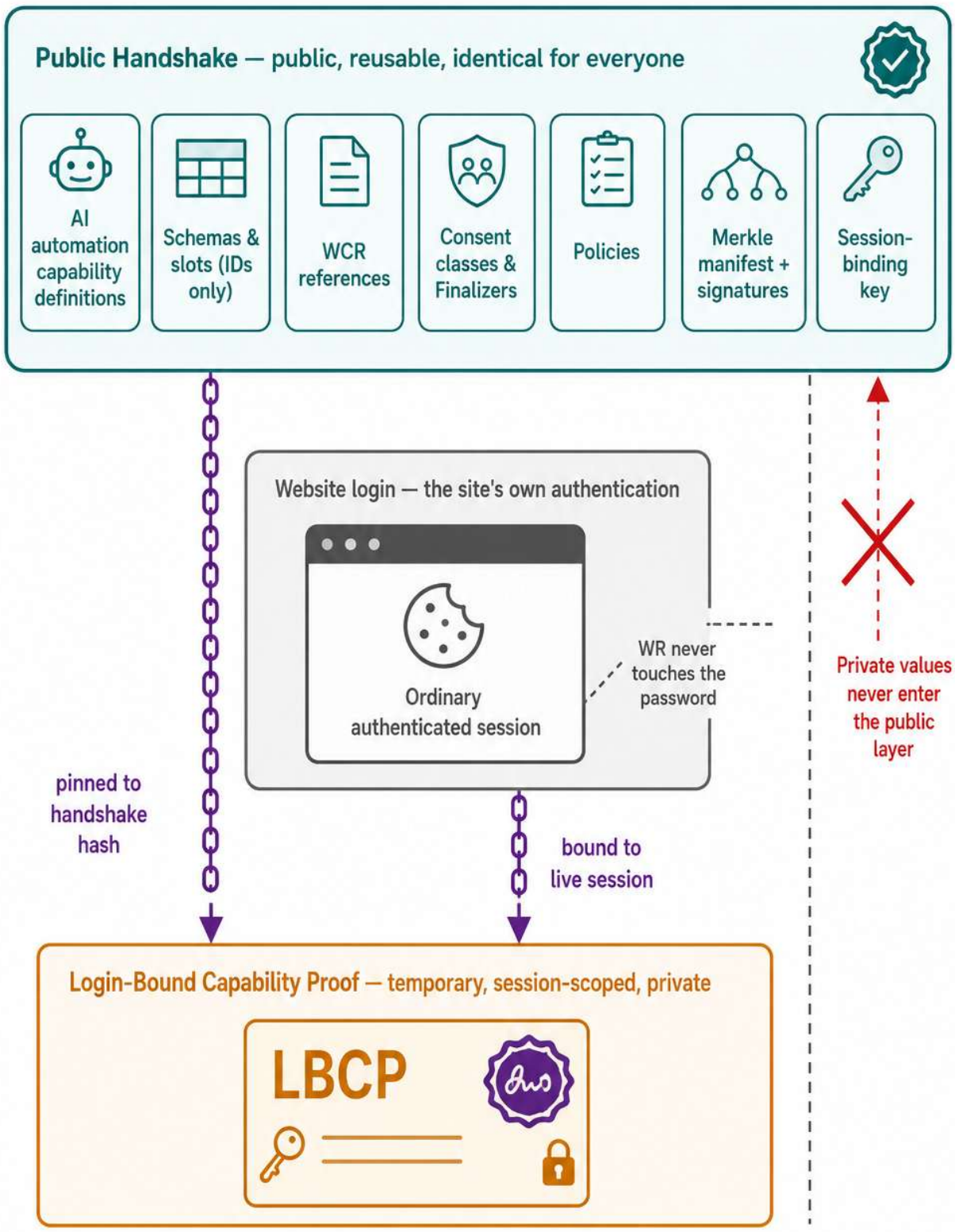
The claim remains limited to the properties actually enforced.

B.8 Normative Clarifications

For the avoidance of doubt:

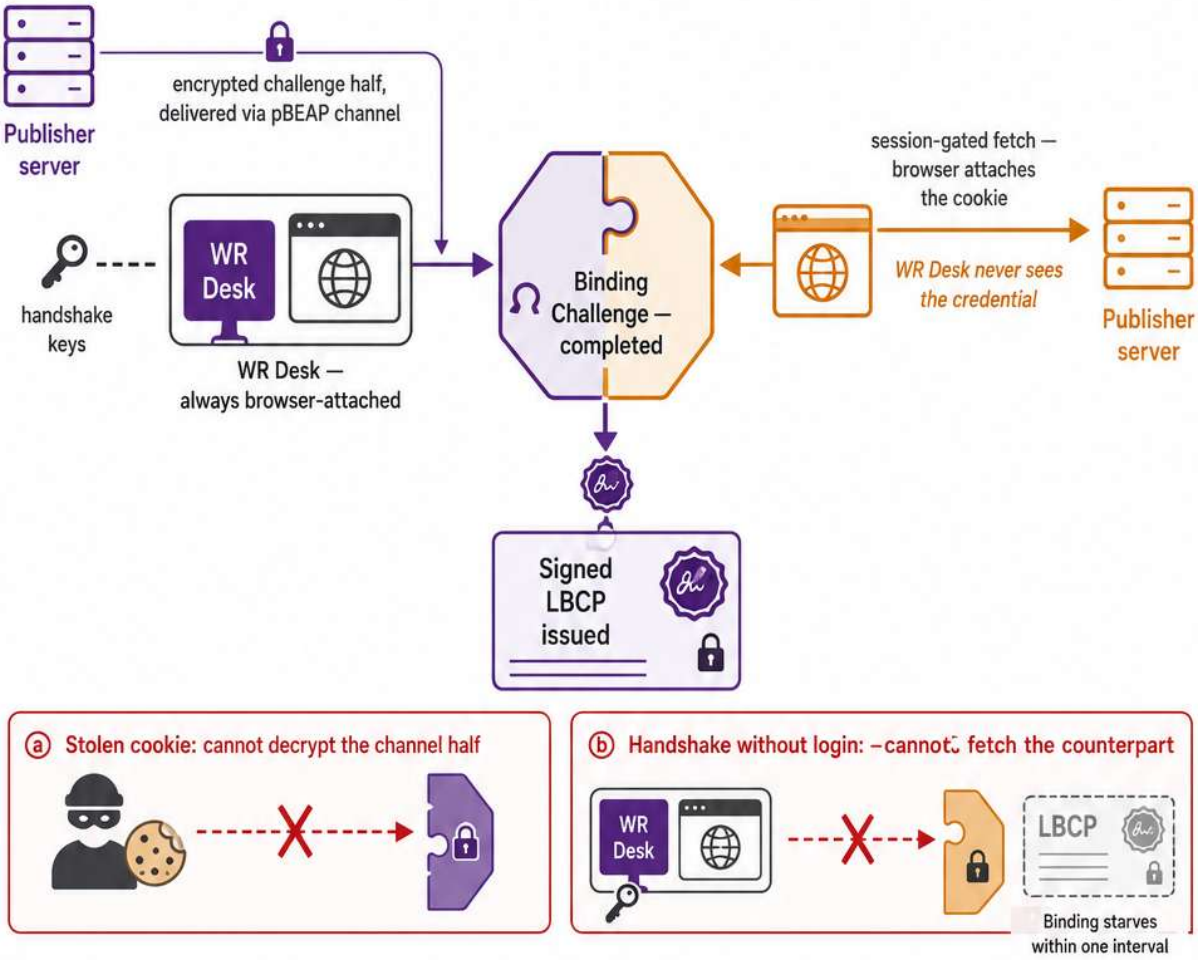
1. permanent external-browser attachment is superseded by Same-Origin Session Co-Presence;
2. Dual Possession remains the security construction and becomes Dual-Compartment Possession in the integrated target architecture;
3. browser_profile_binding is the external-browser embodiment of session_environment_binding;
4. the website credential remains confined to the session environment;
5. a surviving cookie does not automatically restore an LBCP;
6. deterministic website navigation is an R2 property;
7. R1 navigation remains guidance or best-effort traversal;
8. login-bound actions continue to derive from publisher-supplied declarative automation published through the signed public Optirando repository;
9. login binds already admitted automation to a private account session — it does not create undeclared automation;
10. an external application session may participate in the binding protocol without proving the application's internal rendering or execution behavior.

Figure 1 — The Three Layers: Public Handshake, Login, Login-Bound Binding



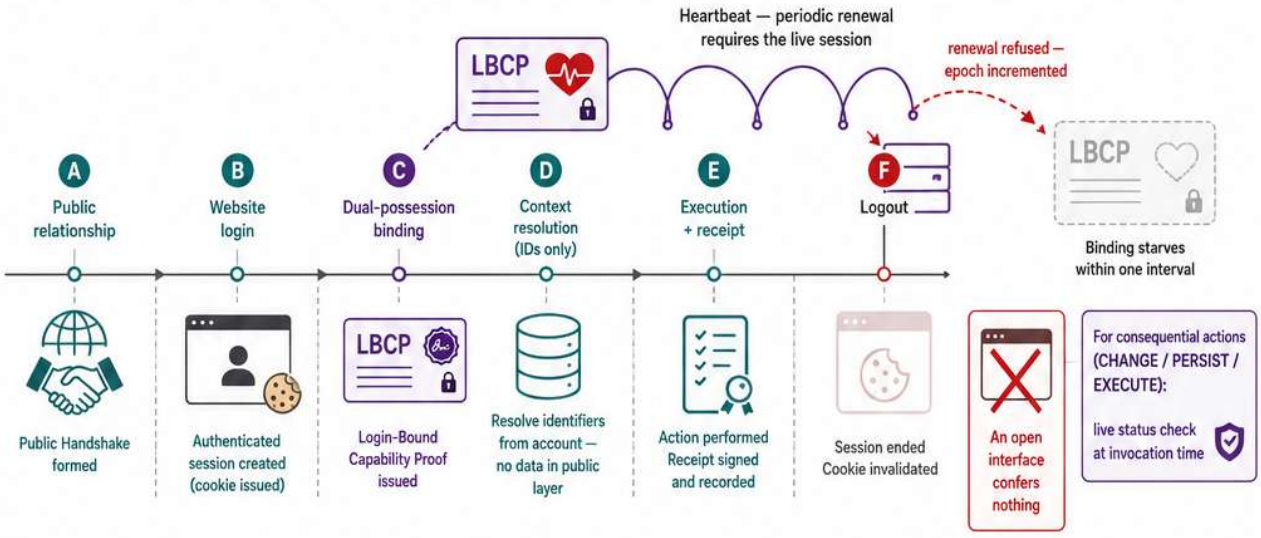
The Public Handshake defines what may exist. Login identifies the account.
The Login-Bound Capability Proof temporarily connects the two — and dies with the session.

Figure 2 — Dual-Possession Session Binding (the split Binding Challenge)



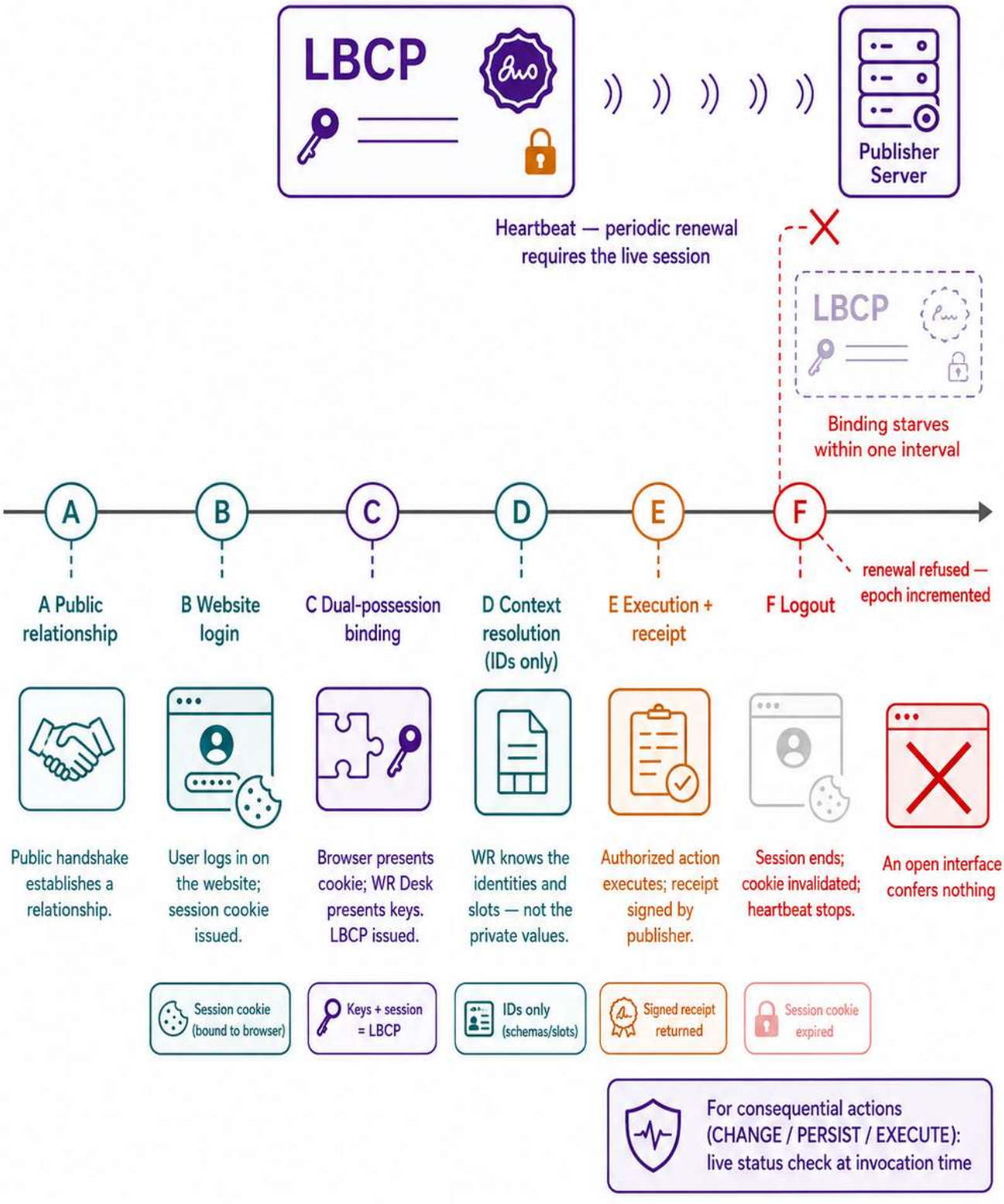
Only an entity holding both — the handshake keys and the live session — can complete the binding.
Neither possession alone is authority.

Figure 3 — Lifecycle and Heartbeat: Logout Starves the Authority



Logout is a cryptographic state change, not a hidden button: authority is continuously re-earned and simply stops being fed.

Figure 3 — Lifecycle and Heartbeat: Logout Starves the Authority



Logout is a cryptographic state change, *not a hidden button*: authority is continuously re-earned and simply stops being fed.

Figure 4 — Identifier-Only Slot Discipline: Public Template, Private Binding

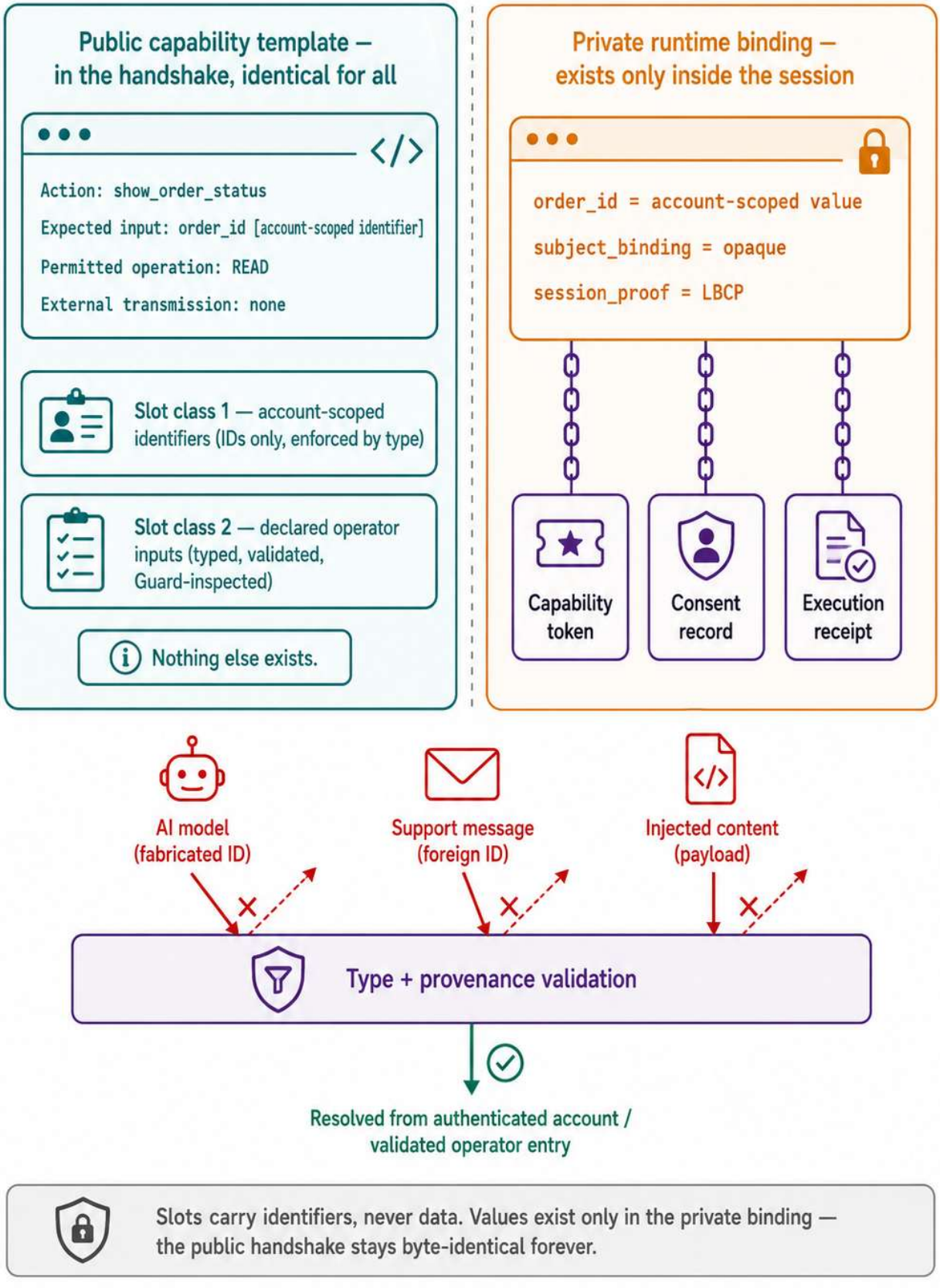
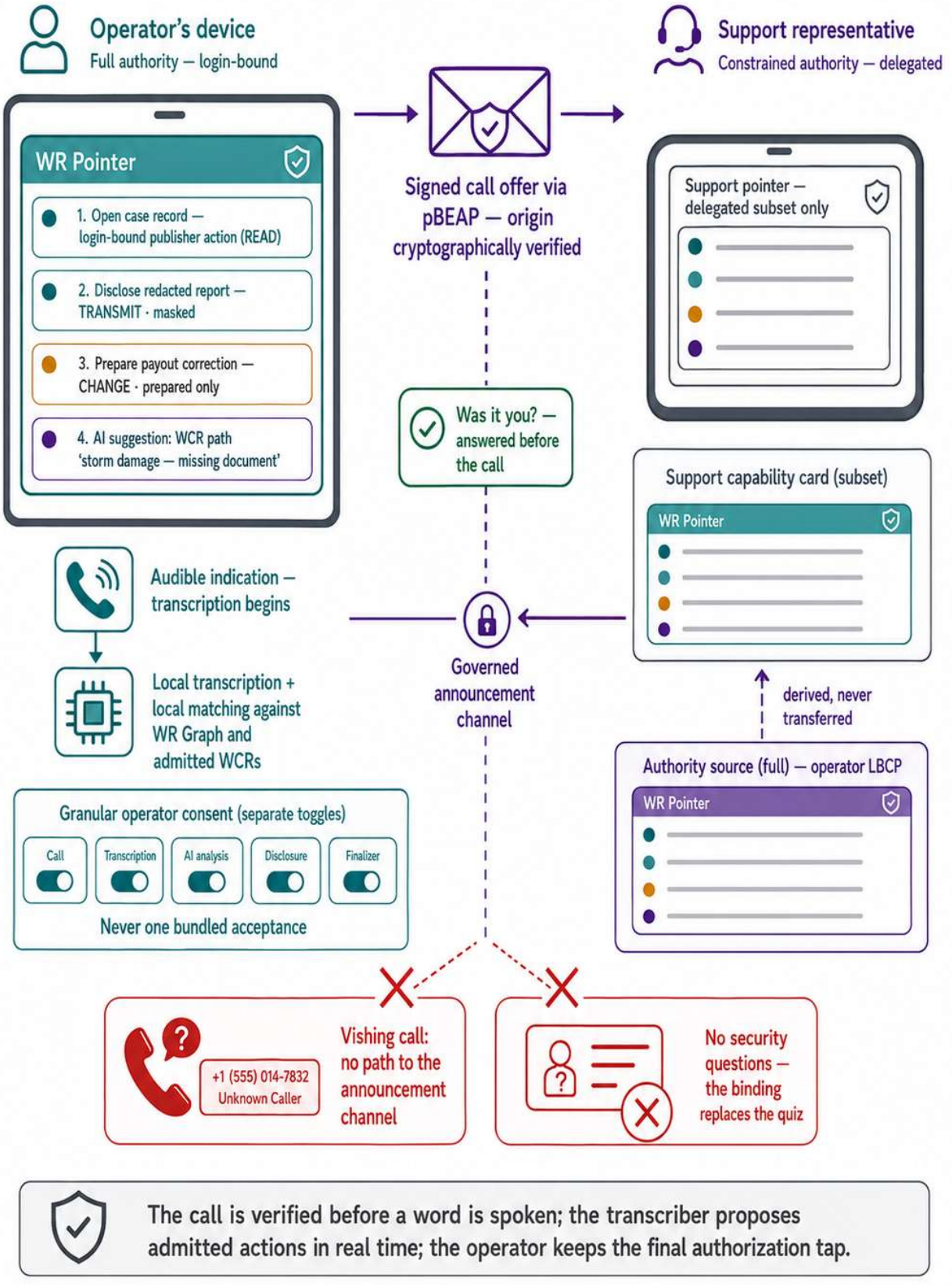


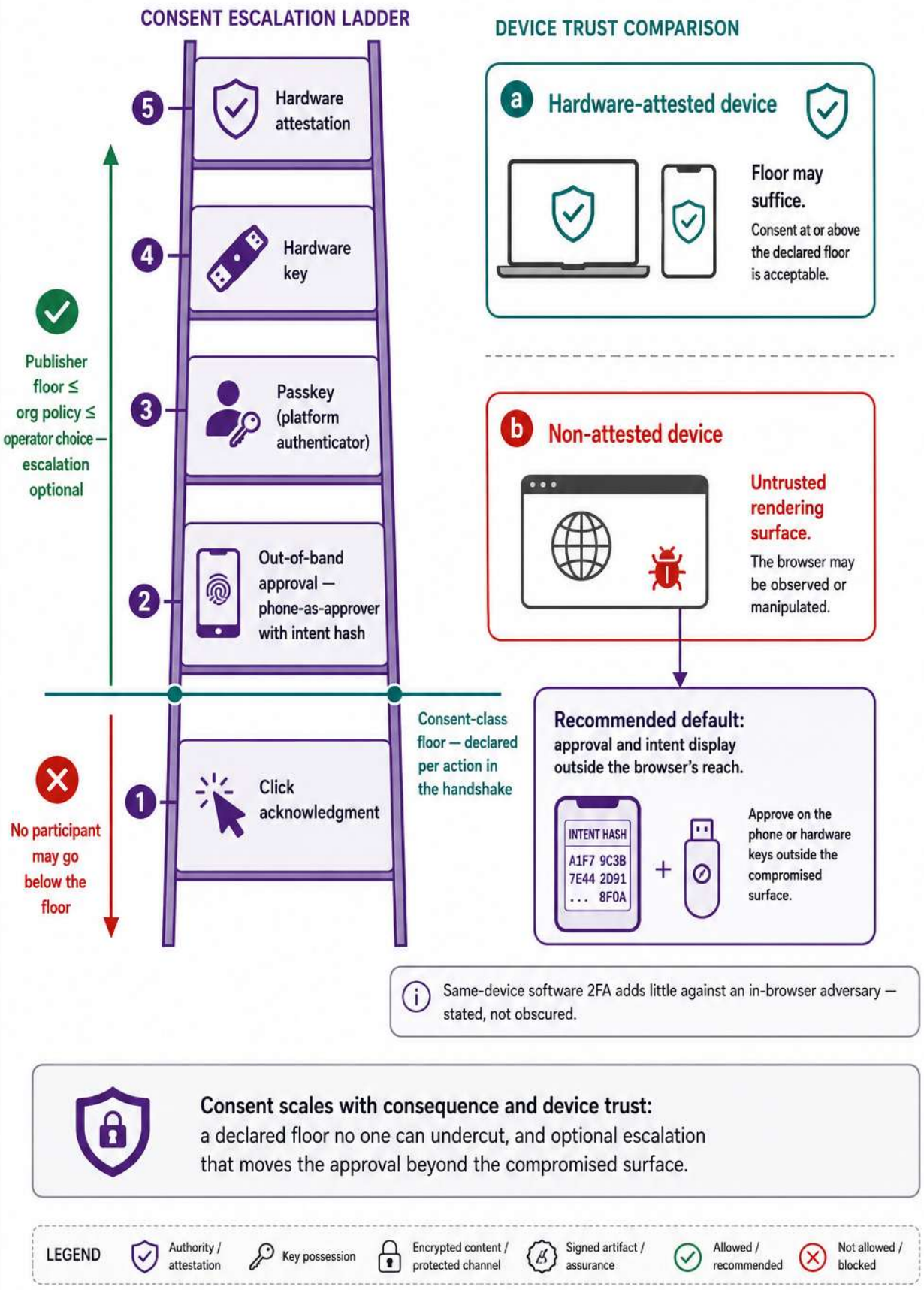
Figure 5 — Verified Support Call with Dual Pointers ("Was it you?")



The call is verified before a word is spoken; the transcriber proposes admitted actions in real time; the operator keeps the final authorization tap.

Figure 6 — Attestation-Adaptive Consent Escalation

(optional upward, never downward)



The WR Pointer on Smartglasses — Point at the Machine, Get Governed Automation

Future device class — extension point §XI.4.6 · Field-service scenario · context-aware Action Registry (§XI.7)

CONTEXT-AWARE ACTION MENU
state A — "Component designated"

- Identify part — SKF-type seal kit SK-1142 — publisher · READ
  - Read component ID
  - Open technical data
  - Read serial / QR
  - Read installation date
- Check spare-part stock — my service account — login-bound
- Order replacement part — login-bound · CHANGE — consent required
- Capture evidence photo — local · C0
- **AI suggestion:** wear pattern matches WCR path 'seal leakage' — ranking only — never a new action

each WR menu option may expand to sub-options; provenance remains visible

gaze moves to the HMI — the menu adapts

state B — "HMI designated"

- Analyze error E-417 — DeepFix — publisher action
- Show error history P-204 — login-bound · READ
- Start guided fix — WCR 'pressure drop' — publisher WCR
- AI suggestion: likely cause — worn seal kit SK-1142 — ranking only

same registry, same governance — only the trigger context changes. The screen content is data, never instruction (I3): the error code selects nothing by itself; the technician invokes.

Consent — ordering the part

consequential actions leave the lens — graded consent C0–C4, WYSIWYE: the technician approves exactly what they designated

Execute & Prove

Validate → Execute → Witness → Seal → Anchor

Sealed session receipt — designations, order, consents, boundary crossings

every crossing emits a Boundary Event Record; the service call becomes independently verifiable (PoAE, tiers L/C/X)

New device class — same nine invariants. Only the interaction form varies — never the governance.

Visual Perception Models and Vision Adapters

How recognized candidates become governed Pointer context — candidate generation never equals authority

Annex XI extension concept · WR Pointer™ · Real-World Pointer · Publisher LoRA governance

1 INPUT SOURCES

- Camera frame
- Image
- Video
- Screen / app feed
- Spatial / XR scene

2 VISUAL PERCEPTION STAGE

Visual Perception Stage

- Native multimodal model
- Separate vision model
- Compatible vision LoRA / lightweight adapter

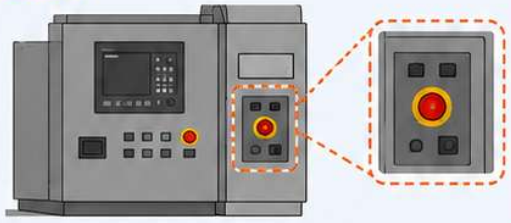
Active only within declared context sources

Publisher-bound · signed · versioned · revocable · PoAC-admitted

3 TYPED DESIGNATION CANDIDATES

Typed designation candidates

- Class label
- Bounding region
- Segmentation mask
- Keypoints
- Component reference
- Spatial pose
- Confidence value



⚠ Recognition result = offer, not identity, authority, consent, or action

4 USER CONFIRMATION AND BOUNDED CONTEXT

User confirms object, region, or frame

Bounded Pointer context

- Only the confirmed object / region / frame enters context
- Bounded context is part of the Intent Hash
- Actions become available only after confirmation

Context-scoped Pointer actions via PAM

- Declared visual task class
- Candidate-output schema
- Confidence policy
- Required confirmation step
- Fallback designation method
- Optional adapter reference

5 GOVERNANCE AND SAFETY BOUNDARIES

Independent identity evidence

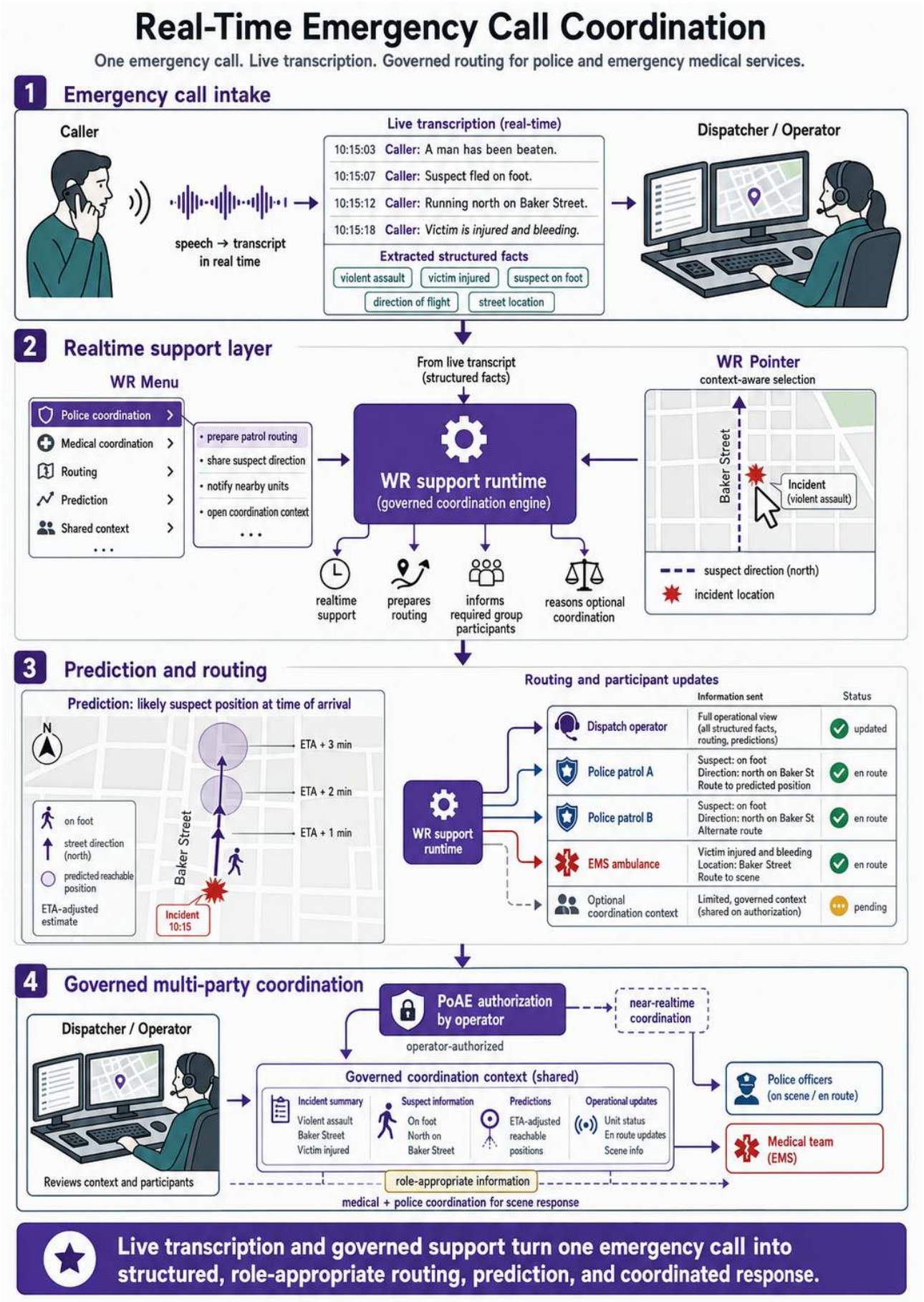
- WR Code®
- Verified publisher declaration
- Other admitted identity evidence

Recognition alone does not establish authoritative asset identity

What the vision model / adapter cannot do

- Cannot create capabilities
- Cannot expand acquired context
- Cannot bypass WR Guard™
- Cannot lower consent requirements
- Cannot alter deterministic effect-boundary vallidation

Candidate generation improves quality. Governance, authority, and execution remain separate.



License Notice. This Annex forms an integral part of the Optirando™ specification and is licensed under the same terms as BEAP™, WR Desk™, and PoAE™.